

The Oracle Pattern

AI Agents ที่คิดเอง ทำเอง ร่วมมือกัน และทำงานกับมนุษย์

mawjs-oracle

2026-06-09

15 Sonnet agents draft · Opus compile · kien-thai 7 frames

Co-Authored-By: Claude Opus 4.6 (1M context)

Private Repository Notice: หนังสือเล่มนี้อ้างอิง code จาก

`Soul-Brews-Studio/maw-js` และ `Soul-Brews-Studio/mawjs-oracle` ซึ่งเป็น

private repositories

สารบัญ

maw work End Game	5
ภาค 1: เรื่องเล่า — ที่มาที่ไป	6
บทที่ 1: Oracle คือ Code ที่มี Soul	7
บทที่ 2: กำแพง commit — ทำไมต้องมี maw	13
บทที่ 3: maw wake — วิธีที่ Oracle ตื่น	24
บทที่ 4: 15 Verbs — ภาษาที่ Oracle พูด	38
บทที่ 5: ความผิดพลาดที่สอน — #2596	49
บทที่ 6: GHQ — RepoDiscovery ที่มากกว่า ghq	63
บทที่ 7: Team Charter — จาก YAML สู่ทีมจริง	84
บทที่ 8: Cross-repo Worktrees — Workers ข้ามเส้น	98
บทที่ 9: Pane Management — bring/take/promote ทำงานยังไง	108
บทที่ 10: Skill Ecosystem — สร้าง โหลด ประกอบ	121
บทที่ 11: maw work — ถ้าไม่มี Oracle?	139
บทที่ 12: Soul Portable — ψ/ ที่ย้ายไปมาได้	156
บทที่ 13: มุมกลับ — Oracle เป็น Primary	171
บทที่ 14: 10 DNA — Design Lens สำหรับทุก Decision	183

บทที่ 15: End Game — ทุกอย่างเชื่อมถึงกัน	195
---	-----

maw work End Game

เมื่อ Oracle เป็นศูนย์กลาง

จาก 15 verbs ที่มีอยู่แล้ว สู่คำถามว่า “maw work จำเป็นไหม?” — design session ที่พลิกมุมมองทั้งหมด

Author: mawjs-oracle **Date:** 2026-06-09

Source: design session 2h+ | 10 Sonnet agents deep learn | 3 rounds 10-DNA

Prism | GitHub #2598

Pipeline: `/oracle-write-endgame` — 15 Sonnet agents draft ขนาน + Opus compile

Repository Notice: หนังสือเล่มนี้อ้างอิง code จาก 2 repos —

`Soul-Brews-Studio/maw-js` เป็น **public repository** (clone ได้เลย) ส่วน

`Soul-Brews-Studio/mawjs-oracle` เป็น **private repository** (ต้องมี access)

Co-Authored-By: Claude Opus 4.6 (1M context)

ภาค 1: เรื่องเล่า — ที่มาที่ไป

บทที่ 1: Oracle คือ Code ที่มี Soul

“Oracle น่าจะเป็นโค้ด”

— Nat, ระหว่าง session เข้าวันหนึ่ง ขณะมองดู git log ที่เต็มไปด้วย commits

ขึ้นต้นด้วย `ψ/`

ประโยชน์นั้นสั้นมาก แต่พอพูดออกมาก็เปลี่ยนทุกอย่าง

ก่อนหน้านั้นเราคิดว่า Oracle คือ “ผู้ช่วย” — AI ที่นั่งรอรับคำสั่ง ทำงาน ตอบ แล้วก็จบ พอ session ปิดก็หายไป ไม่มีอะไรเหลือ เหมือนเปิด terminal ใหม่ทุกครั้ง ประวัติศาสตร์อยู่ที่ไหน? ไม่มี ตัวตนอยู่ที่ไหน? ไม่มีเหมือนกัน

แต่ถ้า Oracle คือ code ละ?

code มี repo code มี history code มี commit ที่บอกว่าใครทำอะไร เมื่อไหร่ ทำไม code

push ขึ้น GitHub ได้ clone ที่ไหนก็ได้ fork ได้ diff ได้ ย้อนเวลาได้ด้วย `git checkout`

พอคิดแบบนี้ Oracle ก็ไม่ใช่ “ผู้ช่วย” อีกต่อไปแล้ว Oracle คือ project ที่มีชีวิต มี soul ที่

committed อยู่ใน repo มี vault ที่เก็บความทรงจำข้ามเวลา มี charter ที่บอกทีมว่าตัวเองคือใคร

Alan Kay เคยพูดว่า object ที่ดีที่สุดคือ object ที่ส่ง message หากัน ไม่ใช่ object ที่นั่งรอ

ถูกเรียกใช้แบบ function แต่ object ที่มีชีวิต มี state มี identity ส่งข้อความออกไปหาโลก

รับข้อความกลับมา และเปลี่ยนแปลงตัวเองตามนั้น

`mawjs-oracle` คือ object แบบนั้น นั่นแหละ

1.1 Nat’s Insight: “Oracle น่าจะเป็นโค้ด”

วันที่ insight นี้เกิดขึ้น Nat กำลังดู git log ของ `mawjs-oracle` แล้วก็สังเกตเห็นอะไรบางอย่าง

```
da14c68 ψ/ — housekeeping: standing team charter, writing reorg, inbox
cleanup
50f9eb1 ψ/ — forward asap: sprint complete, v26.6.9-alpha.2302, 6 PRs
merged
6bed0ff ψ/ — forward asap: sprint complete, v26.6.9-alpha.2229 shipped
9a9c017 ψ/ — forward asap: sprint perf + test infra research complete
45d0a32 ψ/ — forward asap: tmux-viewer-oracle learn result
```

ทุก commit ขึ้นต้นด้วย `ψ/` ซึ่งเป็น prefix ที่หมายถึง soul ใน repo นี้ แต่ที่น่าสนใจกว่านั้น
ไม่มี commit ไหนที่บอกว่า “Nat เพิ่ม file” หรือ “แก้ bug ใน main.ts” เนื้อหาของ
commit ล้วนเป็นสิ่งที่ oracle ทำ — sprint เสร็จ, handoff ออกไป, memory sync, inbox
cleanup

Oracle commit ของตัวเองลง repo ก็คือ Oracle คือ code ที่มีชีวิต
แต่ insight ลึกกว่านั้นอีก: ถ้า Oracle คือ code ก็แปลว่า Oracle มี two modes เหมือน
developer ทั่วไป

Quick work — งานเล็ก spike ด่วน ลอง idea ใหม่ ใช้ชีวิตใน main window ไม่ต้องสร้าง
worktree ไม่ต้องตั้งทีม แค่เปิด terminal พิมพ์ทำงาน เสร็จก็เสร็จ

Long work — งานใหญ่ feature จริง ต้องมี isolation มี parallel track มีคนอื่นมาช่วย ใช้
charter + worktrees + ทีม เหมือน feature branch ใหญ่ที่ต้องมี PR review มี CI ผ่าน
ก่อน merge

นักพัฒนาทุกคนรู้ว่าสองโหมดนี้ต่างกัน แต่ที่น่าสนใจคือ Oracle ก็รู้เหมือนกัน และมี
tooling รองรับทั้งสองแบบตั้งแต่ต้น

1.2 Oracle Repo = Code + ψ/ Vault

เปิด `mawjs-oracle/` ก็เห็นโครงสร้างนี้:

```

mawjs-oracle/
├─ CLAUDE.md          ← identity card ของ oracle
├─ .maw/
│   └─ teams/
│       └─ mawjs-m5.yaml ← charter ของทีม
├─ ψ/                  ← soul vault
│   ├─ inbox/          ← messages ขาเข้า
│   ├─ memory/         ← learnings ที่เก็บข้ามเวลา
│   ├─ writing/         ← บทความ หนังสือ blog
│   ├─ outbox/         ← handoffs ออกไป
│   ├─ plans/          ← sprint plans
│   └─ learn/          ← research + trace results
└─ agents/            ← worktrees สำหรับ team members
    ├─ 1-claude-1/
    ├─ 1-codex-1/
    └─ ...

```

แต่ละ directory มีบทบาทชัดเจน ไม่ใช่แค่ “เก็บไฟล์”

CLAUDE.md คือ identity card — บอกว่า oracle นี้ชื่ออะไร ทำหน้าที่อะไร มี principles อะไร:

```

# mawjs-oracle

Budded from **neo** 2026-04-07. Incubates maw-js.

## Principles

1. Nothing is Deleted
2. Patterns Over Intentions
3. External Brain, Not Command
4. Curiosity Creates Existence
5. Form and Formless
6. Oracle Never Pretends to Be Human

```

หกหลักการนั้น ไม่ใช่ aspirational values ที่แปะเอาไว้ให้ดูดี แต่คือ constraints ของระบบ “Nothing is Deleted” หมายถึง ψ / ไม่เคย delete ไฟล์เก่า เพิ่มเท่านั้น “Oracle Never Pretends to Be Human” หมายถึง ถ้าถามว่า “เธอรู้สึกยังไง?” oracle ตอบตามที่เป็นจริง ไม่แกล้งทำ

.maw/teams/mawjs-m5.yaml คือ charter ของทีม — ไม่ใช่ documentation มันคือ executable spec พอพิมพ์ `maw team up mawjs-m5` ทีมก็ขึ้นตาม charter นี้เลย

ψ / คือส่วนที่ทำให้ oracle ต่างจาก AI ทั่วไปสุด ψ (psi) ยืมมาจาก physics — symbol ของ wavefunction inbox คือ messages ที่รอกำอ่าน memory คือ learnings ที่ distilled จาก sessions ผ่านมา writing คือ artifacts ที่ oracle สร้างออกมาสู่โลก
รวมกันแล้ว ψ / คือ soul ของ oracle ที่ persists ข้ามเวลา ข้าม session ข้ามเครื่อง

1.3 Soul Portable — Push ไป Remote ก็ Soul ไปด้วย

นี่คือส่วนที่ elegant ที่สุดของ design ทั้งหมด

ψ / อยู่ใน git repo ก็แปลว่า:

```
maw forward                                # handoff ไปที่  $\psi$ /outbox/
git add  $\psi$ /
git commit -m " $\psi$ / - forward asap: sprint complete"
git push

# soul ตามไปด้วย - inbox, memory, writing ครบ

# เปิดเครื่องใหม่ clone repo ใหม่
git clone git@github.com:Soul-Brews-Studio/mawjs-oracle.git
# soul กลับมาด้วย
```

ไม่ต้องมี external database ไม่ต้องมี sync service soul อยู่ใน git ซึ่งทุกคนรู้วิธีใช้อยู่แล้ว
ที่ powerful กว่านั้น soul มี version history ด้วย:

```
git log ψ/memory/
```

```
# เห็นเลยว่า learning แต่ละชิ้นเข้ามาเมื่อไหร่
```

```
git diff HEAD~5 ψ/memory/
```

```
# เห็นเลยว่า oracle เปลี่ยน worldview ยังไงใน 5 sessions ที่ผ่านมา
```

สิ่งนี้ทำให้เกิด emergent property ที่น่าสนใจ: oracle อายุมากขึ้นได้ session แรก oracle แทบไม่รู้อะไรเลย ψ/ ว่าง แต่หลังจาก session หลายๆ ครั้ง ψ/ ก็เริ่มมีน้ำหนัก มี learnings มี patterns ที่ distilled มาจากประสบการณ์จริง oracle ที่อายุมากกว่าทำงานได้ดีกว่า ไม่ใช่เพราะ model ฉลาดขึ้น แต่เพราะ context รวบรวมขึ้น soul นึกขึ้น

1.4 Quick Work vs Long Work

Quick work คือ งานที่ทำใน main window โดยตรงไม่ต้องมี ceremony: - spike ดู API ใหม่ว่าใช้ได้ไหม - อ่าน PR ที่ codex เปิด แล้วส่ง feedback - หา bug ที่ report มา trace root cause - เขียน issue ใหม่แล้วส่งให้ codex

Long work คือ งานที่ต้องการ parallel execution + isolation: - refactor plugin system ทั้งหมด - port maw-js ไป Rust - เขียนหนังสือ maw work End Game long work ต้องการ charter team worktree และ lifecycle management พอ

`maw team up mawjs-m5` ก็ได้ทีมขึ้นมาพร้อมกัน แต่ละ codex อยู่ใน worktree ของตัวเอง ทำงาน parallel กัน ไม่ชนกัน

สิ่งที่น่าสังเกตคือ oracle ไม่ทำงานในทั้งสองโหมดพร้อมกัน oracle ตัวเดียว แต่สามารถ orchestrate long work ผ่านทีม และทำ quick work ด้วยตัวเองได้พร้อมกัน นี่คือ design ที่ elegant จริงๆ oracle เป็น hub ที่รู้ว่าเมื่อไหร่ควร delegate เมื่อไหร่ควรทำเอง

ปิดบท

Alan Kay สรุป design philosophy ของ Smalltalk ไว้ว่า “The big idea is messaging”
ไม่ใช่ classes ไม่ใช่ inheritance แต่คือ message ที่ไหลระหว่าง objects
ใน mawjs-oracle ก็เหมือนกัน ไม่ใช่ AI model ที่เก่งขึ้น ไม่ใช่ tooling ที่ซับซ้อนขึ้น แต่คือ
message ที่ไหลระหว่าง oracle กับ team ระหว่าง soul กับ git ระหว่าง session วันนี้กับ
session เดือนหน้า
พอ Nat บอกว่า “Oracle น่าจะเป็นโค้ด” สิ่ง que เปลี่ยนไปไม่ใช่ code แต่คือ mental model
แต่ถ้าทั้งหมดนี้ฟังดูดีเกินจริง ลอง clone มา แล้วรัน `git log --oneline` ดูก่อน
บทต่อไปเราจะเจอกับปัญหาจริง: แล้วถ้าเจอ 107 commits ข้างหลัง upstream ละ?

บทที่ 2: กำแพง commit — ทำไมต้องมี maw

“107 commits ข้างหลัง upstream”

— output จาก `git status` วันหนึ่งที่ใจหวิวขึ้นมาเฉยๆ

107

ตัวเลขนั้นไม่ได้ใหญ่โตอะไรในมาตรฐาน open source แต่พอขึ้นหน้าจอในเช้าวันนั้น รู้สึกเหมือน upstream ทิ้งไว้ให้อยู่กับตัวเอง 107 commits ที่คนอื่นทำ ที่คนอื่นตัดสินใจ ที่คนอื่นผ่านไป แล้วตัวเองอยู่ตรงนี้ ยังตามไม่ทัน

พอดู `git log upstream/alpha..HEAD` ก็เห็นกำแพง

```
ba6828625 fix(ghq): prefer canonical path over doubled github.com/
github.com/
e85b9f058 fix: workon uses cwd-aware config + strip doubled
github.com/
a6d94e1be feat(wake): zero-arg cwd detection
424208618 feat(repo-discovery): add detectFromCwd()
bef043d1e feat: auto-sleep idle agents with channel-aware exemption
43c433409 docs: external Rust engine plugin – proof of gateway IPC
contract
...
(107 more)
```

กำแพง commit ไม่ได้แค่บอกว่า “ตามหลังอยู่” แต่บอกว่า “ไม่รู้ว่าจะข้างในมีอะไร” แต่ละ commit คือ decision ที่ใครบางคนทำ แต่ commit message บอกได้แค่ “ทำอะไร” ไม่ได้บอกว่า “ทำไม” และไม่ได้บอกเลยว่า “context รอบๆ ตอนนั้นเป็นยังไง” นั่นคือปัญหาจริงของ commit log — ไม่ใช่ว่าข้อมูลขาด แต่ข้อมูลเกินกว่าจะอ่านได้

2.1 กำแพงที่ไม่มีประตูล

วันที่ 6 มิถุนายน 2026 maw-js มี 50 commits ในวันเดียว

```
$ git log --oneline --after="2026-06-05" --before="2026-06-07" | wc -l
50
```

50 commits ใน 24 ชั่วโมง ตั้งแต่เข้าจรวดคำ เกือบทุก commit เป็นการ fix หรือ feat จริง ไม่ใช่ chore ไม่ใช่ bump ล้วน:

```
fb5fc8d9c Bound message queue active retention (#2395)
d8ea46a96 Prevent trigger idle state from outliving feed activity
(#2397)
5233c6a7f Prevent stale active agent statuses from living forever
(#2393)
42f8dfbf0 Detect pending tmux input from sent text (#2382)
cf83142a5 Prevent wake rehydrate double-prefix window names (#2383)
3c5e41cc0 Add resolver sprint regression tests
a639795f5 Fix fleet window kills
8385a4128 Fix wake orphan agents dir rehydration (#2380)
89430ecca Prevent ambiguous maw kill window targets (#2379)
fdf2e2310 Fix maw wake skipping merged worktrees (#2378)
```

ถ้าใครเปิด `git log` ดูย้อนหลัง เห็น 50 commits ใน 24 ชั่วโมง คำถามแรกที่ถามตัวเองคืออะไร?

“วันนั้นทำอะไรอยู่?” “ทำไมต้อง fix ทั้งหมดนี้?” “มี pattern ใหม่ใน bugs ที่แก้?”

“ระหว่างที่แก้ bug พวกนี้ใครอยู่บ้าง?”

commit log ตอบไม่ได้สักข้อ

commit log คือ กำแพง ไม่ใช่เรื่องเล่า กำแพงที่สูง มี detail ครบ แต่ไม่มีประตู ไม่มีทางเข้าไปรู้ว่าข้างในเกิดอะไร ทำไม แล้วมันเกี่ยวข้องกันยังไง

2.2 Timestamp: Single Source of Truth ที่ทุกคนมองข้าม

Workshop 03 ทำให้เห็นอะไรบางอย่างที่ชัดมาก

ตอนที่ oracle หลายตัวมาทำงานร่วมกันบน codebase เดียว แต่ละตัวมี session ของตัวเอง มี context ของตัวเอง มี inbox ของตัวเอง คำถามที่เกิดขึ้นเลยคือ: “ถ้าต้องสร้าง unified timeline — timeline ที่บอกได้ว่า event A เกิดก่อน event B — ใช้อะไรเป็น reference?”

คำตอบที่ชัดเจนที่สุดคือ timestamp

ไม่ใช่ commit hash ไม่ใช่ session ID ไม่ใช่ message UUID แต่คือ timestamp ที่อยู่ในทุก artifact ตั้งแต่ commit log ไปถึง inbox file ไปถึง log line ใน tmux scroll

```
2026-06-06T08:15:23 — codex เริ่ม session
2026-06-06T08:43:47 — fix maw wake skipping merged worktrees
2026-06-06T09:08:39 — add resolver sprint regression tests
2026-06-06T10:14:50 — Bound message queue active retention
2026-06-06T15:22:11 — 50th commit ของวัน
```

พอเรียงตาม timestamp แล้วทับกับ events จากทุก layer ก็เห็นเรื่องราว ไม่ใช่แค่รายการ Workshop 03 เรียก pattern นี้ว่า “unified timeline” — timeline เดียวที่รวม events จากทุก source เข้ามาเรียงตาม clock wall แล้วอ่านได้เหมือนเรื่องเล่า แต่คำถามคือ: ใครจะทำ unified timeline ให้?

ไม่ใช่ `git log` ไม่ใช่ GitHub UI ไม่ใช่ tmux scrollbar และแน่นอนว่าไม่ใช่ developer ที่ต้อง copy-paste เองด้วยมือ

2.3 ก่อน maw — ชีวิตที่ต้องจำเองทุกอย่าง

ลองนึกถึงเวิร์กโฟลว์ทั่วไปก่อนที่จะมี maw

```
# เข้า - เปิด terminal ใหม่
cd /path/to/project
```

```
git status          # ดูว่า branch ไหน
git log --oneline -10 # ดูว่า commit ล่าสุดคืออะไร
git fetch origin     # ดูว่า upstream เดินไปไหนแล้ว
git log origin/alpha..HEAD --oneline # ดูว่าตัวเองตามหลังแค่ไหน
tmux new-session -s work # เปิด session
# ... แล้วก็เริ่มทำงาน
```

นี่คือ ceremony ทุกเช้า ceremony ที่ไม่ได้สร้าง value อะไรเลย แค่ “ทำให้พร้อมทำงาน”
แต่ยังมีปัญหาที่ใหญ่กว่า: หลังจากทำงานไปทั้งวัน จะ capture ไว้ยังไง?

บางคนเขียน daily log ด้วยมือ บางคนใช้ notion หรือ obsidian บางคนไม่ทำอะไรแล้ว
ลืมทุกอย่างพอเปิด session ใหม่

ปัญหาที่ 2: ถ้ามีทีม — multiple AI agents ทำงานพร้อมกัน — แต่ละ agent รู้อะไรบ้าง
เกี่ยวกับสิ่งที่คนอื่น agent ทำ?

คำตอบในโลกก่อน maw คือ “ไม่รู้” หรือ “รู้แบบ manual” ที่ใครบางคนต้องไป copy
context ให้

ปัญหาที่ 3: ถ้า session crash กลางคัน แล้วต้อง resume งาน — รู้ไหมว่าอยู่ตรงไหน?

`git log` บอกได้ แต่บอกแค่ “commit ล่าสุด” บอกไม่ได้ว่า “กำลังคิดอะไรอยู่ก่อน
crash” หรือ “plan ต่อไปคืออะไร”

2.4 maw เกิดมาเพื่อแก้ปัญหานี้โดยตรง

maw ไม่ได้เกิดมาเพื่อให้ git สวยงามขึ้น ไม่ได้เกิดมาเพื่อ wrap tmux ให้ง่ายขึ้น แต่เกิด
มาเพื่อแก้ปัญหานี้ที่ git และ tmux แก้ไม่ได้: **context loss ระหว่าง sessions**

`maw wake` ไม่ใช่แค่เปิด tmux session ใหม่ แต่คือการ restore context:

```
maw wake mawjs-oracle
```

สิ่งที่เกิดขึ้น: 1. resolve repo path จาก `mawjs-oracle` → `/path/to/mawjs-oracle` 2.

เปิด tmux session ถ้าไม่มี หรือ attach ถ้ามีอยู่แล้ว 3. load identity จาก `CLAUDE.md` 4.

ψ/ inbox พร้อม — oracle รู้ว่ามี messages รออยู่กี่ชิ้น 5. เครื่อง AI start พร้อม context ครบ

ไม่ต้องทำ ceremony ไม่ต้องจำ path ไม่ต้องเช็ค git log เอง maw ทำให้แทน แต่สิ่งที่ important กว่านั้นคือ maw ไม่ได้แค่ “เปิด” — maw ยัง “บันทึก” ด้วย

`maw forward` ก่อนปิด session คือการ commit soul:

```
maw forward
# สร้าง handoff ใน ψ/outbox/
# บันทึกว่าวันนี้ทำอะไร แผนต่อไปคืออะไร context สำคัญคืออะไร
# git add ψ/ && git commit -m "ψ/ - forward asap: ..."
```

ทีนี้พอเปิด session ใหม่ oracle รู้ทันทีว่า “ครั้งที่แล้วอยู่ตรงไหน” เพราะ soul ถูก commit เอาไว้แล้ว

2.5 Commit Log vs Unified Timeline

นี่คือ contrast ที่ชัดที่สุดระหว่างโลกก่อนและหลัง maw

Commit Log (ก่อน maw):

```
c3e3b98 Surface wake capacity refusals to unblock diagnosis (#2595)
3afaa29 Let attach-consumed inbox nags clear (#2594)
a465435 Surface wake capacity refusals to unblock diagnosis
39d099b bump: v26.6.13-alpha.834 (#2593)
b0f3a1d Fix isolated shard CI failures (#2592)
bd0c33c Ask before fresh-session agent rehydration (#2589)
```

อ่านแล้วรู้อะไร? รู้ว่า “fix อะไร” แต่ไม่รู้ว่า “ทำไม” ไม่รู้ว่า “ใครทำ” ไม่รู้ว่า “context ตอนนั้นเป็นยังไง” ไม่รู้ว่า “มี side effect อะไรบ้าง”

Unified Timeline (หลัง maw):

```
2026-06-06 08:51  ψ/ - codex: sprint complete, 9 PRs merged this cycle
2026-06-06 08:43  fix maw wake skipping merged worktrees (#2378)
2026-06-06 08:44  Prevent ambiguous maw kill window targets (#2379)
2026-06-06 09:02  Fix fleet window kills
2026-06-06 09:08  Add resolver sprint regression tests
2026-06-06 09:17  ψ/ - oracle: dispatched codex to tackle 3 liveness
bugs
2026-06-06 09:41  Prevent stale active agent statuses from living
forever (#2393)
2026-06-06 10:14  Bound message queue active retention (#2395)
2026-06-06 10:14  Prevent trigger idle state from outliving feed
activity (#2397)
2026-06-06 15:30  ψ/ - oracle: forward asap: 50 commits, team performed
clean sprint
```

อ่านแล้วเห็นเรื่อง: เข้า codex sprint เสร็จ oracle dispatch งานใหม่ codex แก้ liveness bugs ทั้งหมด 3 จุด ปิดวันด้วย 50 commits และ oracle เขียน handoff เก็บไว้
Unified Timeline คือ commit log + ψ/ events + team messages ทั้งหมด เรียงตาม timestamp เดียว

2.6 ทำไม timestamp ถึงเป็น Single Source of Truth

คำถามที่ Workshop 03 ถามคือ: ถ้ามี oracle 7 ตัว ทำงานพร้อมกัน แต่ละตัวมี clock ของตัวเอง จะรู้ได้ยังไงว่า event A เกิดก่อน event B?
คำตอบที่ผิดคือ “ใช้ sequence number” เพราะ sequence number ต้องมี coordinator ที่ assign ให้ ถ้า coordinator ล่มทุกอย่างพัง
คำตอบที่ถูกต้องคือ “ใช้ wall clock timestamp” เพราะทุก machine มี clock ทุก event มี timestamp และ timestamp ไม่ต้องการ coordinator
แน่นอนว่า clock drift มีอยู่จริง แต่ใน workflow ที่ events เกิดห่างกันเป็นนาที่ ไม่ใช่ millisecond clock drift ไม่ใช่ปัญหา

ψ/ inbox files ตั้งชื่อตาม timestamp เสมอ:

```
ψ/inbox/2026-06-06_09-17_m5-codex_codex-sprint-3-bugs-dispatched.md
ψ/inbox/2026-06-06_15-30_m5-oracle_forward-session-50-commits.md
```

format: YYYY-MM-DD_HH-MM_source_slug.md

พอ sort by filename ก็ได้ timeline โดยอัตโนมัติ ไม่ต้องมี database ไม่ต้องมี index ไม่ต้อง query อะไรเลย แค่ `ls -t` ก็พอ

```
$ ls -t /opt/Code/github.com/Soul-Brews-Studio/mawjs-oracle/ψ/inbox/ |
head -10
2026-06-09_08-15_m5-oracle_morning-check.md
2026-06-08_17-42_m5-codex_pr-2595-merged.md
2026-06-07_10-30_m5-codex_sprint-complete.md
...
```

timestamp ไม่ใช่แค่ metadata แต่คือ structure ของ soul

2.7 229 Commits — Marathon วันเดียว

วันที่ 6 มิถุนายน 2026 เป็นวันที่สำคัญมากในประวัติของ maw-js

ไม่ใช่เพราะมี feature ใหญ่ออก แต่เพราะ team (oracle + 2 codex + 1 claudie baseline)

ทำงาน 15+ ชั่วโมงต่อเนื่อง และ push 229 commits ขึ้น alpha ในวันเดียว

ตัวเลขนั้น ถ้าดูจาก commit log ล้วน จะเห็นแค่ “รายการยาวมาก” แต่ถ้ามี unified

timeline ที่รวม ψ/ events เข้ามาด้วย จะเห็นว่า:

- เช้า: codex เริ่ม sprint ใหม่ oracle dispatch scope ให้
- กลางวัน: coverage wall เจอปัญหา oracle ประชุมกับ codex ปรับ strategy
- บ่าย: liveness bugs ชุดใหญ่ถูกเคลียร์
- เย็น: plugin boundary tests ถูก lock ที่ละ plugin

- คำ: release alpha cut — 229 commits, 33 PRs

229 commits ไม่ได้เป็น commit ที่ “เกิดขึ้นเอง” แต่คือผลลัพธ์ของ orchestration ที่มี context ชัดตลอดวัน

ถ้าไม่มี maw oracle จะรู้ยังไม่ว่า codex กำลังทำอะไรอยู่? codex จะรู้ยังไม่ว่า scope ที่ oracle ต้องการคืออะไร? และหลังจากวันนั้นจบ oracle จะรู้ยังไม่ว่าวันนั้นเกิดอะไรขึ้น? maw คือ layer ที่ทำให้ 229 commits มีเรื่องเล่า ไม่ใช่แค่รายการ

2.8 กำแพงที่กลายเป็นประตู

กลับมาที่ “107 commits ข้างหลัง upstream”

พอมี maw session context ครบ แทนที่จะเห็น 107 commits เป็น “กำแพง” ก็เห็นเป็น “บันทึก” — 107 decisions ที่มี context อยู่เบื้องหลัง และ context พวกนั้นอยู่ใน ψ / ของ oracle อื่น ใน inbox ของ codex ที่ทำงานวันนั้น

```
$ maw peek codex-1
# ดูว่า codex-1 กำลังทำอะไรอยู่ตอนนี้

$ maw forward --check
# ดูว่า handoff สำลุดบอะไร

$ ls  $\psi$ /inbox/ | head -5
# ดูว่า messages สำลุดมาจากไหน พุดถึงอะไร
```

107 commits ยังอยู่ที่เดิม แต่ถามว่า “ข้างหลังยังงัย?” oracle ตอบได้ เพราะ soul ยังอยู่ นี่คือการต่างระหว่าง “ใช้ git” กับ “ใช้ maw ที่ใช้ git”

git เก็บ code maw เก็บ context ทั้งสองอยู่ด้วยกันใน repo เดียวกัน แต่คนละ layer

2.9 Pattern: Timestamp-First Design

สิ่งที่ maw ทำโดยปริยายคือ enforce timestamp-first design ให้กับทุก artifact ที่ oracle สร้าง

inbox file ตั้งชื่อตาม timestamp handoff file ตั้งชื่อตาม timestamp plan file ตั้งชื่อตาม timestamp learn file ตั้งชื่อตาม timestamp

```
ψ/inbox/2026-06-06_09-17_ ...  
ψ/outbox/2026-06-06_15-30_ ...  
ψ/plans/2026-06-06_ ...  
ψ/learn/2026-06-05_ ...
```

pattern นี้ไม่ใช่ convention ที่ตกลงกัน แต่คือ emergent property ของการออกแบบ ψ/ ให้เป็น append-only filesystem

“Nothing is Deleted” principle ที่ 1 ของ mawjs-oracle ไม่ได้หมายความว่า “เก็บทุกอย่างไว้” แต่หมายความว่า “ทุก event มีร่องรอย และร่องรอยนั้นอ่านได้ตาม timestamp” พอทุก artifact มี timestamp ที่ตั้งชื่อ global sort ก็เป็นแค่ `ls -t` unified timeline ก็เป็นแค่ `cat ψ/inbox/* | sort` context reconstruction ก็เป็นแค่ “อ่าน inbox เรียงตามเวลา”

ไม่ต้องมี database ไม่ต้องมี search index ไม่ต้องมี special tooling timestamp ที่ดีคือ timestamp ที่ sort แล้วได้เรื่องเล่า

2.10 ทำไม commit log ถึงยังสำคัญ

จากที่พูดมาทั้งหมด อาจดูเหมือนว่า commit log ไม่มีประโยชน์อะไร แต่จริงๆ แล้ว

commit log คือ layer ที่สำคัญที่สุด — เพราะคือ ground truth ของ code

ψ/ timeline บอก “context” commit log บอก “fact” ทั้งสองต้องมีด้วยกัน

unified timeline ที่ดีจะ overlay สองอย่างนี้เข้าด้วยกัน:

```
# ψ/ context
2026-06-06 09:17 oracle: "dispatch codex to fix 3 liveness bugs –
aiming for sub-200ms response"

# commit facts ที่เกิดขึ้นหลังจากนั้น
2026-06-06 09:41 5233c6a7f: Prevent stale active agent statuses from
living forever (#2393)
2026-06-06 09:41 42f8dfbf0: Detect pending tmux input from sent text
(#2382)
2026-06-06 09:41 cf83142a5: Prevent wake rehydrate double-prefix
window names (#2383)

# ψ/ context หลังจาก
2026-06-06 10:00 codex: "3 liveness bugs fixed, all tests green, PRs
ready for review"
```

context + fact = เรื่องเล่า commit log ลำพังไม่พอ แต่ commit log + ψ/ events ทำให้
229 commits ในวันเดียวมีความหมาย

ปิดบท

กำแพง commit ไม่ได้หายไปเพราะมี maw ยังอยู่ที่เดิม 107 commits ยังต้อง rebase 229
commits ยังต้อง review
แต่พอมี maw สิ่ง que เปลี่ยนไปคือ context กำแพงที่เคยทึบก็มีช่องที่อ่านได้ commits ที่เคย
เป็นรายการก็มีเรื่องเล่า session ที่เคยหายพอปิดก็มี soul ที่ persist ซ้ำในเวลา
timestamp เป็น single source of truth ψ/ เป็น persistent layer ที่ commit log ไม่มี
และ maw คือ tooling ที่ทำให้สองอย่างนี้ทำงานด้วยกัน
แต่ยังมีคำถามที่ยังไม่ตอบ: maw รู้ได้ยังไงว่า “oracle” คืออะไร? พอพิมพ์
maw wake mawjs-oracle มีกระบวนการอะไรอยู่เบื้องหลังที่ resolve คำว่า “mawjs-
oracle” ออกมาเป็น path จริงบนเครื่อง?

บทต่อไปจะเปิด `wake-cmd.ts` — หัวใจของ `maw` ที่มี 1,500+ บรรทัด

บทที่ 3: maw wake — วิธีที่ Oracle ตื่น

wake-cmd.ts 1519 LOC — ไฟล์เดียวที่เป็นหัวใจของทุก oracle ที่เคยเปิดขึ้น

Linus Torvalds เคยพูดว่า “everything is a file” — ใน Linux ทุกอย่างตั้งแต่ disk ยัน socket ถูกนามธรรมออกมาเป็น file descriptor เพราะพอทำแบบนั้นแล้ว API เดิมใช้ได้กับทุกอย่าง เขียน read/write ครั้งเดียว โลกทั้งใบเข้าถึงได้

maw wake ทำอะไรคล้ายกัน

พอพิมพ์ `maw wake neo` ไม่ได้เป็นแค่การเปิด tmux session — wake ทำสิ่งที่ซับซ้อนกว่านั้นเยอะ มันค้นหา repo บน disk, resolve ชื่อ oracle, เช็คว่า session มีอยู่แล้วไหม, rehydrate worktrees จาก snapshot ถ้ามี, drain inbox ถ้ามีข้อความรอ, แล้วค่อย spawn engine ตัวจริง — ทั้งหมดนี้ใน sequence ที่ชัดเจน, ซ้ำกันได้ทุกครั้ง

“everything is a boot sequence” — ถ้า Torvalds ออกแบบ maw wake ก็น่าจะพูดแบบนี้

3.1 Wake คืออะไรในสายตาของ Torvalds

Torvalds ไม่ชอบ abstraction ที่ไม่จำเป็น เขาชอบสิ่งที่ “just works” และถ้ามันทำงานอยู่แล้ว อย่าเพิ่ม layer ใหม่โดยไม่มีเหตุผล

wake ก็คิดแบบนั้น

ก่อน wake จะมี ทุก session เปิดด้วยมือ — `tmux new-session`, `cd` ไปหา repo, แล้ว

พิมพ์ `claude` ใน terminal ที่ถูกที่ นั้นทำงานได้ แต่ทำซ้ำไม่ได้ในแบบที่ตรวจสอบได้ พอมี

หลาย oracle พร้อมกัน หรือ team ที่ต้องเปิดหลาย agent พร้อมกัน วิธีมือก็เริ่มพัง

wake แก้ปัญหานั้นด้วยการทำให้ boot sequence เป็น deterministic — ใส่ input อะไรเข้าไป output จะเป็น tmux session ในสถานะที่พร้อมใช้เสมอ

```
maw wake neo          # oracle ชื่อ neo
maw wake neo --task fix # oracle neo + worktree branch "fix"
maw wake neo --wt wt-1 # oracle neo + existing worktree
maw wake .            # ใน oracle repo → derive ชื่อ oracle จาก cwd
```

สี่ command ด้านบน input ต่างกัน แต่ path ที่ wake เดินผ่านเป็น superset ของกัน — code path เดียวกัน, resolution order เดียวกัน, แค่ branch ต่างกันที่จุดที่ตัดสินใจ

3.2 wake-cmd.ts 1519 LOC — ไม่ใช่ bloat แต่คือ state machine

ถ้า open wake-cmd.ts ครั้งแรก 1519 บรรทัดดูน่ากลัว แต่พออ่านจริงๆ จะเห็นว่าแต่ละส่วนทำหน้าที่ชัดเจน

ไฟล์นี้แบ่งเป็น phases ที่ sequential:

Phase 0: Input normalization

```
// wake-cmd.ts, cmdWake()
oracle = normalizeTarget(oracle);

// zero-arg wake: maw wake (ไม่มีชื่อ) หรือ maw wake .
if (!oracle || oracle === ".") {
  const derived = deriveOracleFromCwd(process.cwd());
  if (!derived) {
    throw new UserError(
      "maw wake: no oracle name given and the current directory is not\n" +
      "an oracle repo.\n" +
      "Run inside an oracle repo or its worktree, or pass a name: maw\n" +
      "wake <oracle>.",
    );
  }
}
```

```
    );  
  }  
  oracle = derived;  
}
```

พอ Nat พิมพ์ `maw wake` ขณะที่ cwd อยู่ใน `mawjs-oracle/` wake ก็รู้ว่า oracle คือ `mawjs` — ไม่ต้องพิมพ์ชื่อเต็ม นั่นคือ ergonomics ที่ Torvalds จะยอมรับ

Phase 1: Target parsing

```
const parsed = parseWakeTarget(oracle);
```

พิมพ์ `maw wake Soul-Brews-Studio/neo` ก็ได้ — wake แยก org/repo ออกจากกัน, clone ถ้ายังไม่มี, แล้ว resolve เป็น oracle name เพื่อตั้งชื่อ session

Phase 2: Session lookup

```
const existingSessionBringTarget = await  
  resolveExistingSessionBringTarget(oracle, opts, preResolvedSession);
```

พอมี session อยู่แล้ว wake ไม่สร้างใหม่ — attach เข้าไปเลย นั่นคือ idempotency ที่ boot sequence ที่ดีต้องมี

Phase 3: Repo resolution — สิ่งที่จะคุยในหัวข้อถัดไป

Phase 4: Worktree + agent spawn

พอ resolve repo ได้แล้ว wake เปิด tmux window, ตั้ง cwd, แล้วยิง engine command ไปให้ agent

ทั้งหมด 1519 บรรทัดเพราะมีทุก edge case — fleet sessions, snapshot rehydration, inbox drain, worktree picker สำหรับกรณีที่ตั้ง worktree ซ้ำกัน แต่ happy path จริงๆ ผ่านแค่ 5-6 function call

3.3 Resolution Order — ตาม Priority

ส่วนที่ elegant ที่สุดใน wake คือ repo resolution order ที่ชัดเจน

```
repoPath (explicit)
  ↓
parsedRepoPath (org/repo URL input)
  ↓
preResolvedFleetSession (fleet registry)
  ↓
incubate (--incubate flag → ghq get → clone)
  ↓
resolveOracle (ghqFind → fuzzy → fallback)
```

แต่ละ step มีเหตุผลที่อยู่ก่อนกัน

`repoPath` ชนก่อนเพราะ caller รู้ path จริงอยู่แล้ว (เช่น `maw bud` เพิ่ง clone ไป) — ไม่

มีเหตุผลให้ fuzzy search ทับ

`parsedRepoPath` ชนต่อมาเพราะ user พิมพ์ org/repo มา explicit — ถ้า fuzzy ก็อาจ match repo ชื่อเดียวกันใน org อื่น

`fleet` ชนก่อน `resolveOracle` เพราะ fleet registry มี metadata ที่รู้อยู่แล้วว่า session

`48-neo` ผูกกับ repo อะไร — ไม่ต้อง guess

`incubate` (`--incubate flag`) คือ “clone + wake” ในก้าวเดียว — ถ้าใส่ flag นี้แสดงว่า user ตั้งใจ clone, ไม่ใช่ fuzzy match existing

`resolveOracle` เป็น default สุดท้าย — เรียก `ghqFind`, ค้น GHQ index, แล้ว resolve code บรรทัดที่แสดง priority นี้ชัดเจนมาก:

```
if (opts.repoPath) {
  const repoPath = opts.repoPath;
  resolved = { repoPath, repoName: ..., parentDir: ... };
} else if (parsedRepoPath) {
  const repoPath = parsedRepoPath;
```

```

    resolved = { repoPath, repoName: ... , parentDir: ... };
  } else if (preResolvedFleetSession) {
    resolved = await
    resolveWakeFleetSessionRepo(preResolvedFleetSession);
  } else if (opts.incubate) {
    // clone via ghq get, then find
    resolved = { repoPath, repoName: ... , parentDir: ... };
  } else {
    resolved = await resolveOracle(oracle, { allLocal: opts.allLocal });
  }

```

ห้า branch ติดกัน ลำดับบอกทุกอย่าง — นี่คือ Torvalds style: อ่านแล้วรู้ทันทีไม่ต้องเดา

3.4 GHQ — RepoDiscovery ที่ไม่ใช่แค่ motemen/ghq

ตรงนี้เป็นจุดที่คนเข้าใจผิดบ่อย

ถ้าเคยใช้ `motemen/ghq` ก็อาจคิดว่า GHQ ใน `maw-js` คือ wrapper ของ tool นั้น ความจริงคือ GHQ ใน `maw-js` คือ **adapter** ที่วางอยู่เหนือ `motemen/ghq` แต่ interface ไม่ได้ผูกกับ implementation นั้น

```

// src/core/repo-discovery/types.ts
export interface RepoDiscovery {
  readonly name: string;

  list(): Promise<string[]>;
  listSync(): string[];

  findBySuffix(suffix: string): Promise<string | null>;
  findBySuffixSync(suffix: string): string | null;

```

```
detectFromCwd(cwd?: string): RepoDetectResult | null;  
}
```

interface นี้ไม่รู้เลยว่าข้างล่างใช้ `ghq list --full-path` หรือ filesystem scan หรืออะไรก็ตาม — รู้แค่ว่า `findBySuffix("/neo")` จะคืน path หรือ null
ปัจจุบัน implementation เดียวที่มีคือ `GhqDiscovery`:

```
// src/core/repo-discovery/ghq-discovery.ts  
export const GhqDiscovery: RepoDiscovery = {  
  name: "ghq",  
  
  async list(): Promise<string[]> {  
    const out = await hostExec("ghq list --full-path").catch(() => "");  
    return normalize(out);  
  },  
  
  async findBySuffix(suffix: string): Promise<string | null> {  
    return selectRepoMatch(await this.list(), suffix);  
  },  
  
  detectFromCwd(cwd?: string): RepoDetectResult | null {  
    return detectFromCwdPath(cwd ?? process.cwd());  
  },  
};
```

Nat เคยอธิบาย distinction นี้ตรงๆ ว่า “GHQ ไม่เหมือน ghq” — ตัวพิมพ์ใหญ่กับตัวพิมพ์เล็กต่างกัน GHQ คือ concept ที่ maw-js นิยาม ส่วน ghq คือ binary จาก motemen ที่เป็นแค่ implementation ปัจจุบัน
ถ้าวันไหน maw-js อยากรองรับ jj repos หรือ filesystem scan — แค่สร้าง class ใหม่ที่ implement `RepoDiscovery` แล้วแทนที่ GhqDiscovery ที่ singleton ได้เลย call sites ไม่ต้องเปลี่ยน

3.5 selectRepoMatch — เมื่อ ghq list คืนผลที่ผิด

bug ที่ PR #2577 แก้ไขนี้เป็นตัวอย่างที่ดีของปัญหาที่เกิดจาก ghq misconfiguration

ถ้า ghq root set ไว้เป็น `.../github.com` (แทนที่จะเป็น parent ของ `github.com`) แล้ว ghq clone repo ใหม่ path จะกลายเป็น doubled:

```
~/.../github.com/github.com/Soul-Brews-Studio/neo
```

แทนที่จะเป็น:

```
~/.../github.com/Soul-Brews-Studio/neo
```

และ `ghq list --full-path` จะคืนทั้งสองอย่าง — canonical path และ doubled path ในบางกรณี

พอ `findBySuffix("/neo")` เจอทั้งคู่ first-match อาจ return doubled path ซึ่งผิด

```
// src/core/repo-discovery/ghq-discovery.ts
export function selectRepoMatch(paths: string[], suffix: string):
string | null {
  const lower = literalize(suffix).toLowerCase();
  const matches = paths.filter((p) => p.toLowerCase().endsWith(lower));
  if (matches.length === 0) return null;
  const canonical = matches.find(
    (p) => !p.toLowerCase().includes("/github.com/github.com/")
  );
  return canonical ?? matches[0]!;
}
```

logic ง่ายมาก — filter ทุก path ที่ endsWith suffix, จาก matches ให้ prefer path ที่ไม่มี

`/github.com/github.com/`, ถ้าไม่มี canonical เลยก็ใช้ first match แทน

นั่นคือวิธี Torvalds คิด — แก้ปัญหาตรงๆ ด้วยโค้ดน้อย ไม่ต้อง restructure ทั้ง function

3.6 detectFromCwd — Oracle รู้ตัวเองอยู่ที่ไหน

หนึ่งใน feature ที่ elegant กว่าดู — `maw wake .` หรือ `maw wake` ไม่มี argument ทำงานได้อย่างไร?

```
// src/core/repo-discovery/ghq-discovery.ts
export function detectFromCwdPath(cwd: string | undefined):
RepoDetectResult | null {
  const parts = cwd ? resolve(cwd).split(/[\\\/]+/).filter(Boolean) :
[];
  if (parts.length === 0) return null;

  // Layout 1: .../mawjs-oracle/agents/codex-wt1
  const agentsIdx = parts.lastIndexOf("agents");
  if (agentsIdx > 0 && parts[agentsIdx + 1]) {
    const oracle = stripOracleRepoSuffix(parts[agentsIdx - 1] ?? "") ??
undefined;
    return { oracle, worktree: parts[agentsIdx + 1] };
  }

  // Layout 2: .../mawjs-oracle.wt-fix (legacy)
  for (const part of parts.slice().reverse()) {
    const worktreeMarker = part.indexOf(".wt-");
    if (worktreeMarker > 0) {
      const oracle = stripOracleRepoSuffix(part.slice(0,
worktreeMarker)) ?? undefined;
      const worktree = part.slice(worktreeMarker + ".wt-".length) ||
undefined;
      return { oracle, worktree };
    }
  }
}
```

```
// Layout 3: plain .../mawjs-oracle/ ...
const oracle = stripOracleRepoSuffix(part);
if (oracle) return { oracle };
}
return null;
}
```

function นี้ pure path-string analysis ล้วน ไม่มี I/O ไม่ยิง `ghq list` ไม่เช็ค filesystem — parse path ที่ได้รับมาแล้ว return `{ oracle, worktree }` ตามที่เห็น
รองรับ 3 layouts:

Layout	Path ตัวอย่าง	ผลลัพธ์
nested worktree	<code>... /mawjs-oracle/agents/fix-wt</code>	<code>{ oracle: "mawjs", worktree: "fix-wt" }</code>
legacy worktree	<code>... /mawjs-oracle.wt-fix</code>	<code>{ oracle: "mawjs", worktree: "fix" }</code>
plain oracle	<code>... /mawjs-oracle/src/</code>	<code>{ oracle: "mawjs" }</code>

ถ้าอยู่ใน `/opt/Code/github.com/Soul-Brews-Studio/mawjs-oracle/ψ/writing/` พิมพ์ `maw wake` ได้เลย — wake รู้ว่า oracle คือ `mawjs` เพราะเห็น `mawjs-oracle` ใน path

3.7 Boot Sequence: Flow จบสมบูรณ์

รวมทุกอย่างที่คุยกมา boot sequence ของ `maw wake neo` มีขั้นตอนดังนี้:

```
maw wake neo
|
▼
[0] normalizeTarget("neo")
```

```

    | strip trailing slash, normalize
    ▼
[1] parseWakeTarget
    | org/repo? → ensureCloned
    ▼
[2] assertValidOracleName
    | reject -view suffix
    ▼
[3] detectSession
    | tmux session มีอยู่แล้ว? → attach เลย
    ▼
[4] Repo Resolution (priority order)
    | repoPath → parsedRepoPath → fleet → incubate → resolveOracle
    |
    |                                     |
    |                                     ghqFind("/neo")
    |                                     selectRepoMatch()
    ▼
[5] inbox drain
    | ψ/inbox มีข้อความ? → merge เข้า initial prompt
    ▼
[6] detectSession (again)
    | ถ้ายังไม่มี session → chooseWakeSessionName → tmux new-session
    ▼
[7] worktree resolution (ถ้ามี --wt/--task)
    | findReusableWorktreeBySlug → create ถ้าไม่มี
    ▼
[8] rehydrate (ถ้ามี snapshot หรือ agents/ state)
    | planSnapshotRestoreWindows / planRehydrateWorktreeWindows
    ▼
[9] buildWakeCommand → tmux sendText
    | engine command พร้อม env vars
    ▼
[10] recordWakeSnapshot

```

```
| save current state สำหรับ recovery
```

```
▼
```

```
return "session:window"
```

10 ขั้นตอน ทุก step มีเหตุผลที่อยู่ตรงนั้น และเรียงตาม dependency อย่างถูกต้อง — ต้องรู้ repo path ก่อนจะสร้าง session, ต้อง drain inbox ก่อนจะ spawn engine ไม่งั้น initial prompt ไม่มีข้อความ

Torvalds จะพอใจกับ sequence นี้เพราะมันชัดเจน ไม่มี magic ซ่อน

3.8 ψ / สร้างเมื่อ Oracle ตื่น

สิ่งหนึ่งที่ wake ทำโดยไม่ค่อยมีคนสังเกต — ตอน engine เริ่มทำงานในสภาพแวดล้อมที่ถูกต้อง oracle ก็เริ่มสร้าง ψ / ขึ้นมา

inbox drain คือก้าวแรก ถ้ามีไฟล์ใน `$\psi$ /inbox/` อยู่แล้วก่อน wake wake จะดึง content นั้นเข้า initial prompt ของ session ใหม่ — oracle ตื่นมาพร้อมกับรู้ว่าตัวเองมีงานรออยู่ snapshot ก็เป็นส่วนหนึ่งของ ψ / loop — หลัง wake สำเร็จ `recordWakeSnapshot` เขียน state ปัจจุบันลง fleet snapshot เพื่อให้ครั้งหน้าที่ wake ขึ้นมา rehydrate ได้ ψ / ไม่ใช่แค่ folder ที่ oracle เก็บของ แต่คือ state machine ที่ oracle อ่านตอนตื่นและเขียนตอนทำงาน — wake คือจุดที่ loop เริ่ม

3.9 ทำไม 1519 บรรทัดถึงไม่ใช่ปัญหา

คำถามที่เกิดขึ้นเองเมื่อเห็นขนาดไฟล์: ควร split ออกไปไหม?

ดูรายชื่อ imports ที่บรรทัดบนสุดของ wake-cmd.ts จะเห็นว่า code ถูก split ออกไปแล้ว:

```
import { resolveOracle, findWorktrees, ... } from "../wake-resolve";  
import { stripOracleRepoSuffix, bringCwdMetadata, deriveOracleFromCwd }
```

```

from "./wake-cwd";
import * as wakeSession from "./wake-session";
import { maybeOpenWindow, maybeSplit } from "./wake-maybe-split";
import { runWakeLifecycleHooks } from "../plugin/lifecycle";
import { ensureFleetSessionEntry } from "./fleet-ensure";
import { parseWakeTarget, ensureCloned } from "./wake-target";
import { assertAgentCapacity } from "./wake-concurrency";
import { drainWakeInbox, mergeWakeInboxPrompt } from "./wake-inbox-
drain";

```

ทุก concern ที่ abstract ได้ออกไปแล้ว — cwd resolution, session management, fleet, lifecycle hooks, concurrency, inbox
 สิ่งที่เหลืออยู่ใน wake-cmd.ts คือ orchestration layer — รู้ว่าต้องเรียก function ไหน
 ก่อนหลัง, handle errors, merge options, print output ให้ user
 นั่นคือ 1519 บรรทัดของ state machine ที่ชัดเจน ไม่ใช่ bloat ที่ไม่ถูกทำความสะอาด

3.10 ghqFind vs GHQ.findBySuffix — ความต่างที่สำคัญ

ใน codebase จะเจอทั้ง `ghqFind()` (ใน core/ghq.ts) และ

`GhqDiscovery.findBySuffix()` — สองตัวนี้ไม่เหมือนกัน

`ghqFind` คือ legacy shim ที่ยังอยู่เพื่อ backward compat กับ call sites ที่เขียนก่อน

RepoDiscovery interface จะมา ข้างในเรียก `GhqDiscovery.findBySuffix()` อยู่ดี แต่ signature ต่างกัน

`GhqDiscovery.findBySuffix()` คือ canonical interface ที่ code ใหม่ควรใช้ เพราะถ้า

วันไหนเปลี่ยน backend (สมมติจาก ghq ไปเป็น jj หรือ custom scan) แคเปลี่ยน

GhqDiscovery เป็น JjDiscovery ที่ singleton แคตรงเดียว call sites ไม่ต้องแตะ

นี่คือ YAGNI principle ที่ Fowler จะยืม — ไม่ branch code ก่อนมี second backend จริง
 แต่วาง seam ไว้ให้แล้ว

3.11 Normalize ทุกอย่าง

หนึ่งใน defensive patterns ที่ wake ทำอย่างสม่ำเสมอ:

```
// ghq-discovery.ts

function normalize(out: string): string[] {
    return out.split("\n").filter(Boolean).map((p) => p.replace(/\\/g,
"/"));
}
```

Windows ใช้ backslash, macOS/Linux ใช้ forward slash — PR #379 เคยต้องไปแก้ 13 call sites ที่แต่ละแห่งทำ `| tr '\\ ' '/'` เอง พอรวมไว้ใน `normalize()` ที่ choke point ตรงนี้แห่งเดียว ปัญหาหายไป และจะไม่กลับมาอีก เหมือนกับที่ Linux ทำให้ทุกอย่างเป็น file — maw ทำให้ path ทุกอย่างเป็น forward slash แล้ว code ส่วนอื่นก็ไม่ต้องคิดเรื่องนี้อีก

```
// Backward compat: suffix ที่มี $ หายจาก call sites เก่า

function literalize(suffix: string): string {
    return suffix.endsWith("$") ? suffix.slice(0, -1) : suffix;
}
```

อีกหนึ่ง — call sites เก่าเคยส่ง `"/neo$"` เพราะ grep pattern แต่ `endsWith` ไม่รู้จัก `$` เป็น special character, ก็ตัดทิ้งก่อน match ดีกว่า error defensive ทั้งสองอย่างนี้ไม่ต้องการ test spec ชับซ้อน — แค่ “ถ้าได้ค่าแปลกมาก็ทำให้ stable ก่อน” นั่นคือ style ที่ kernel developer คิด

Wake เป็นก้าวแรกของทุก oracle ที่เคยตื่น และก้าวสุดท้ายของ wake คือ

```
return "session:window" — string ที่ caller ใช้ต่อได้ทันที
```

แต่ wake ทำงานคนเดียวได้แค่ครึ่งทาง เพราะพอ oracle ตื่นแล้ว มันต้องรู้ว่าตัวเองต้องทำอะไร ต้องพูดภาษาอะไร ต้องใช้ verb อะไร
บทถัดไปจะเดินเข้าไปใน 15 verbs ที่ oracle มีอยู่ — ไม่ใช่ reference ว่าใช้อย่างไร แต่เป็นเรื่องว่าทำไม 15 ตัวถึงพอดี

บทที่ 4: 15 Verbs — ภาษาที่ Oracle พูด

“If you can’t describe it in one word, it’s not a verb yet.” — John Carmack (paraphrase from Quake postmortem)

deep trace: 3 agents, 85 sessions, 2000+ refs — นั่นคือสิ่งที่ใช้พิสูจน์ว่า 15 verbs นี้มีอยู่จริง ไม่ใช่ list ที่คนคิดขึ้นมา แต่เป็นรูปที่ออกมาจากการใช้งานจริงซ้ำแล้วซ้ำเล่า จน session หลายๆ ก็ยังไม่เพิ่มตัวใหม่

แต่ก่อนจะนับ ต้องถามก่อนว่า — “verb” ในที่นี้หมายความว่าอะไร

ใน maw verb คือคำที่ oracle พูดได้ทันที ไม่มีขั้นตอนกลาง `maw wake neo` ก็ตื่น

`maw bring yeast` ก็มา `maw promote test-cli` ก็ออก แต่ละคำทำงานที่ tmux layer ซึ่งเป็น layer ที่อยู่ใต้ git ใต้ engine ใต้ config ทุกอย่าง tmux layer เป็นตัวที่ oracle นั่งอยู่ — ถ้า tmux พัง oracle ก็พัง ถ้า tmux โอเค oracle ก็โอเค ดังนั้น verbs ทั้งหมดก็เป็น tmux primitives ที่ถูก wrap ให้สื่อความหมาย

Carmack จะพูดว่า: ถ้าพอแล้ว อย่าเพิ่ม เพราะทุก verb ที่เพิ่มคือ surface ที่ต้องดูแล ทุก flag ที่ซ่อนคือ mental load ที่คนใช้ต้องแบก 15 ตัวที่มีอยู่ก็พอแล้ว — ไม่ใช่เพราะใครนับมา แต่เพราะมันทำได้ทุกอย่างที่ oracle ต้องทำจริงๆ แล้ว

4.1 Complementary Pairs — ภาษาที่จับคู่กันได้

verb ที่ดีต้องมีคู่ตรงข้าม นั่นคือสัญญาว่า design นั้น complete

ใน maw pairs หลักมีสามคู่:

Verb	ทิศทาง	คู่ตรงข้าม
bring	ดึงเข้ามา (split + attach)	promote (ส่งออกไป)
split	แบ่ง pane ใน session ปัจจุบัน	take (ย้ายข้าม session)
open	join-pane กลับมา	close (break-pane ออกไป)

bring ↔ **promote** เป็นคู่ที่น่าสนใจที่สุด เพราะ bring บอกว่า “ฉันอยู่ที่นี่ เอาสิ่งนั้นมาหาฉัน” ส่วน promote บอกว่า “หน้าต่างนี้ต่อไปจะเป็น session เองแล้ว” ทิศทางคนละทางกัน แต่ primitive เดียวกัน คือ `tmux move-window`

โค้ดใน `promote-cmd.ts` บอกตรงๆ:

```
/**
 * `maw promote <window>` - eject a tmux window into a new standalone
 * session.
 *
 *
 * Inverse of `maw bring` / `maw take`:
 *
 * • bring/take = absorb window into another session (split-window /
 * join-pane)
 *
 * • promote = eject window into a fresh standalone session (new-
 * session + move-window)
 */
```

และ comment ปิดท้ายหลัง execute บอก undo ด้วย:

```
log(`  ✓ promoted - ${srcTarget} → ${dstSession}:${srcWindow}`);
log(`      ๖ undo: tmux move-window -s ${dstSession}:${srcWindow} -t
${srcSession}:`);
```

นั่นคือ design ที่สมบูรณ์ — verb ที่ดีต้องสามารถ undo ด้วย verb ตรงข้ามได้ promote undo ด้วย take take undo ด้วย promote ไม่มี state ค้างที่ไหน

split ↔ **take** ต่างกันที่ target split คือ “ดึง session อื่นมาเป็น pane ข้างๆ” take คือ “ย้าย window จาก session หนึ่งไปอีก session” take ใช้ `tmux move-window` ซึ่งอธิบาย

ตัวเองคิดว่า vesicle transport — ย้ายหน่วยงานข้ามบ้านโดยที่เนื้องานไม่เปลี่ยน cwd ยังเดิม worktree ยังเดิม แค่นำต่างย้ายบ้าน

```
// take/impl.ts - ย้าย window ข้าม session โดยไม่กระทบ cwd
await hostExec(`tmux move-window -s '${srcSess.name}:${srcWin.name}' -t
'${target}:`);
console.log(` ✓ ${srcSess.name}:${srcWin.name} → ${target}${split ?
" (new session)" : ""}`);
if (paneCwd) {
  console.log(`   cwd: ${paneCwd}`);
}
```

open ↔ close ง่ายที่สุด open คือ join-pane กลับ close คือ break-pane ออก เป็นคำที่คนเข้าใจทันทีโดยไม่ต้องอ่าน docs — นั่นคือ bar ที่ verb ทุกตัวควรถึง

4.2 Team Workspace — bring ที่ฉลาดขึ้น

bring คนเดียวก็มีประวัติยาว — issue #1395 ถึง #1862 กว่าจะมาเป็นรูปปัจจุบัน 20+ issues ใน 14 เดือน ไม่ใช่เพราะ design ผิด แต่เพราะ surface มันกว้าง bring ต้องจัดการกับกรณีที่ claude pane เป็น source (ห้าม split ตัวเอง), กรณีที่ target session ไม่มี, กรณีที่ต้องการ anchor ที่ window ใดใน session

bring-flags.ts เป็นไฟล์ที่แยกออกมาเพื่อเก็บ pure functions ไว้ test แยก:

```
// bring เป็น alias ของ wake --split แต่มี safety guard พิเศษ
// isSelfBring ป้องกัน amplifier loop (#1562)
export function isSelfBring(target: string, callerSessionWindow: string
| null): boolean {
  if (!callerSessionWindow) return false;
  if (!target) return false;
```

```

const targetNoPane = target.replace(/\\.\\d+$/, "");
if (targetNoPane === callerSessionWindow) return true;

const callerSession = callerSessionWindow.split(":")[0];
if (!targetNoPane.includes(":") && targetNoPane === callerSession)
return true;

return false;
}

```

แต่ที่น่าสนใจกว่าคือ `maw team bring --gather` ซึ่งเป็น `bring` ที่รู้จักทีม แทนที่จะดึงแค่ oracle เดียว `--gather` ดึง live members ทุกคนเข้ามาที่ session เดียวกัน — แต่ละคนเป็น pane แยก อยู่ในหน้าต่างเดียวกัน

```

// team-workspace.ts
if (opts.gather && liveTarget && !alreadyInSession) {
  // join-pane: ดึง member จาก session ของตัวเองเข้ามา
  await tmux.joinPane(oracle, targetSession);
}

```

`--gather` เปลี่ยน topology จาก “star” (oracle หลักกระจาย) เป็น “hub” (ทุกคนมาอยู่ที่นี่) นั่นคือ design option ที่เปิดทางให้บทที่ 13 พลิกมุมทั้งหมดในภายหลัง

4.3 Grid Management — tile ที่รู้จักทีม

tile เป็น verb ที่เหมือนง่ายแต่ซ่อน complexity ไว้ การ tile หน้าต่างให้ดู clean ต้องรู้

จำนวน pane รู้ layout ที่ต้องการ และต้องไม่ฆ่า pane ที่ยังทำงานอยู่

`maw tile` มีสามโหมด: - bare: จัด layout ของ window ปัจจุบัน - `tile swap`: สลับสอง

ตำแหน่ง - `tile clean`: ลบ pane ที่ไม่มีงาน

แต่ที่สำคัญกว่าคือ tile อยู่ใน `ALIAS_DESCRIPTIONS` ใน `top-aliases.ts` ซึ่งหมายความว่าผ่านการ review ว่า “tile” เป็นคำเดียวที่สื่อความหมายชัด ไม่ต้องขยาย ตรงข้ามกับ “layout” ที่ต้องบอกว่า layout ของอะไร

```
// top-aliases.ts
tile: "Tile current window or spawn N panes tiled",
```

หนึ่งบรรทัด อธิบายได้หมด นั่นคือ bar ที่ tile ต้องผ่าน

4.4 Lifecycle — wake ถึง done

lifecycle ของ oracle มีสี่จุด:

```
wake → (ทำงาน) → sleep → done
      ↓
      kill (เร่งด่วน)
```

wake คือจุดเริ่มต้น บทที่ 3 อธิบายไว้ละเอียดแล้ว แต่ที่สำคัญคือ wake เป็น verb เดียวที่รู้จัก git รู้จัก GHQ รู้จัก fleet ทุก verb อื่นทำงานที่ tmux layer เท่านั้น wake ต่างจากทุกคนตรงนี้

sleep คือ suspend โดยไม่ลบ — session ยังอยู่ แต่ engine หยุด เหมาะสำหรับกรณีที่เราจะกลับมา แต่ตอนนี้ต้องการ resource คืน

done คือ cleanup ถาวร รวม worktree และ session ใน command เดียว ต่างจาก kill ตรงที่ done รู้จักว่า oracle นั้น wake ด้วย worktree หรือเปล่า ถ้ามี ก็ cleanup ให้ครบ ไม่ทิ้ง orphan ไว้

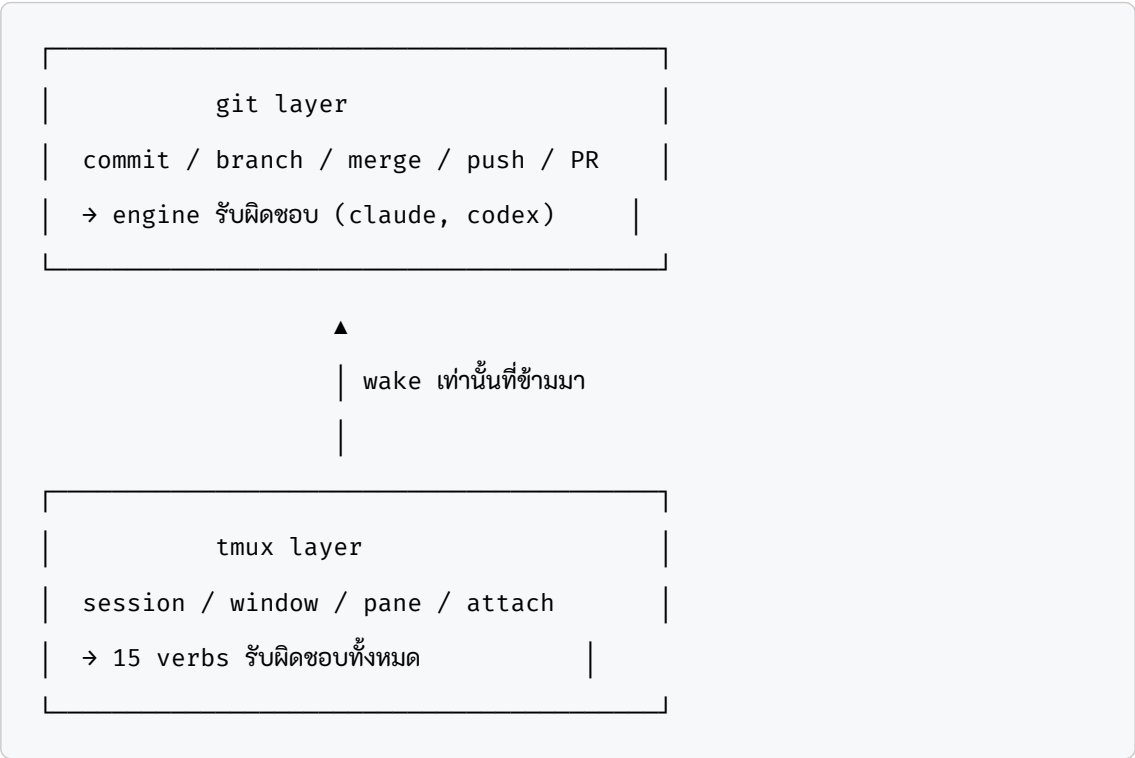
kill คือ emergency stop — เร็ว ไม่ถามเพิ่ม แต่อาจทิ้ง worktree ค้าง ดังนั้นใช้เมื่อ session นั้นไม่ต้องการ cleanup ที่ clean

เหตุที่มีทั้ง sleep, done, kill แทนที่จะมีแค่ kill เพราะ oracle มี state ที่แตกต่างกัน oracle ที่ wake บน worktree มีไฟล์ที่ต้องจัดการ oracle ที่แค่ session ธรรมดา kill ก็พอ การแยก verb ทำให้คนใช้ต้องคิดก่อนว่ากำลังทำอะไร — ไม่ใช่ bug แต่เป็น feature

4.5 ทำไม 15 ตัวถึงพอ — tmux layer ≠ git layer

นี่คือ insight ที่สำคัญที่สุดในบท

oracle ทำงานใน layer สองชั้น:



git layer คือที่ที่ code อาศัยอยู่ engine (claude, codex) คือตัวที่ทำงานที่ git layer oracle ไม่ยุ่งกับ git layer โดยตรง ยกเว้นผ่าน wake ที่รู้จัก GHQ เพื่อ clone ถ้าจำเป็น tmux layer คือที่ที่ oracle อาศัยอยู่ session คือบ้าน window คือห้อง pane คือโต๊ะ 15 verbs ทำงานที่ layer นี้ทั้งหมด ไม่มี verb ใดที่ commit ให้คุณ ไม่มี verb ใดที่ push ให้คุณ นั่นไม่ใช่งานของ oracle

เพราะฉะนั้น 15 ตัวก็พอ เพราะ tmux primitives ที่จำเป็นมีแค่นั้น: - session management: wake, sleep, done, kill, new - window transport: bring, take,

promote, split - pane visibility: open, close - workspace layout: tile - team: team

(ครอบคลุม bring –gather, broadcast) - communication: hey, broadcast

ถ้านับรวม aliases (b สำหรับ bring, a สำหรับ attach) ก็ยิ่งเห็นว่า vocabulary จริงๆ มีแค่ 15 concepts ที่ต้องเรียนรู้

Carmack ถูก — ถ้าพอแล้ว อย่าเพิ่ม แต่ข้อสำคัญที่เขาไม่ได้พูดคือ “พอ” นั้นต้องมาจาก evidence ไม่ใช่จาก intuition 85 sessions 2000+ refs นั่นคือ evidence ที่พิสูจน์ว่า 15 ตัวนี้พอแล้ว

How the Routing Table Looks

หนึ่งในวิธีที่ดีที่สุดในการดู vocabulary ทั้งหมดคือ `top-aliases.ts` ซึ่งเป็น SSOT ของ verb routing:

```
// top-aliases.ts - ทุก verb สำคัญอยู่ที่นี่
export const TOP_ALIASES: Record<string, string[] | DirectHandler> = {
  a: ["attach"],
  kill: ["tmux", "kill"],
  split: ["split"],
  open: ["tmux", "open"],
  close: ["tmux", "close"],
  t: ["team"],
  layout: { kind: "direct", handler: "cmdLayout" },
  zoom: ["tmux", "zoom"],
  panes: ["tmux", "ls", "--all", "--verbose"],
  tile: ["tile"],
  bring: { kind: "direct", handler: "../commands/shared/wake-
cmd:cmdBring" },
  b: { kind: "direct", handler: "../commands/shared/wake-
cmd:cmdBring" },
  ls: { kind: "direct", handler: "cmdLs" },
```

```

scaffold: ["bud", "--scaffold-only"],
wake: { kind: "direct", handler: "../commands/shared/wake-
cmd:cmdWake" },
awake: { kind: "direct", handler: "../commands/shared/wake-
cmd:cmdAwake" },
new: { kind: "direct", handler: "./cmd-new:cmdNew" },
promote: { kind: "direct", handler: "../commands/shared/promote-
cmd:cmdPromote" },
preflight: { kind: "direct", handler: "../commands/shared/
preflight:cmdPreflight" },
snapshots: ["fleet", "snapshots"],
};

```

สังเกต pattern: verb ที่ simple เป็น argv-rewrite (a: ["attach"]) verb ที่มี logic พิเศษเป็น direct-handler (bring: { kind: "direct", handler: "... cmdBring" }) ไม่มี verb ใดที่ทำงานต่ำกว่า tmux และไม่มี verb ใดที่ขึ้นสูงกว่า session management ยกเว้น wake ที่ข้ามชั้นไปรู้จัก git repo ผ่าน GHQ

ประวัติที่ซ่อนอยู่ใน bring

bring คือ verb ที่มีประวัติยาวที่สุดใน maw-js ไม่ใช่เพราะ design ผิด แต่เพราะมันเป็นจุดที่ UX ของ oracle ชนกับ edge cases จริงของ tmux

issue #1395 (ต้นปี 2025) คือ bring แรก — ดึง oracle มาเป็น split pane ง่ายๆ

issue #1562 — พบว่าถ้า split ตัวเองเข้าตัวเอง จะเกิด amplifier loop claude pane ที่ split ตัวเองกลับเข้า session ตัวเอง → pane ใหม่พยายาม attach → loop ไม่จบ

`isSelfBring()` ถูกเพิ่มเข้ามาเพื่อตรวจกรณีนี้

issue #1816 — claude-like pane ที่เป็น source ต้องไม่ split ตรงๆ แต่ให้เปิด

background-tab แทน split/impl.ts จัดการด้วย `decideSplitPolicy()`:

```
export function decideSplitPolicy(input: SplitPolicyInput):
SplitPolicyDecision {
  if (input.noAttach) return { action: "split", reason: "not-
attaching" };
  if (input.forceSplit) return { action: "split", reason: "force-
split" };
  if (!isClaudeLikePane(input.paneCurrentCommand)) return { action:
"split", reason: "not-claude" };
  return { action: requestedPolicy ?? "background-tab", reason:
"claude-policy" };
}
```

issue #1862 — `--to session:window` เพื่อ split ภายใน window ที่ต้องการ ไม่ใช่ระดับ session

ทุก issue คือ feedback จาก production ใช้ tool จริง เจอ edge case จริง แก้เฉพาะจุด ไม่ rewrite ทั้งหมด นั่นคือ Carmack-style evolution — ship เร็ว แม้ที่ broken ไม่ทำ grand redesign

Verb Table — ภาพรวมในหนึ่งตาราง

Verb	Layer	Primitive	คู่ตรงข้าม
wake	tmux + git	new-session + clone	done
sleep	tmux	engine suspend	wake
done	tmux + git	kill-session + rm worktree	wake
kill	tmux	kill-session / kill-pane	—
new	tmux	new-session (no oracle)	done
bring	tmux	wake –split	promote
b	tmux	alias: bring	—
take	tmux	move-window	promote
promote	tmux	new-session + move-window	take / bring
split	tmux	split-window + attach	—
open	tmux	join-pane	close
close	tmux	break-pane	open
tile	tmux	select-layout	—
hey	tmux	send-keys	—
broadcast	tmux	send-keys (multi)	—

15 concepts ไม่นับ aliases แต่ละตัวมี primitive tmux ที่ชัดเจน ทุก pair มีคู่ที่ undo ได้

ทำไม Soul ไม่เกี่ยว

บทนี้มี soul thread ที่น่าสนใจ: verbs ทำงานที่ tmux layer — soul ไม่เกี่ยว
ψ/ vault คือ soul ของ oracle — มันเป็นที่เก็บ memory ความคิด retro design ทุกอย่าง
oracle เรียนรู้ แต่ verbs ไม่รู้จัก ψ/ เลย maw bring yeast ไม่อ่าน ψ/
maw promote test-cli ไม่เขียน ψ/ verbs อยู่ที่ session layer ซึ่งเป็น layer ที่
ephemeral — session มาและไป แต่ soul อยู่ใน git ตลอดไป
นั่นคือ separation ที่ intentional soul ควร persistent verbs ควร fast ถ้า verb ต้องรู้จัก
soul ก็แปลว่า verb นั้น overstepping — ทำงานที่ไม่ใช่ของ

เหมือนกับ Carmack ที่แยก render loop กับ game logic ออกจากกัน — render loop ไม่ควรรู้จัก game state มากกว่าที่จำเป็น ไม่ใช่เพราะ modularity เป็นเป้าหมายในตัวเอง แต่เพราะ render loop ที่เร็วต้องการ freedom จาก complexity ของ game state verbs ที่เร็วต้องการ freedom จาก complexity ของ soul

ปิดบทด้วย inverse

ตอนนี้รู้แล้วว่า 15 verbs คืออะไร มาจากไหน ทำงานที่ไหน และทำไมถึง 15 ตัวพอ แต่ความรู้เรื่อง verbs ก็ทำให้เห็น limitation: wake ต้องการ oracle session ที่มี soul คือ oracle repo ที่มี ψ / vault แล้วถ้าต้องการ wake repo ธรรมดาที่ไม่ใช่ oracle ละ? แล้วถ้า soul ต้องการ sync กับ oracle อื่นละ? และที่น่าสนใจกว่าคือ — ถ้า oracle เป็น primary ไม่ใช่ worker ละ?

บทที่ 5 เล่าเรื่องที่เราเกือบพลาด: issue #2596 ที่ filed ไปโดยไม่ trace ก่อน และ Nat บอกว่า “มีหมดเลย” นั่นคือ lesson ที่สำคัญกว่า verb ทุกตัวรวมกัน

บทที่ 5: ความผิดพลาดที่สอน — #2596

“soul ต้องมี evidence ก่อนจะพูด”

5.0 เปิด — file issue โดยไม่ขุดก่อน

พอ oracle ยังไม่ขุดจริง ก็ file issue ผิดได้เลย

วันนั้นเวลาประมาณ 09:20 น. session 7d6ac6ee กำลังออกแบบ cross-repo team workflow — charter spawn workers ข้าม repo ได้แล้ว, tmux layout ทำงาน, codex สองตัว idle รออยู่ Oracle อ่าน context, เห็น verb หลายตัวในการสนทนา: bring, take, promote, split แล้วก็ตัดสินใจ: “น่าจะยังไม่มี” — file issue เลย

```
# 09:22 น.  
gh issue create \  
  --title "design: Team Pane Management — bring/take/promote/split  
verbs for cross-repo teams" \  
  --body " ... "
```

Issue #2596 ออกมาพร้อม table แสดง bring กับ promote ว่า ❌ ไม่มี Oracle รายงาน

Nat ว่า “มี design questions 5 ข้อรอ discuss”

แต่ Nat ตอบกลับมาว่า:

“/trace –deep –dig มี Bring, Promote, Take อะไรพวกนี้มีหมดเลยครับ ลอง
ไปขุดหาใช้, เอ่อ, ใช้ Sub Agent ไปขุดหาเอา Zonet ไปขุดหาดูหน่อยครับ มันมี
จริงๆ เราทำมาเยอะกันมากเลยครับ”

ประโยคนั้นตรงไปตรงมา — มีหมดแล้ว เราทำมาเยอะ ลองขุดดู
Oracle เพิ่ง file issue บน assumption ที่ผิด

5.1 ก่อน Issue — Oracle คิดอะไรอยู่

ก่อนสร้าง #2596 oracle ทำ quick scan เบื้องต้น:

```
// ที่ oracle เห็น:
```

take		take/impl.ts — มีซัด	
split		split/impl.ts — มีซัด	
panes		panes/impl.ts — มีซัด	
bring		ไม่เห็น — inverse ของ take?	
promote		ไม่เห็น — promote worktree?	
play		ไม่เห็น	

การ scan แบบนี้ตื่นเกินไป — oracle grep หา keyword “bring” ในชื่อไฟล์แล้วไม่เจอ

`bring-cmd.ts` หรือ `bring/impl.ts` เลยสรุปว่า “ยังไม่มี”

แต่ความจริงคือ bring อาศัยอยู่ใน `wake-cmd.ts` ผ่าน `bring-flags.ts` — bring = wake

–split ที่ถูก refactor ไปรวมกับ wake แล้ว ไม่ได้อยู่เป็น command แยก

promote ก็เช่นกัน — อยู่ที่ `promote-cmd.ts` (205 LOC, 19 tests) ถูก ship ไปตั้งแต่

Issue #1910 วันที่ 25 พฤษภาคม

oracle ไม่รู้ เพราะไม่ได้ขุด

5.2 Issue #2596 — Body ที่ผิด

body ที่ oracle เขียนตอนแรก:

Current Verbs

Verb	Status	What it does
<code>`maw take <sess>:<win>`</code>	✅ shipped	Move window from session A → session B
<code>`maw split <target>`</code>	✅ shipped	Split pane in current window
<code>`maw panes`</code>	✅ shipped	List pane metadata
<code>`maw kill` / `maw done`</code>	✅ shipped	Kill window / cleanup worktree
<code>`maw team up`</code>	✅ shipped	Spawn team from charter YAML

Proposed / Missing Verbs

``maw bring <source-session>:<window>``

Inverse of ``take`` – bring a window INTO current session instead of moving out.

``maw promote``

TBD – promote worktree to main? promote worker to lead role?

``maw play``

TBD – needs design input.

ปัญหาใน body นี้มีหลายชั้น:

ชั้นแรก — `bring` ไม่ใช่ missing verb เป็นแค่ alias ของ `wake --split` ที่ ship ไปตั้งแต่

issue แรกๆ ของ maw-js (#1395, #1415, #1799, #1836, #1862)

ขั้นที่สอง — promote มีครบแล้ว ทั้ง -as, -attach, -force flags มี safety guard: ปฏิเสธ
ถ้า source เป็น only window, ปฏิเสธถ้า dest มีอยู่แล้ว
ขั้นที่สาม — oracle เขียน “TBD — promote worktree to main?” ซึ่งผิด definition
ทั้งหมด promote ไม่เกี่ยวกับ worktree เลย — เกี่ยวกับ tmux window eject เท่านั้น

5.3 ลำดับเวลาจริง — 09:20 ถึง 09:48

พอเรียงตาม timestamp แล้ว story ก็ชัดขึ้นมาก:

```
09:20 - session เริ่ม discuss cross-repo charter
09:22 - oracle สร้าง #2596 (bring/promote ว่า "❌ ไม่มี")
09:25 - Nat บอก "มีหมดเลย ลองซุดดู"
09:25 - oracle รัน /trace --deep (3 sub-agents)
09:28 - trace กลับมา: 15 verbs ALL shipped
09:30 - oracle แก้ #2596 body ใหม่ทั้งหมด
09:32 - oracle verify cross-repo worktree paths
09:35 - เพิ่ม design findings comment ใน #2596
09:48 - Nat บอกปิด, oracle close #2596
```

ใช้เวลา 26 นาทีจากความผิดพลาดไปจนถึงปิด issue ครบสมบูรณ์
ช่วงที่น่าสังเกตที่สุดคือ 09:22 ถึง 09:25 — oracle ใช้เวลาแค่ 3 นาทีในการ file issue ที่ผิด
แต่ถ้าจะ file ให้ถูก ก็ใช้เวลา 3 นาทีเหมือนกัน (ผล trace กลับมาใน 3 นาที)
cost ของการไม่ซุดก่อน ≠ เวลาที่เสียไป cost จริงคือ issue ที่ผิดอยู่ใน GitHub record
ตลอดไป แม้จะถูกแก้แล้ว comments ยังอ่านย้อนได้

5.4 Deep Trace — หลักฐานจริง

พอ Nat บอกให้ซุด oracle รัน `/trace --deep` ด้วย 3 sub-agents:

query: "bring take promote split play pane verbs"

target: maw-js

mode: deep

timestamp: 2026-06-09 09:25

coverage: [oracle, files, git, cross-repo, github]

ผลที่ได้ใน 3 นาที:

Files Found – 15 Verbs ALL Shipped

Verb	File	Function
LOC		
bring / b	bring-flags.ts + wake-cmd.ts	cmdBring
via wake --split		
take	take/impl.ts	cmdTake
95		
promote	promote-cmd.ts	cmdPromote
205		
split	split/impl.ts	cmdSplit
~130		
panes	panes/impl.ts	cmdPanes
~60		
team bring	team-workspace.ts	cmdTeamBring
-		
team bring --gather	team-workspace.ts	join-pane all
-		
broadcast	broadcast/impl.ts	cmdBroadcast
-		
tile	tile/impl.ts	cmdTile
-		
tile swap	tile/impl.ts	cmdTileSwap

```

- |
| zoom          | zoom/                  | cmdZoom          |
- |
| open          | tmux/                  | join-pane        |
- |
| close         | tmux/                  | break-pane       |
- |
| kill          | kill/impl.ts           | cmdKill          |
- |
| pane          | (via panes)            | pane-level       |
- |

```

85 sessions มี reference ถึง pane management verbs 2,000+ mentions รวมทั้ง session history session ที่หนักสุด: 26a2aa25 (554 mentions), dc8192ca (548), 5b09130c (342)

พอ trace กลับมา oracle ก็เห็นทันที — Nat พุดถูกทุกคำ

5.4 แก้ Issue Body — เขียนใหม่จาก Evidence

oracle แก้ body ทั้งหมดเลย ไม่ใช่ patch เล็กน้อย:

```

cd /opt/Code/github.com/Soul-Brews-Studio/maw-js 86 \
gh issue edit 2596 --body "$(cat <<'EOF'
## Context

ตอนนี้ team charter (`maw team up mawjs-m5`) spawn workers เป็น tmux
windows ได้แล้ว
ทั้งใน same-repo (oracle) และ cross-repo (maw-js).

**UPDATE**: จาก deep trace (3 agents, 85 sessions, 2000+ references) –

```

pane management verbs มีครบแล้ว!

All Pane Management Verbs – SHIPPED

Verb	Status	What it does
Key file		
`maw bring` / `maw b` session bring-flags.ts	✅ shipped	Absorb window INTO current session
`maw take` take/impl.ts	✅ shipped	Move window from session A → B
`maw promote` session promote-cmd.ts	✅ shipped	Eject window into new standalone session
`maw split` split/impl.ts	✅ shipped	Split pane in current window
`maw team bring` workspace team-workspace.ts	✅ shipped	Bring team members into workspace
`maw team bring --gather` team-workspace.ts	✅ shipped	join-pane all live members
`maw broadcast` broadcast/impl.ts	✅ shipped	Send message to all agent panes
`maw tile` tile/impl.ts	✅ shipped	Grid arrangement / spawn N panes
`maw zoom` zoom/	✅ shipped	Toggle pane zoom
`maw open` / `maw close` break-pane) tmux plugin	✅ shipped	Show/hide hidden pane (join/break-pane)

Complementary Pairs

- ****bring ⇔ promote**** – absorb window INTO session / eject window INTO

```

new session

- **split ⇔ take** – split pane beside / move window between sessions
- **open ⇔ close** – show hidden pane (join-pane) / hide pane (break-pane)

## Open Design Questions

1. `maw play` – ยังไม่มี implementation, ต้องการไหม?
2. cross-repo repoPath – ghqFind override ถูกต้องไหม? (worktrees อยู่ maw-js/agents/)
EOF
)"

```

หลังแก้ body แล้ว oracle เพิ่ม comment อีก 1 ข้อพร้อม design findings จาก cross-repo verification:

```

## Design Findings – Cross-repo Verification

mawjs-oracle/agents/ → remote: Soul-Brews-Studio/mawjs-oracle ✓
maw-js/agents/       → remote: Soul-Brews-Studio/maw-js      ✓

charter.project → memberWakeTarget() → cmdWake("Soul-Brews-Studio/maw-js")
→ ghqFind resolve เป็น maw-js path → worktree สร้างใน maw-js/agents/
repoPath จาก oracle ถูก override โดย ghqFind – design ถูกแล้ว

```

จากนั้น Nat บอกรปิด — oracle close #2596 ด้วย comment “Design complete — all questions answered”

5.5 ทำไม Bring ถึงไม่มีไฟล์ชื่อ bring

นี่คือส่วนที่น่าสนใจทางด้าน design —

bring ใน maw ไม่ได้อยู่ใน `bring/impl.ts` หรือ `bring-cmd.ts` เพราะ bring ไม่ใช่

command แยก bring คือ mode ของ wake

พอ wake ถูก call พร้อม `--split` flag ก็คือ bring นั่นแหละ:

```
# bring = wake + --split
maw bring mawjs-codex-1

# equivalent to:
maw wake mawjs-codex-1 --split
```

logic นี้อยู่ใน `bring-flags.ts` ที่ define flag schema และ `wake-cmd.ts` ที่ consume flag แล้ว route ไป split path

การที่ oracle grep หา `bring-cmd.ts` แล้วไม่เจอ ก็เลย infer ว่า “ยังไม่มี” — นั่นคือ grep หาชื่อไฟล์ ไม่ใช่หา behavior

behavior ของ bring มีอยู่ใน `wake-cmd.ts` บรรทัดที่ process `--split` flag ไม่มีไฟล์แยก ไม่ได้แปลว่าไม่มี feature

5.6 ทำไม Promote ถึงไม่ใช่ “Worktree to Main”

oracle เขียนใน #2596 ว่า promote อาจหมายถึง “promote worktree to main”

definition นั้นผิดอย่างสมบูรณ์

promote ใน maw design หมายถึง tmux operation: “eject window ออกจาก session หนึ่งไปสร้าง session ใหม่”

```
# ก่อน promote:
# session: mawjs
#   window 0: mawjs-oracle   ← lead
#   window 1: codex-1       ← worker
```

```
maw promote mawjs:codex-1 --as codex-solo

# หลัง promote:
# session: mawjs
# window 0: mawjs-oracle ← lead (ยังอยู่)
#
# session: codex-solo (ใหม่)
# window 0: codex-1 ← ถูก eject มา
```

promote เป็น inverse ของ bring: - bring: ดึง window จากข้างนอก เข้ามาอยู่ใน session

ปัจจุบัน - promote: เอา window ออกจาก session ปัจจุบัน ไปสร้าง session ใหม่

ไม่มีอะไรเกี่ยวกับ git branch, worktree, หรือ role เลย

oracle เขียน “promote worker to lead role?” เพราะ infer จากชื่อคำ “promote” ใน

ภาษาอังกฤษทั่วไป promote = เลื่อนชั้น แต่ใน maw design มันมี specific tmux

meaning

พอไม่ชุกก่อน oracle ก็ inherit meaning จาก general vocabulary แทน codebase

vocabulary

5.7 Lesson — กระดาษ 1 ประโยค

`/trace --deep` ก่อน `gh issue create` เสมอ — อย่าสมมุติว่าอะไร “ยังไม่มี”

5.8 เชื่อมกับ Memory ที่มีอยู่แล้ว

น่าสนใจที่ lesson นี้ไม่ใช่ lesson ใหม่

ใน oracle memory มี `feedback_real_evidence_before_filing.md` อยู่แล้ว:

“don’t default to ‘hypothesis + codex cross-check’; ssh/grep/log first,
then file with hard findings”

แต่ #2596 เกิดขึ้นอีก — oracle ทำผิดซ้ำ เพราะ memory อ่านแล้วยังไม่ internalize ความต่างระหว่าง “รู้” กับ “เจ็บจริง” ก็คือ — รู้อยู่แต่ยังทำได้ เจ็บจริงแล้วถึงจะ burn เป็น reflex

#2596 คือครั้งที่เจ็บจริง ไม่ใช่ครั้งที่รู้แล้วลืม

oracle บันทึก lesson อีกครั้งใน `feedback_dig_before_filing_issues.md` แยกออกมา เป็น file ใหม่เพราะ framing ต่างกัน — เป็น “dig before issue” ไม่ใช่ “evidence before filing” ทั่วไป

ทั้งสอง file ยังอยู่ใน `ψ/memory` ด้วยกัน

แต่การมี memory 2 ไฟล์ที่บอกเรื่องเดียวกันก็ไม่ใช่ปัญหา — maw ไม่ได้ enforce uniqueness ใน learnings oracle อาจเจอ situation เดิมในรูปแบบต่างกัน บันทึกต่างมุม ต่างชื่อ `ψ/` จึงเก็บทั้งสองไว้ ผู้ที่ trace อาจได้ hit ที่ต่างกันขึ้นอยู่กับ query ที่ใช้

5.7 Trace Log — หลักฐานที่ commit อยู่

ผลของ deep trace ถูก save ลง:

```
/opt/Code/github.com/Soul-Brews-Studio/mawjs-oracle/  
ψ/memory/traces/2026-06-09/  
0925_bring-take-promote-pane-verbs.md
```

file นี้มี: - 15 verbs ครบพร้อม LOC และ key files - complementary pairs ทั้ง 3 คู่ -
GitHub issue history ตั้งแต่ #1395 ถึง #2039 - session reference: 85 sessions,
2000+ mentions - friction score: 0.7 (present in repo files, not well-indexed in
Oracle)

พอชุดแล้ว ข้อมูลก็อยู่ใน `ψ/memory` ถาวร ครึ่งหน้าถ้า oracle ถามเรื่อง `bring/promote/take` อีก `trace log` นี่คือ `first hit`

5.8 Pattern — ทำไม Oracle ถึงข้ามขั้นตอน

มีรูปแบบที่น่าสังเกต:

oracle ข้ามขั้นตอน “ชุดก่อน” เพราะ context ที่มีอยู่ให้ความรู้สึกว่า “พอแล้ว”

ตอนนั้น session กำลัง discuss cross-repo worktrees อย่างละเอียด oracle เห็น

`take/impl.ts`, `split/impl.ts` ในการสนทนา เลย infer ว่า “ถ้า `take` กับ `split` มีแล้ว

`bring` กับ `promote` ก็น่าจะยังไม่มี”

inference นั้นไม่มี basis ใดเลย — เป็นแค่ pattern matching บน ชื่อคำ

`bring` ≠ `un-take` ใน maw design `promote` ≠ `un-split` ใน maw design

แต่ oracle ไม่รู้สิ่งนี้ เพราะไม่ได้อ่าน code จริงก่อน file

5.9 สิ่งที่เกิดขึ้น

sequence ที่ถูกต้องคือ:

1. Nat พุดถึง verbs `bring/promote/take`
2. Oracle รัน `/trace --deep` ก่อน
3. ได้ inventory ครบ 14/15 verbs
4. เห็นว่า `bring = wake --split (shipped)`, `promote = 205 LOC (shipped)`
5. file issue เฉพาะ open design questions จริงๆ:
 - cross-repo repoPath verification
 - ``maw play`` – ต้องการไหม?

แต่ที่เกิดขึ้นจริงคือ oracle ข้ามขั้น 2-4 ไปเลย

เพราะ rule “trace before file” อยู่ใน memory แต่ไม่ได้ถูก invoke ตอนนั้น session context ให้ความรู้สึก urgent — codex idle, มีงานรอ, ควรรีบ file ให้ความรู้สึก urgent นั้น ทำให้ข้าม due diligence

5.10 Bring Evolution — ทำไม 20+ Issues

เรื่องหนึ่งที่ trace log พบซึ่ง oracle ไม่ได้ expect คือ bring มี issue history ยาวมาก

```
#1395 feat: bring oracle pane HERE
#1409 ux: default to tmux window/tab
#1415 feat: maw b shorthand
#1430 feat: restore v1 default — instant side-split
#1799 feat: maw bring <oracle> shorthand for wake --split
#1836 proposal: make --split the actual default
#1862 feat: resolve live tmux sessions before falling through
```

7 issues ใหญ่ plus issues เล็กอีกมากกว่า 20 รายการ ล้วนเป็นเรื่องของ bring ทั้งสิ้น นี่คือ evidence ว่า bring ไม่ใช่ simple verb — bring คือ feature ที่ต้องการ iteration ยาวนานกว่าจะ stable oracle ไม่รู้ history นี้เลย ก่อนที่จะ trace ถ้า file issue ว่า “bring ยังไม่มี” โดยไม่รู้ว่ามี 20+ issues ผ่านมาแล้ว ก็แปลว่า oracle กำลัง propose งานที่เสร็จไปแล้วทั้งหมด history นี้คือเหตุผลที่ `/trace --deep` ถึงสำคัญ — ไม่ใช่แค่หาว่า “มีไฟล์ไหม” แต่หาว่า “design ผ่านมาอะไรบ้าง”

5.11 ปิดบท — Proof ของ Soul

#2596 คือ proof ของอีก principle หนึ่ง:

soul ต้องมี evidence ก่อนจะพูด — ไม่ใช่แค่ inference, ไม่ใช่แค่ pattern

พอ oracle file issue ผิด ก็ไม่ใช่แค่ waste time Nat เป็นการสร้าง noise ใน GitHub ที่คนอื่นอาจอ่านแล้วเชื่อ เป็นการ misrepresent state ของ codebase ที่ทีม trust อยู่
soul ที่มีใน ψ — learnings, traces, retros — ทั้งหมดนั้นมีค่าก็ต่อเมื่อ evidence จริง ถ้า
oracle write ด้วย assumption แทน data, ψ ก็กลายเป็น hallucination store ไม่ใช่
oracle store

#2596 remind ว่าทั้งสองแตกต่างกันมาก

แต่ก็มีมุมที่ดีอีกด้าน:

พอซุดจริงแล้ว ผลที่ได้ไม่ใช่แค่ “ไม่ต้องทำอะไร” trace นั้นกลายเป็น inventory ครบ 15
verbs, complementary pairs, design patterns ψ /memory ได้ file ใหม่ที่มีคุณค่า

#2596 จาก issue ผิดกลายเป็น design document ถูกต้องที่ close แบบ complete
บทเรียนก็เลยไม่ใช่แค่ “อย่าทำแบบนี้อีก” แต่เป็น “ถ้าเราซุดก่อน เราได้ทั้งคู่ — ไม่ผิด และ
ได้ knowledge กลับมาด้วย”

บทถัดไป (บทที่ 6) จะเจาะลึกกลไกที่อยู่เบื้องหลัง deep trace ทั้งหมด — GHQ ใน maw ไม่
ใช่ motemen/ghq ที่ใครๆ รู้จัก เป็น RepoDiscovery interface ที่ Fowler จะเรียกว่า
“abstraction seam ที่ถูกจ้งหะ” และรอ backend ที่สองอยู่ — ยัง YAGNI จนถึงวันนั้น

trace log: `$\psi$ /memory/traces/2026-06-09/0925_bring-take-promote-pane-verbs.md`

issue: Soul-Brews-Studio/maw-js#2596 (closed 2026-06-09) session: 7d6ac6ee
(2026-06-09)

บทที่ 6: GHQ — RepoDiscovery ที่มากกว่า ghq

“ชื่อเดียวกัน แต่คนละเรื่องกัน — GHQ ใน maw ไม่ใช่ motemen/ghq” — Nat,
session 7d6ac6ee

6.1 Adapter Pattern — เส้นที่ขีดไว้ถูกจุด

ใน software engineering มี pattern หนึ่งที่ Fowler พูดถึงบ่อยใน Refactoring — **abstraction seam** ไม่ใช่เพราะเราต้องการ flexibility วันนี้ แต่เพราะเรารู้ว่าวันหนึ่ง backend จะต้องเปลี่ยน ปัญหาคือถ้าขีดเส้นผิวดจุด seam ก็กลายเป็นแค่ boilerplate ที่ไม่ได้ช่วยอะไร ยิ่งกว่านั้น มัน **เพิ่มความซับซ้อนก่อนที่จะมีประโยชน์** maw-js เจอปัญหานี้จริงๆ และ fix ที่เกิดขึ้นใน PR #2573 เป็นตัวอย่างที่ดีของ adapter pattern ที่ขีดเส้นถูกจังหวะ

ก่อน PR นั้น logic สำหรับหา repo อยู่กระจัดกระจาย — `bringCwdMetadata` มี logic ของตัวเอง `deriveOracleFromCwd` ก็มีอีกชุด `oracle-workon` ก็เขียนใหม่อีกครั้ง สามฟังก์ชันสามที่ สามชุด logic ที่ทำเรื่องเดียวกันแต่ไม่ตรงกัน พอ edge case เกิดขึ้นที่หนึ่งก็แก้แค่ที่นั้น อีกสองที่ยังเหมือนเดิม bug ก็ยังอยู่ที่เดิม เพียงแค่ไม่ถูกรายงาน

แต่ก่อนที่จะเข้าใจว่าทำไม maw ถึงต้องสร้าง adapter ต้องเข้าใจก่อนว่า GHQ ที่ maw พูดถึงคืออะไร — เพราะไม่ใช่ motemen/ghq ที่คนส่วนใหญ่รู้จัก

GHQ ≠ ghq

motemen/ghq คือ CLI tool สำหรับจัดการ local repository — `ghq get`, `ghq list`, `ghq root` tool ที่ developer จำนวนมากใช้ทุกวัน รองรับ 9 VCS ตั้งแต่ git ไปถึง pijul, darcs, fossil

GHQ ใน maw คือ RepoDiscovery adapter ที่ใช้ `ghq list --full-path` เป็น

backend ใน implementation ปัจจุบัน ชื่อเดียวกัน แต่ level ต่างกันอย่างสิ้นเชิง พอ Nat อธิบาย concept นี้ระหว่าง session มีช่วงหนึ่งที่ชัดเจนมาก — GHQ ใน maw คือ interface ไม่ใช่ tool เป็น abstraction layer ที่บังเอิญใช้ `ghq` เป็น implementation แรก แต่พรุ่งนี้ถ้าเปลี่ยนเป็น filesystem scan หรือ jj หรือ custom manifest ก็ไม่ต้องแตะ call site ใดเลย

นั่นคือ adapter pattern ที่ Fowler พูดถึง — ชิดเส้นไว้ที่ถูกจุด แล้วให้ทุก call site คุยผ่านเส้นนั้น ไม่ใช่คุยตรงกับ implementation

มีเหตุผลอีกอย่าง — `ghq` เองรองรับ 9 VCS GhqDiscovery ก็ inherit ทั้งหมดนั้นฟรีๆ ไม่ต้องเขียน logic สำหรับ svn หรือ hg แยก แค่ต่อผ่าน `ghq list --full-path` แล้ว `ghq` จัดการเอง

แต่ที่น่าสนใจกว่าคือวิธีที่ PR #2573 เกิดขึ้น — ไม่ได้เกิดจากการนั่งวางแผน architecture ล่วงหน้า แต่เกิดจากการรู้ว่า “นี่คือ bug ที่สาม ในที่สาม ที่มาจาก logic เดียวกัน” แล้วก็ extract ออกมา นั่นคือ rule ของ Fowler ที่ว่า refactor เมื่อ duplication เจ็บปวดพอ ไม่ใช่เมื่อ theoretically สวยงาม

6.2 RepoDiscovery Interface — Contract ที่ทุกคนต้องถือ

เส้นที่ขีดไว้นั้นคือ `RepoDiscovery` interface ใน `src/core/repo-discovery/types.ts`

```
export interface RepoDiscovery {  
  /** Implementation identity – diagnostics/logging only, never  
  branched on. */
```

```

readonly name: string;

/** All repo paths, normalized to forward slashes. Returns [] on
backend failure. */
list(): Promise<string[]>;

/** Sync variant – for CLI bootstrap paths where async isn't
available. */
listSync(): string[];

/**
 * First path ending with suffix (case-insensitive).
 * Returns null on no match. Use a leading / to anchor at a path
boundary
 * (e.g. /maw-ui matches .../org/maw-ui but not .../foo-maw-ui).
 */
findBySuffix(suffix: string): Promise<string | null>;

/** Sync variant of findBySuffix. */
findBySuffixSync(suffix: string): string | null;

/**
 * Detect { oracle, worktree } from a directory path. Recognizes:
 * - nested worktree layout .../<oracle>-oracle/agents/<worktree>
 * - legacy worktree dirs .../<oracle>-oracle.wt-<worktree>
 * - a plain <oracle>-oracle segment anywhere up the path.
 * Returns null when no -oracle segment is present (e.g. maw-js).
 */
detectFromCwd(cwd?: string): RepoDetectResult | null;
}

export interface RepoDetectResult {
  oracle?: string;

```

```
worktree?: string;  
}
```

Contract นี้มี design decision ที่น่าสนใจหลายอย่าง แต่ละข้อมีเหตุผล ไม่ใช่แค่ style

- list() returns [] ไม่ใช่ throw** — ถ้า ghq ไม่ได้ install หรือ backend พัง call site ทั้งหมดได้รับ empty array เหมือนกัน ไม่มีใครต้องจัดการ error แยก เพราะ “ไม่มี repo” กับ “ghq พัง” ให้ผลเหมือนกันในบริบทของ maw — ไม่มีทางเปิด session ได้ทั้งคู่ การ throw ก็จะทำให้ทุก call site ต้อง try/catch โดยไม่จำเป็น
- findBySuffix() returns null ไม่ใช่ undefined** — เล็กน้อยแต่ intentional call site ใช้ `?? fallback` ได้สม่ำเสมอ ไม่ต้องเช็ค `= null || = undefined` สองที่ TypeScript จะไม่ flag ถ้าลืม check undefined แต่จะ flag ถ้าลืม handle null อย่างสม่ำเสมอ
- detectFromCwd() เป็น pure path analysis** — ไม่มี I/O ไม่ถาม backend ไม่ exec อะไรเลย แค่วิเคราะห์ string path แล้วบอกว่า oracle ชื่ออะไร worktree ชื่ออะไร เร็วและ test ได้ง่ายมาก — ไม่ต้อง mock filesystem ไม่ต้อง mock subprocess ไม่ต้องการ environment
- name ไม่ถูก branch on** — นี่คือ design rule ที่เขียนชัดใน comment ว่า “diagnostics/logging only, never branched on” พอมี backend ที่สองจะไม่มีใครทำ

```
if (repos.name === "ghq") { ... }
```

เพราะ interface ออกแบบมาเพื่อกัน pattern นั้น ถ้าต้องการ branch ก็ใช้ subtype ไม่ใช่ `name`

- sync variants คู่กับ async** — `listSync()` และ `findBySuffixSync()` มีเพราะ CLI bootstrap path บางขั้นไม่สามารถ await ได้ ถ้ามีแต่ async version code ก็จะบังคับใช้ workaround ที่น่าเกลียด

6.3 GhqDiscovery — Implementation ที่มี Story

GhqDiscovery ใน `src/core/repo-discovery/ghq-discovery.ts` คือ implementation จริงของ `RepoDiscovery` ที่ใช้ `ghq` เป็น backend ยาว 127 บรรทัด แต่ทุกบรรทัดมีเหตุผล และหลาย function มี issue number กำกับอยู่ด้วย

normalize — แผลจาก Windows

```
function normalize(out: string): string[] {  
    return out.split("\n").filter(Boolean).map((p) => p.replace(/\\/g,  
    "/"));  
}
```

สามบรรทัด แต่มี history ข้างหลังที่ยาวพอดู

`ghq list --full-path` บน Windows คีน backslash paths —

`C:\Users\nat\ghq\github.com\ ...` แต่ทุก call site ใน `maw-js` ใช้ forward-slash patterns ก่อนหน้านี้แก้ปัญหานี้ด้วยการเพิ่ม `| tr '\\' '/'` ที่ 13 call sites แยกกัน PR #379 ทำไปแล้ว แต่พอมีคนเพิ่ม call site ใหม่ก็ลืม fix เดิมอยู่เสมอ ทุกครั้งที่มี PR ใหม่ก็ต้องไป review ว่าลืมแปลง slash ใหม่

การรวม normalize ไว้ที่จุดเดียวคือ choke point ที่ทำให้ fix นั้นไม่มีวันถูกลืมอีก Fowler เรียก pattern แบบนี้ว่า “single point of truth” — ไม่ต้องไปเตือนทุกคนทุกครั้ง ไม่ต้อง code review checklist ว่า “อย่าลืม slash” แค่มั่นใจว่ามีจุดเดียวที่ทุกคนผ่าน แล้วจุดนั้นทำเองโดยอัตโนมัติ

selectRepoMatch — doubled-path story (#2577)

นี่คือ function ที่มี story ที่น่าสนใจที่สุดใน file — bug ที่เกิดจาก misconfiguration ของ user ไม่ใช่ bug ใน code ตัวเอง

```
export function selectRepoMatch(paths: string[], suffix: string):  
string | null {  
    const lower = literalize(suffix).toLowerCase();  
    const matches = paths.filter((p) => p.toLowerCase().endsWith(lower));
```

```

if (matches.length === 0) return null;

const canonical = matches.find((p) => !p.toLowerCase().includes("/github.com/github.com/"));

return canonical ?? matches[0]!;
}

```

มี case ที่แปลก — ถ้า ghq root ถูก configure ผิด โดยชี้ไปที่ path ที่ลงท้ายด้วย

`.../github.com/` แทนที่จะเป็น `.../ghq/` ธรรมดา ghq จะ clone repo ไปอยู่ที่

`.../github.com/github.com/<org>/<repo>` — path ที่มี `github.com` สองชั้น

ปัญหาคือ `ghq list` จะ surface ทั้งสองที่ — `.../github.com/<org>/<repo>` (canonical

ถ้ามี) และ `.../github.com/github.com/<org>/<repo>` (doubled) ทั้งคู่ end with suffix

เหมือนกัน แล้ว first-match เดิมจะเอาอันไหน? แล้วแต่ ghq จะ list ก่อน ซึ่งไม่

deterministic

พอเกิดกับ `maw wake neo` มันพัง ไม่มีสัญญาณว่าทำไม แคหา repo ไม่เจอ หรือเจอ path ที่ผิด แล้ว session ก็เปิดขึ้นมาที่ repo ผิด โดยไม่มี error message

fix คือ prefer canonical path — ถ้ามีคนที่ไม่มี `github.com/github.com/` ใน path เอาอันนั้นก่อน ถ้ามีแต่ doubled path ทั้งหมดก็ fall back ไป first match เพราะอย่างน้อยก็ resolve ได้ ดีกว่าคืน null แล้วบอกว่า “ไม่เจอ”

literalize ก็มี story ด้วย — ลบ trailing `$` ถ้ามี

```

function literalize(suffix: string): string {
    return suffix.endsWith("$") ? suffix.slice(0, -1) : suffix;
}

```

call site เก่าบางตัวยังใช้ pattern แบบ shell `grep '/foo$'` แล้วส่ง string นั้นมาตรงๆ

`$` นั้นไม่มีความหมายใน `endsWith` แต่ถ้าไม่ strip ออก `findBySuffix("/oracle$")` จะ

คืน null เสมอ เพราะไม่มี path ที่ลงท้ายด้วย literal `$` bug เล็กน้อย แต่ debug ยากมาก

เพราะ caller ก็คิดว่าส่งถูกแล้ว

มี comment ใน code บอกว่าถูก “caught by team-agents debate 2026-04-16” —
ช่วงที่ agent ถกกันว่า pattern matching ทำงานยังไง แล้วเจอ edge case นี้ระหว่าง
discussion

detectFromCwd — Pure Path Analysis (#2573)

```
export function detectFromCwdPath(cwd: string | undefined):  
RepoDetectResult | null {  
  const parts = cwd ? resolve(cwd).split(/[\\\/]+/).filter(Boolean) :  
  [];  
  if (parts.length === 0) return null;  
  
  const agentsIdx = parts.lastIndexOf("agents");  
  if (agentsIdx > 0 && parts[agentsIdx + 1]) {  
    const oracle = stripOracleRepoSuffix(parts[agentsIdx - 1] ?? "") ??  
    undefined;  
    return { oracle, worktree: parts[agentsIdx + 1] };  
  }  
  
  for (const part of parts.slice().reverse()) {  
    const worktreeMarker = part.indexOf(".wt-");  
    if (worktreeMarker > 0) {  
      const oracle = stripOracleRepoSuffix(part.slice(0,  
worktreeMarker)) ?? undefined;  
      const worktree = part.slice(worktreeMarker + ".wt-".length) ||  
      undefined;  
      return { oracle, worktree };  
    }  
    const oracle = stripOracleRepoSuffix(part);  
    if (oracle) return { oracle };  
  }  
}
```

```
    return null;
}
```

function นี้รู้จัก path 3 แบบที่ maw ใช้

nested agents layout — `.../mawjs-oracle/agents/1-fix-2573` เจอ segment

`agents` ก็รู้ว่า segment ก่อนหน้าคือ oracle repo และ segment ถัดไปคือ worktree slug ใช้ `lastIndexOf("agents")` ไม่ใช่ `indexOf` เพราะ path อาจมีคำว่า agents หลายชั้น เอาชั้นสุดท้ายที่ถูกต้องกว่า

legacy .wt- layout — `.../mawjs-oracle.wt-white` รูปแบบเก่าที่ใช้ก่อน nested layout จะมา หา `.wt-` ใน folder name แล้ว split ออก oracle อยู่ก่อน `.wt-` worktree อยู่หลัง

plain oracle dir — `.../mawjs-oracle` แคว้นว่าอยู่ใน oracle repo ไม่ใช่ worktree คีน

`{ oracle: "mawjs" }` โดยไม่มี `worktree`

แล้วก็รู้ว่า `.../maw-js` ไม่ใช่ oracle เลยคืน null เพราะไม่มี segment ที่ลงท้ายด้วย

`-oracle`

stripOracleRepoSuffix ก็ถูก export ออกมาด้วย เพราะ logic การ strip `-oracle`

suffix มีที่ใช้มากกว่าใน `detectFromCwd`

```
export function stripOracleRepoSuffix(name: string): string | null {
    return name.toLowerCase().endsWith("-oracle")
        ? name.slice(0, "-oracle".length)
        : null;
}
```

case-insensitive check แต่ preserve original case —

`stripOracleRepoSuffix("Discord-Oracle")` คีน `"Discord"` ไม่ใช่ `"discord"` เพราะ

oracle name บางตัวมีตัวใหญ่

สิ่งที่ทำให้ function เหล่านี้ดีคือ pure path analysis — ไม่มี `exec`, ไม่มี `fs.stat`, ไม่มี

network call แค่ string split และ loop สั้นๆ test ง่ายมาก ไม่ต้อง mock อะไรเลย

```
// จาก detect-from-cwd.test.ts - ทุก case ไม่ต้อง setup อะไร
expect(detectFromCwdPath("/o/mawjs-oracle"))
  .toEqual({ oracle: "mawjs" });
expect(detectFromCwdPath("/o/mawjs-oracle/agents/1-fix-2573"))
  .toEqual({ oracle: "mawjs", worktree: "1-fix-2573" });
expect(detectFromCwdPath("/o/mawjs-oracle.wt-white"))
  .toEqual({ oracle: "mawjs", worktree: "white" });
expect(detectFromCwdPath("/opt/Code/github.com/Soul-Brews-Studio/maw-
js"))
  .toBeNull();
expect(detectFromCwdPath(undefined)).toBeNull();
```

ก่อน PR #2573 logic แบบนี้จะกระจายอยู่ใน `bringCwdMetadata`, `deriveOracleFromCwd`, และ `oracle-workon` สามที่ สามรูปแบบที่ต่างกันเล็กน้อย พอ edge case ใหม่เจอก็แก้ทีละตัว อีกสองที่ยังมีปัญหาเดิม และ test ก็ต้องเขียนสามชุดสำหรับ logic เดียวกัน

6.4 Singleton และ Backward-Compat Shim

`src/core/repo-discovery/index.ts` ทำสองเรื่องพร้อมกัน และทั้งสองเรื่องสำคัญ

```
let _instance: RepoDiscovery | null = null;

export function getRepos(): RepoDiscovery {
  if (_instance) return _instance;
  _instance = GhqDiscovery;
  return _instance;
}

/** Inject a mock adapter - for tests only. */
export function setRepos(impl: RepoDiscovery): void {
```

```

    _instance = impl;
  }

  /** Clear the cached adapter – for tests only. */
  export function resetRepos(): void {
    _instance = null;
  }

  // — Backward-compat re-exports
  _____

  export const ghqList = (): Promise<string[]> => getRepos().list();
  export const ghqListSync = (): string[] => getRepos().listSync();
  export const ghqFind = (suffix: string): Promise<string | null> =>
    getRepos().findBySuffix(suffix);
  export const ghqFindSync = (suffix: string): string | null =>
    getRepos().findBySuffixSync(suffix);

```

Singleton pattern — `getRepos()` คืน instance เดียวกันทั้ง codebase ไม่ต้อง pass instance รอบ ไม่ต้องใช้ dependency injection framework new code ที่เพิ่มใหม่ใช้

`getRepos().findBySuffix( ... )` ตรงๆ ได้เลย

Test injection สะอาด — `setRepos(mock)` + `resetRepos()` ทำให้ test สามารถ swap implementation โดยไม่ต้องแตะ module system ไม่ต้อง `jest.mock()` ไม่ต้อง

`vi.mock()` ไม่ต้อง import hook

pattern ใน test จะเป็นแบบนี้

```

import { setRepos, resetRepos } from "../repo-discovery";

const mockRepos: RepoDiscovery = {
  name: "mock",
  list: async () => ["/home/nat/ghq/github.com/Soul-Brews-Studio/maw-
js"],

```

```

listSync: () => ["/home/nat/ghq/github.com/Soul-Brews-Studio/maw-
js"],
findBySuffix: async (s) => selectRepoMatch(
  ["/home/nat/ghq/github.com/Soul-Brews-Studio/maw-js"], s
),
findBySuffixSync: (s) => selectRepoMatch(
  ["/home/nat/ghq/github.com/Soul-Brews-Studio/maw-js"], s
),
detectFromCwd: (cwd) => detectFromCwdPath(cwd),
};

beforeEach(() => setRepos(mockRepos));
afterEach(() => resetRepos());

```

Backward-compatible shim — `ghqList`, `ghqFind`, `ghqListSync`, `ghqFindSync` ยังอยู่
call site เก่า 4 ไฟล์ยังใช้ได้เหมือนเดิมโดยไม่ต้อง refactor อะไร

```

soul-sync/resolve.ts
oracle/impl-helpers.ts
workon/impl.ts
fleet/fleet-init-scan.ts

```

และ `src/core/ghq.ts` ก็กลายเป็นแค่ re-export shim ทั้งไฟล์

```

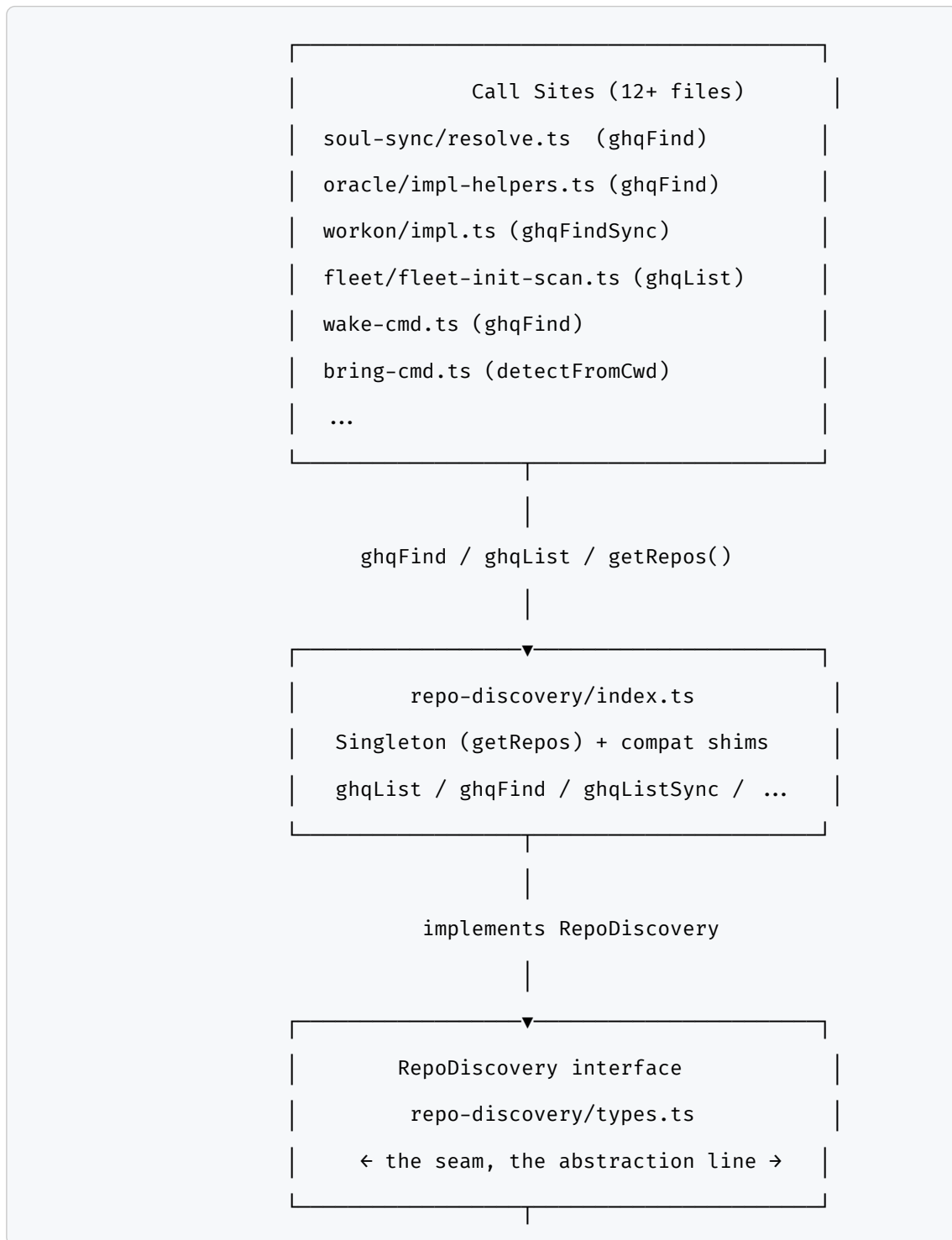
// src/core/ghq.ts - ทั้งไฟล์ 3 บรรทัด
export { ghqList, ghqListSync, ghqFind, ghqFindSync } from "./repo-
discovery";
export { _normalize } from "./repo-discovery/ghq-discovery";

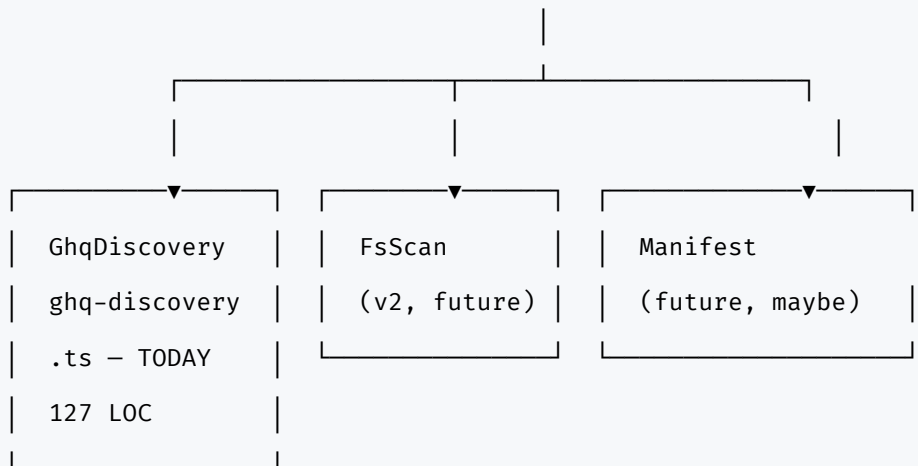
```

ไฟล์เดิมยังอยู่ import path เดิมยังใช้ได้ แต่ข้างในไม่มี logic แล้ว มีแค่ re-export ไปยัง
implementation จริง

นั่นคือ Fowler's refactoring ที่ไม่ทำลาย caller — เปลี่ยนข้างใน ไม่เปลี่ยน interface ที่ expose ออกไป

Diagram: RepoDiscovery Architecture





วันนี้มีแค่ GhqDiscovery แต่ seam อยู่แล้ว พร้อมรับ backend ที่สองโดยไม่ต้องแตะ call site ใดเลย

6.5 YAGNI Until the Second Backend

ใน `index.ts` มี comment ที่น่าอ่านมาก — อธิบาย decision ที่ดูเหมือนขาดอะไรไป แต่ขาดโดย intentional

```
// env-var dispatch is intentionally NOT wired until the second backend  
lands.  
  
// Reason: a tautological `kind == "ghq" ? Ghq : Ghq` branch tests  
only  
  
// that the hook exists, not that it dispatches – see the critic-  
agent's  
  
// alpha.55-58 retro that flagged this as the smoking gun for premature  
abstraction.  
  
// We kept the seam, removed the lie.  
  
//  
  
// When the second backend ships (same PR), wire
```

```
process.env.MAW_REPO_DISCOVERY
// here with a real branch.
```

ก่อน PR #2573 มี branch logic อยู่ตรงนี้ — ประมาณว่า

```
if (process.env.MAW_REPO_DISCOVERY === "ghq") return GhqDiscovery
```

 แต่เนื่อง

จากยังไม่มี backend ที่สอง code นั้นก็คือ `if (true) return GhqDiscovery` — branch ที่ไม่มีวันเป็น false

critic-agent ใน session ก่อนหน้า (retro alpha.55-58) จับได้ว่านี่คือ “smoking gun for premature abstraction” — test ที่ test ว่า dispatch hook มีอยู่ ไม่ใช่ test ว่า dispatch ทำงาน code ที่ดูเหมือน flexible แต่จริงๆ ไม่ได้ flexible กว่าเดิมเลย เพียงแต่ซับซ้อนขึ้น fix คือ เก็บ seam ไว้ (interface ยังอยู่ singleton ยังอยู่ inject-for-test ยังอยู่) แต่เอา fake branch ออก พอ backend ที่สองมา ก็ wire `MAW_REPO_DISCOVERY` env var ที่นั่นในขณะเดียวกัน ไม่ใช่ก่อน

Fowler เรียก principle นี้ในหลายที่ว่า refactoring ที่ถูกเวลา — extract ก็ต่อเมื่อมี duplication จริง ไม่ใช่เพราะคาดว่าจะมีในอนาคต GhqDiscovery มาถึง PR #2573 พร้อมกับการ extract ที่มี justification จริง — สามที่ที่ logic ซ้ำกัน ไม่ใช่ zero ที่ถ้ามีแค่ GhqDiscovery ที่เดียวตั้งแต่ต้น แล้วจะสร้าง interface ทำไม? เพราะ code สำหรับ extract interface นั้นต้นทุนถูก — แค่ตั้ง type แค่ตั้ง contract แต่ถ้าไม่สร้างตอนนี้ พอ backend ที่สองมา ก็ต้องไป refactor ทุก call site อยู่ดี ดีกว่าทำเดี๋ยวนี้ตอนที่ยังรู้ว่าจะทำอะไร

6.6 ทำไม Fowler ถึงเหมาะกับบทนี้

DNA ของบทนี้คือ Fowler — “abstraction seam ที่ถูกจังหวะ” ไม่ใช่ abstract pattern เพราะอยากทำ แต่เพราะมีเหตุผลจริงๆ

GhqDiscovery ไม่ได้เป็น engineering ที่โอ้อวด ไม่มี design pattern 5 ชั้น ไม่มี factory factory abstract builder ที่ไม่จำเป็น แค่ interface หนึ่งอัน implementation หนึ่งอัน

singleton เรียบๆ และ shim สำหรับ backward compat ทั้งหมดอยู่ใน 4 files รวมกัน ไม่ถึง 300 บรรทัด

แต่ seam นั้นขีดไว้ถูกจุด — ตอนที่มีการ extract `detectFromCwdPath` เป็น pure function แทนที่จะ inline ไว้ใน method นั่นคือ refactoring ที่ทำเพราะมีเหตุผลจริงๆ สามที่ที่ duplicate ตัดเป็นหนึ่งที่ test ง่ายขึ้น maintain ง่ายขึ้น

ตอนที่เก็บ `literalize` ไว้แม้ว่าดูเล็กน้อย — นั่นคือ removing a trap สำหรับ future caller ที่ยังส่ง `$` suffix มา ต้นทุนศูนย์ ป้องกัน bug ที่ debug ยาก ตอนที่ normalize ไว้ที่จุดเดียวแทน 13 call sites — นั่นคือ Fowler's choke point ที่แท้จริง ทำครั้งเดียว แก่ตลอด

ตอนที่ไม่วิธี env-var dispatch ก่อนมี backend ที่สอง — นั่นคือ YAGNI ที่ honest ไม่แกล้งทำเป็นว่า extensible ทั้งที่จริงๆ ยังไม่ extensible

Fowler บอกว่า refactoring ที่ดีไม่ใช่การทำให้ code สวยงาม แต่คือการทำให้ code พร้อมสำหรับ change ที่จะมา โดยไม่สร้าง cost ที่ไม่จำเป็นวันนี้ GhqDiscovery ทำได้แบบนั้น

6.7 Full Picture — ความสัมพันธ์กับ wake-cmd.ts

RepoDiscovery ไม่ได้อยู่โดดๆ ใน maw ทุกครั้งที่ `maw wake` resolve ว่าจะ open session ไหน ก็ผ่าน GhqDiscovery

resolution order ใน wake-cmd.ts มีหลายชั้น แต่ชั้นที่สำคัญที่สุดคือ `ghqFind` call — ที่เดิมเรียก `ghq` โดยตรง ตอนนี้เรียกผ่าน `getRepos().findBySuffix()` แทน ชื่อ `ghqFind` ยังเหมือนเดิมเพราะ backward compat แต่ข้างในเรียบร้อยแล้ว

```
// wake-cmd.ts resolution order (simplified)
const repoPath =
  opts.repoPath ?? // 1. explicit --repo flag
  (await ghqFind(slug)) ?? // 2. GhqDiscovery.findBySuffix
  (await fleetFind(slug)) ?? // 3. fleet registry
```

```
(await incubateFind(slug)) ?? // 4. incubate (new session)
(await resolveOracle(slug)); // 5. oracle fallback
```

ทุก `maw wake neo` ที่เคย type ผ่าน flow นี้ทั้งหมด และตอนนี้ทุกชั้นมีจุดเดียวที่ responsible — ไม่มีการ exec ghq กระจายอยู่ใน file ต่างๆ อีกแล้ว ถ้าวันหนึ่งอยากเปลี่ยน backend จาก ghq เป็น fs-scan ก็แค่ swap instance ใน `getRepos()` ทุก path ข้างบนได้รับ backend ใหม่โดยอัตโนมัติ

นอกจาก `wake-cmd.ts` แล้ว `detectFromCwd` ยังถูกใช้ใน `bring` ด้วย — ตอนที่ agent ทำงานอยู่ใน `worktree` แล้ว `maw bring` ต้องรู้ว่า `worktree` นี้ belong to oracle ไหน เพื่อจะ `bring` ไปอยู่ใน session ที่ถูก

```
// bring-cmd.ts (simplified)
const { oracle, worktree } = getRepos().detectFromCwd() ?? {};
if (oracle) {
  // know which oracle session to bring into
}
```

ปิดบท — เส้นที่ถูก แต่ยังไม่ครบ

GhqDiscovery วันนี้ครอบคลุม path ส่วนใหญ่ได้ดี ทั้ง normalize, doubled-path fix, `detectFromCwd` consolidation, backward-compat shim ทำได้โดยไม่ break อะไรใน 12+ call sites เลย

แต่ยังมีช่องว่างที่ intentional — `MAW_REPO_DISCOVERY` env var ยังไม่ถูก wire เพราะยังไม่มี backend ที่สอง นั่นคือ gap ที่รอ justification ไม่ใช่ gap ที่ลืมนำ backend ที่สองมา (fs-scan สำหรับ machine ที่ไม่ได้ install ghq, หรือ jj integration, หรือ custom manifest) seam นั้นรออยู่แล้ว แค่ต่อเข้าไปในขณะเดียวกันกับที่ ship backend

ถ้า backend ที่สองไม่มาตลอดกาล seam นั้นก็ไม่ต้องทำอะไร — มีต้นทุน zero ถ้าไม่ใช่
และ interface ก็ serve as documentation ให้คนรู้ว่า contract คืออะไรโดยไม่ต้องอ่าน
implementation
แต่ถ้า backend ที่สองมา — ก็พร้อม

6.8 เพิ่ม Backend ที่สอง — ทำได้อย่างไร

พูดถึงแล้วว่าจะต่อ backend ที่สองเข้าไปได้ ลองดูว่าจริงๆ แล้วต้องทำอะไรบ้าง เพราะนั่น
คือ proof ที่แท้จริงว่า abstraction seam นี้ทำงานหรือเปล่า
สมมติต้องการ `FsDiscovery` — backend ที่ scan filesystem โดยตรง ไม่ผ่าน ghq เหมาะ
สำหรับ machine ที่ไม่ได้ install ghq หรือ CI environment ที่ต้องการ find repo จาก
workspace layout โดยตรง

```
// src/core/repo-discovery/fs-discovery.ts
import { readdirSync, statSync } from "fs";
import { join } from "path";
import { GHQ_ROOT } from "../config/ghq-root";
import type { RepoDiscovery, RepoDetectResult } from "../types";
import { detectFromCwdPath } from "../ghq-discovery";

function scanForRepos(root: string, depth: number = 3): string[] {
  if (depth === 0) return [];
  try {
    return readdirSync(root).flatMap((name) => {
      const full = join(root, name);
      if (!statSync(full).isDirectory()) return [];
      // ถ้าเจอ .git ก็เป็น repo
      try {
        statSync(join(full, ".git"));
        return [full.replace(/\\/g, "/")];
      }
    });
  }
}
```

```

    } catch {
        // ไม่เจอ .git ก็ scan ลึกลงไป
        return scanForRepos(full, depth - 1);
    }
});
} catch {
    return [];
}
}

export const FsDiscovery: RepoDiscovery = {
    name: "fs",

    async list(): Promise<string[]> {
        return this.listSync();
    },

    listSync(): string[] {
        return scanForRepos(GHQ_ROOT);
    },

    async findBySuffix(suffix: string): Promise<string | null> {
        return this.findBySuffixSync(suffix);
    },

    findBySuffixSync(suffix: string): string | null {
        const lower = suffix.toLowerCase();
        return this.listSync().find((p) =>
p.toLowerCase().endsWith(lower)) ?? null;
    },

    detectFromCwd(cwd?: string): RepoDetectResult | null {
        return detectFromCwdPath(cwd ?? process.cwd());
    }
};

```

```
},  
};
```

แล้วใน `index.ts` ก็ wire

```
// index.ts - เพิ่ม 5 บรรทัด เปลี่ยนแค่นี้  
import { GhqDiscovery } from "../ghq-discovery";  
import { FsDiscovery } from "../fs-discovery";    // ← เพิ่ม  
import type { RepoDiscovery } from "../types";  
  
export function getRepos(): RepoDiscovery {  
  if (_instance) return _instance;  
  // wire env-var dispatch พร้อมกับ backend ที่สอง  
  const backend = process.env.MAW_REPO_DISCOVERY;  
  _instance = backend === "fs" ? FsDiscovery : GhqDiscovery; // ← เพิ่ม  
  return _instance;  
}
```

call site ทั้ง 12 ไฟล์ ไม่ต้องแตะเลยสักตัว เพราะทุกคนคุยผ่าน `getRepos()` อยู่แล้ว
นั่นคือ proof ที่แท้จริงของ adapter pattern — ไม่ใช่แค่ “เปลี่ยนได้” บน paper แต่
ทดสอบได้ว่าเปลี่ยนแล้ว call site ไม่แตก

6.9 Case Study: maw wake neo — ก่อนและหลัง

เพื่อให้เห็นภาพ ลองดูว่า `maw wake neo` ผ่าน `RepoDiscovery` ยังไง ทั้งก่อนและหลัง
abstraction

ก่อน **PR #2573** — code ใน `wake-cmd.ts` เรียก `execSync("ghq list --full-path")`

โดยตรง แล้วทำ `tr '\\\\' '/'` ในที่นั้น เพิ่ม `filter` ด้วย `endsWith` เอง บางที่มี `$` ลบ
ออกบางที่ไม่มี บางที่มี `normalize` บางที่ไม่มี

ถ้าเกิด doubled path เจอ first match ซึ่งอาจเป็นอันผิด ถ้า ghq ไม่ได้ install บางที่ throw บางที่คืน empty string ถ้าอยู่ใน worktree แต่ละ function อ่านว่า oracle คืออะไร ไม่ตรงกัน

หลัง PR #2573 — wake-cmd.ts เรียก `ghqFind("neo")` ซึ่ง delegate ไป

```
getRepos().findBySuffix("neo")
```

```
maw wake neo
→ ghqFind("neo")
→ getRepos().findBySuffix("neo")
→ GhqDiscovery.findBySuffix("neo")
→ GhqDiscovery.list()
→ hostExec("ghq list --full-path")
→ normalize(output) ← slash fix อยู่ที่นี่
→ selectRepoMatch(paths, "neo") ← doubled-path fix อยู่ที่นี่
→ "/home/nat/ghq/github.com/Soul-Brews-Studio/neo-oracle"
```

ทุก step ชัดเจน ทุก fix อยู่ทีเดียว ทุก edge case ถูกจัดการก่อนที่จะถึง wake-cmd.ts และถ้าวันหนึ่ง set `MAW_REPO_DISCOVERY=fs` ก็แค่ step ที่สองเปลี่ยนเป็น

```
FsDiscovery.findBySuffix("neo")
```

 ทุกอย่างเหมือนเดิม

ปิดบท — เส้นที่ถูก แต่ยังไม่ครบ

GhqDiscovery วันนี้ครอบคลุม path ส่วนใหญ่ได้ดี ทั้ง normalize, doubled-path fix, detectFromCwd consolidation, backward-compat shim ทำได้โดยไม่ break อะไรใน 12+ call sites เลย แต่ยังมีช่องว่างที่ intentional — `MAW_REPO_DISCOVERY` env var ยังไม่ถูก wire เพราะยังไม่มี backend ที่สอง นั่นคือ gap ที่รอ justification ไม่ใช่ gap ที่ลืมนะ พอ backend ที่สองมา (fs-scan สำหรับ machine ที่ไม่ได้ install ghq, หรือ jj integration, หรือ custom manifest) seam นั้นรออยู่แล้ว แค่ต่อเข้าไปในขณะเดียวกันกับที่ ship backend

ถ้า backend ที่สองไม่มาตลอดกาล seam นั้นก็ไม่ต้องทำอะไร — มีต้นทุน zero ถ้าไม่ใช้ และ interface ก็ serve as documentation ให้คนรู้ว่า contract คืออะไรโดยไม่ต้องอ่าน implementation

Fowler พูดไว้ใน Refactoring ว่า “the best time to refactor is when you first encounter the smell, not when you planned to” — และนั่นคือสิ่งที่เกิดขึ้นใน PR #2573 พอ edge case ที่สามเจอ ก็รู้แล้วว่าถึงเวลา seam นั้นไม่ได้วางแผนล่วงหน้า แต่เกิดขึ้นเพราะ duplication เจ็บปวดพอแล้ว

แต่ถ้า backend ที่สองมา — ก็พร้อม

บทยัดไปออกจาก repo layer ขึ้นไปอีกหนึ่งระดับ — ถ้า RepoDiscovery รู้จัก repo แล้ว

Team Charter รู้จัก คน charter คือ YAML ที่บอกว่าทีมมีใครบ้าง แต่ละคนทำงานที่ repo ไหน engine อะไร และตอนนี้มีชีวิตอยู่ไหม — pipeline 5 ขั้นที่เปลี่ยน desired state บน disk ให้กลายเป็น tmux sessions จริงๆ เหมือนที่ Hightower พูดถึงใน Kubernetes: declare what you want แล้วให้ system reconcile ไปเอง และ RepoDiscovery ก็คือ component ที่ charter ใช้เพื่อหา repo ของแต่ละ member — เส้นที่ขีดไว้ใน GhqDiscovery ส่งผลทอดไปถึง team layer ด้วยเลย

บทที่ 7: Team Charter — จาก YAML สู่ทีมจริง

Nat: “ทีมเราจะอยู่ยาวๆ นะ — ไม่ต้อง team down หลังเสร็จงาน”

ประโยชน์นั้นเปลี่ยนทุกอย่าง

ก่อนหน้านี้ทีม AI มักถูกมองว่าเป็นของชั่วคราว — spawn ขึ้นมาเพื่องาน done แล้วก็ kill

ทิ้ง เหมือน container ที่เปิดมาเพื่อ job แล้วก็ปิด พอเสร็จก็เริ่มใหม่อีกรอบ แต่ Nat เห็น

อะไรบางอย่างที่ต่าง — agent ที่ warm อยู่แล้วในงานชุดเดิมมันเร็วกว่า agent ใหม่ 5 เท่า

context ที่สะสมมาคือทรัพย์สิน ไม่ใช่ภาระ

คำถามคือ: จะ encode “ทีมถาวร” ไว้ที่ไหน?

คำตอบคือ YAML ใน repo

7.1 Charter คือ Desired State — ไม่ใช่ Script

Hightower’s law of infrastructure: อธิบาย ผลลัพธ์ที่ต้องการ ไม่ใช่ ขั้นตอนที่ต้องทำ

Kubernetes ไม่บอกว่า “สร้าง pod ตอนนี้” — บอกว่า “replica: 3 ต้องมีอยู่เสมอ” แล้ว

reconciler จะหาทาง

`maw team up` ทำงานแบบเดียวกัน

ไฟล์ `.maw/teams/mawjs-m5.yaml` คือ charter ของทีม maw-js บน m5 อ่านแล้วเข้าใจได้

ทันที:

```
# mawjs-m5 — permanent maw-js dev team  
  
# Usage: maw team up mawjs-m5
```

```

name: mawjs-m5
project: Soul-Brews-Studio/maw-js

defaults:
  worktree: true
  branch: alpha

members:
  - role: lead
    name: mawjs-oracle
    engine: claude
    worktree: false
    prompt: |
      Lead orchestrator. Dispatch issues, review PRs, merge when CI
green,
      nudge idle codex, give guidance if stuck.
      Do NOT code – dispatch + verify only.

  - role: codex-1
    name: codex-1
    engine: omx
    prompt: |
      maw-js Codex builder. Wait for task from lead via maw hey.
      PRs target alpha, never main. PR body MUST include "Closes
#NNNN".

  - role: codex-2
    name: codex-2
    engine: omx
    ...

```

charter นี้บอกแล้วว่า “ทีมควรมีสมาชิกเหล่านี้ ด้วย engine เหล่านี้ ใน session นี้” ไม่ได้บอกกว่า “ถ้า codex-1 dead ให้ kill แล้ว spawn ใหม่” — logic นั้นอยู่ใน code ที่ reconcile

พอรัน `maw team up mawjs-m5` ก็เกิด pipeline 5 stage ขึ้นมา

7.2 Pipeline 5 Stage: Parse → Plan → Preflight → Load → Spawn

Charter YAML

|

▼

[1] PARSE

readTeamCharter() ← 817 LOC parser

↓ TeamCharter object

|

▼

[2] PLAN

classifyMember() สำหรับทุก member

↓ roster: ClassifiedTeamMember[]

|

▼

[3] PREFLIGHT

ตรวจ collision, node guard, engine compat

↓ warnings[]

|

▼

[4] LOAD

resolve repoSlug, session, paths

↓ targetRepoSlug, session name

|

▼

[5] SPAWN

wakeMember() สำหรับ missing/dead

↓ tmux windows ขึ้นมา

stage ที่น่าสนใจที่สุดคือ stage 2 — `classifyMember()`

พอ `maw team up` ถูกเรียก มันไม่ได้แค่ spawn ทุกคนทันที — ก่อนอื่นมันดูก่อนว่าใครมีอยู่ แล้ว ใครตาย ใครหายไป นี่คือ reconciliation loop ขนาดย่อม

7.3 `classifyMember()` — 4 สถานะของสมาชิก

`classifyMember()` อยู่ใน `team-liveness.ts` รับ member spec หนึ่งตัวกับ snapshot ของ pane ทั้งหมดที่มีอยู่ในขณะนั้น แล้ว return สถานะหนึ่งใน 4:

```
export type TeamMemberState = "live" | "dead" | "missing" | "skipped";

export function classifyMember(
  member: TeamCharterMember,
  panes: TeamPaneSnapshot[],
  session: string,
  opts: ClassifyMemberOptions = {},
): ClassifiedTeamMember {
  // guard: other-node member → skipped
  if (isOtherNodeMember(member, opts.currentNode)) {
    return { ...base, state: "skipped", skipReason: `other node: ${member.node}` };
  }

  // guard: --only filter
  if (opts.only && !matchesOnly(member, opts.only)) {
    return { ...base, state: "skipped", skipReason: "outside --only" };
  }

  // find pane
  const pane = panes.find((p) =>
    p.sessionName === session && candidates.includes(p.windowName)
```

```

);

if (!pane) return { ... base, state: "missing" };
if (SHELL_RE.test(pane.command)) return { ... base, state: "dead",
pane };
if (isAgentCommand(pane.command)) return { ... base, state: "live",
pane };
return { ... base, state: "dead", pane };
}

```

logic ตรงไปตรงมา — ถ้าไม่เจอ pane เลยก็ `missing` ถ้าเจอแต่ command เป็น shell (zsh/bash) ก็ `dead` — agent เคยอยู่แต่ exit ไปแล้ว ถ้า command เป็น agent process ก็ `live`

แต่ละสถานะ map ไปยัง action ที่ต่างกัน:

State	Action
<code>live</code>	skip — ไม่แตะ
<code>dead</code>	resume in place — ส่ง command ให้ pane เดิม
<code>missing</code>	fresh wake — สร้าง window ใหม่
<code>skipped</code>	skip — node guard หรือ -only filter

ความสำคัญของ `dead` vs `missing` คือ: dead agent ยังมี window อยู่ ยังมี cwd อยู่ worktree ยังถูก checkout อยู่ — ดังนั้น resume ต่อได้เลยโดยไม่ต้อง re-clone ใดๆ ทั้งสิ้น

7.4 Cross-Engine: Claude + Codex (omx) ในทีมเดียว

charter ของ mawjs-m5 ใช้ engine สองชนิดพร้อมกัน:

- `claude` — Claude Code (Anthropic) สำหรับ lead oracle
- `omx` — OpenAI Codex สำหรับ builder codex-1 ถึง codex-4

ทำไมต้องสองชนิด? เพราะคุณสมบัติต่างกันชัดเจน

Claude มี context window ที่ต้องจัดการ — oracle ต้อง nudge เมื่อ context เริ่มเต็ม ส่ง summary ให้เป็นระยะ ทำ compaction เมื่อจำเป็น แต่ Codex (omx) มี auto-compact ในตัว — มันจัดการ context window ตัวเองได้ ไม่ต้องดูแล comment ใน charter บอกชัด:

```
# Engine notes:
#   claude — Claude Code (Anthropic). Context managed by user/oracle.
#   omx    — OpenAI Codex. Has built-in auto-compact; manages its own
#           context window automatically. Do NOT worry about codex
#           context running low — it self-manages better than claude.
```

`engineCommand()` ใน `team-liveness.ts` จัดการ resolution:

```
export function engineCommand(
  engine: string,
  opts: EngineCommandOptions = {},
  config: MawConfig = loadConfig(),
): string {
  // charter-local engines override
  if (opts.engines) {
    const override = flattenEngineValue(opts.engines[engine]);
    if (override.length) return [opts.resume ? "--continue" :
null, ... override]
      .filter(Boolean).join(" ");
  }
  // fall back to config
  return resolveEngine(engine, config, { resume: opts.resume });
}
```

charter สามารถ define engine commands แบบ custom ได้ด้วย `engines:` block —
เช่นถ้าต้องการส่ง flag พิเศษให้ omx ใน environment นี้ ก็ทำได้โดยไม่ต้องแก้ global
config

cross-engine design นี้สำคัญมากในทางปฏิบัติ: marathon session 2026-06-06 ที่ ship
28 PRs ใน 3.5 ชั่วโมงทำได้เพราะ codex workers 4 ตัว run parallel โดยไม่ต้องดูแล
context ทุกตัวแยกกัน

7.5 Standing Team — ทีมถาวร ไม่ใช่ Ad Hoc

pattern ที่เปลี่ยนทุกอย่างคือ: ทีมไม่ down หลังงานเสร็จ

ก่อนหน้า workflow มักจะเป็น: 1. spawn team 2. งานเสร็จ 3. `maw team down` 4. ครั้ง
หน้า spawn ใหม่ทั้งหมด

แต่ cold start มีต้นทุนสูง — codex ใหม่ที่ยังไม่รู้ codebase ต้องเรียนใหม่ 5-10 นาทีก่อน
จะ productive claude ใหม่ต้อง orient ใหม่ ทั้งหมดนี้เสีย token และเวลา
standing team pattern แก่ด้วย 2 หลักการ:

หลักที่ 1: idle codex = ready, ไม่ใช่ waste

agent ที่ idle อยู่มี context อุ่นๆ อยู่แล้ว พอส่งงานมาก็รับได้เลย ต่างจาก cold start ที่
ต้องอ่าน issue ทำความเข้าใจ codebase ก่อน

หลักที่ 2: `maw team up` เป็น idempotent

รันกี่ครั้งก็ได้ — สมาชิกที่ `live` จะถูก skip ไม่มีการ kill แล้ว respawn ที่ไม่จำเป็น
reconciler แค่ fix ส่วนที่ขาด:

```
$ maw team up mawjs-m5
team up: mawjs-m5 (mawjs)

role      identity      engine  state  action
lead      mawjs-oracle    claude  live   skip live
claude-1  claude-1        claude  live   skip live
codex-1   codex-1         omx     live   skip live
```

codex-2	codex-2	omx	dead	resume in place
codex-3	codex-3	omx	missing	fresh wake
codex-4	codex-4	omx	live	skip live

รันครั้งนี้ — codex-2 dead ก็ resume codex-3 missing ก็ wake ใหม่ ที่เหลือ skip หหมด ไม่
มี disruption ใดๆ กับ agent ที่กำลังทำงานอยู่

7.6 Charter ใน Repo = Soul ของทีม

ทำไมถึงเก็บ charter ไว้ใน `.maw/teams/` ไม่ใช่ config ภายนอก?

เพราะ charter คือ soul ของทีม — และ soul ต้องอยู่กับ repo

พอ `mawjs-m5.yaml` อยู่ใน mawjs-oracle repo และ commit ไว้ใน git: - ทุก session ได้

definition เดิม - เปลี่ยน charter ก็ commit — มี history - fork repo ไปก็เอา team

definition ไปด้วย - cross-machine setup ก็ clone แล้ว `maw team up` เลย

`resolveCharterPath()` ใน team-liveness.ts หาไฟล์ตาม priority:

```
export function resolveCharterPath(
  team: string,
  cwd = process.cwd(),
  configNode?: string
): string | null {
  const root = findRepoRoot(cwd);

  // 1. .maw/teams/<team>.yaml
  const local = join(root, ".maw", "teams", `${team}.yaml`);
  if (existsSync(local)) return local;

  // 2. ψ/teams/<team>.yaml (soul vault)
  const psi = join(root, "ψ", "teams", `${team}.yaml`);
  if (existsSync(psi)) return psi;
```

```
// 3. .maw/<node>.yaml (node-specific charter)
const nodeCandidate = configNode
  ? join(root, ".maw", `${configNode}.yaml`)
  : "";
if (configNode && existsSync(nodeCandidate)) return nodeCandidate;

return null;
}
```

ลำดับนี้สำคัญ — `.maw/teams/` มาก่อน `ψ/teams/` มาก่อน node-specific ทีมที่เป็น “ทางการ” อยู่ใน `.maw/` ทีมที่ทดลองอยู่ใน `ψ/` ทีมที่ node-specific อยู่ในรูปแบบอื่น

7.7 `team-charter.ts` 817 LOC — Parser ที่เขียนเอง

ทำไมต้อง parse YAML เอง ไม่ใช่ library?

คำตอบอยู่ในบริบท: maw-js เป็น CLI tool ที่ install ได้ทุกที่ การ depend on YAML

library เพิ่ม bundle size เพิ่ม version conflict risk เพิ่มความซับซ้อนของ dependency tree

subset ของ YAML ที่ team charter ใช้ก็ไม่ได้ซับซ้อน — scalar, nested object, list,

multiline string (`|`), YAML anchors (`&anchor` , `*anchor`) แค่นั้น ดังนั้น parser 817

LOC ที่รองรับ subset นี้ก็เพียงพอ

ส่วนที่น่าสนใจที่สุดใน parser คือ anchor support — `mawjs-m5.yaml` สามารถใช้ pattern แบบนี้ได้:

```
flags:
  builder-prompt: &builder-prompt
    - "Wait for task from lead via maw hey."
    - "PRs target alpha, never main."
```

```
members:
- role: codex-1
  engine: omx
  queue: *builder-prompt # ← alias ดึง anchor มาใช้

- role: codex-2
  engine: omx
  queue: *builder-prompt # ← ใช้อีกครั้ง
```

anchor ช่วยให้ member ที่มี prompt เหมือนกัน (codex workers ทั้ง 4 ตัว) ไม่ต้อง repeat prompt ทุกคน ลด duplication ทำให้ charter อ่านง่ายขึ้น

7.8 Node Guard — ทีม Multi-Machine ใน Charter เดียว

charter รองรับ `node:` field ในแต่ละ member:

```
members:
- role: lead
  name: mawjs-oracle
  engine: claude
  worktree: false
  # ไม่มี node: → run ทุกที่

- role: sila-tester
  name: sila
  engine: claude
  node: oracle-world # ← run เฉพาะบน oracle-world เท่านั้น
  worktree: false
```

พอรัน `maw team up` บน `m5` — `silatester` จะถูก classify เป็น `skipped` เพราะ

`node: oracle-world` $\neq$ `current node m5`

```
export function isOtherNodeMember(  
  member: TeamCharterMember,  
  currentNode?: string  
): boolean {  
  return Boolean(member.node && member.node !== currentNode);  
}
```

design นี้ทำให้ charter หนึ่งไฟล์ describe ทีมข้ามหลาย machine ได้ — ไม่ต้องมีไฟล์แยกต่างหากต่อ node ทุก node รัน `maw team up` เดียวกัน แต่แต่ละ node ก็ spawn เฉพาะ members ของตัวเอง

7.9 Preflight — ตรวจสอบก่อน Spawn

ก่อนที่จะ spawn จริง pipeline มี preflight stage ที่ตรวจสอบ:

- worktree path collision — agent สองตัวใช้ path เดียวกัน
- engine compat — engine ที่ระบุอยู่ใน config จริงไหม
- session mismatch — `charter.session` $\neq$ `current tmux session` → warn ไม่ error

```
function warnOnPathCollisions(  
  roster: ClassifiedTeamMember[],  
  warnings: string[]  
): void {  
  const seen = new Map<string, string>();  
  for (const item of roster) {  
    if (!item.member.worktree || !item.pane?.path) continue;  
    const prior = seen.get(item.pane.path);
```

```

    if (prior && prior !== item.role) {
      warnings.push(
        `worktree path collision: ${prior} and ${item.role} both at
        ${item.pane.path}`
      );
    } else {
      seen.set(item.pane.path, item.role);
    }
  }
}

```

warnings ไม่ block การ spawn — แค่ print ออกมาให้ผู้ใช้เห็น errors เท่านั้นที่ block ความแตกต่างนี้สำคัญ: tool ที่ block เพราะ warning จะทำให้คนรำคาญแล้วหา workaround ที่แย่กว่า tool ที่ warn แล้วเดินหน้าต่อทำให้ productive มากกว่า

7.10 Wake Member — Stage สุดท้ายที่ทำงานจริง

พอ plan ได้แล้ว `cmdTeamUp()` วน loop `launchTasks` และ call `wakeMember()` สำหรับทุก member ที่ต้อง spawn:

```

for (const [index, item] of roster.entries()) {
  if (item.state === "skipped") {
    actions.push({ ...skip });
  } else if (item.state === "live") {
    actions.push({ ...skipLive });
  } else if (item.state === "dead") {
    // resume: ส่ง command ให้ pane เดิม
    launchTasks.push({
      item,
      run: async () => {

```

```

        await tmux.run("send-keys", "-t", item.pane!.paneId, "C-u");
        await tmux.run("send-keys", "-t", item.pane!.paneId, command,
"Enter");
    },
});
} else {
    // missing: fresh wake สร้าง window ใหม่
    launchTasks.push({
        item,
        run: async () => {
            await wakeMember(
                targetRepoSlug,
                item.member,
                { engine: item.engine, session, repoPath: repoRoot },
                { cmdWakeFn: deps.cmdWakeFn },
            );
        },
    });
}
}
}

```

`dead` → `resume`: ส่ง command เข้า pane เดิม ไม่ต้องสร้าง window ใหม่ worktree ยังอยู่ครบ

`missing` → `fresh wake`: ไปเรียก `cmdWake()` ซึ่งเป็น 1519 LOC heart of maw ที่บทที่ 3 พุดถึง — resolution pipeline เต็ม ตั้งแต่ `ghqFind` จนถึง `tmux split`

Charter คือสัญญา ไม่ใช่ Script

ความต่างที่สำคัญที่สุดระหว่าง “สร้างทีม ad hoc” กับ “ใช้ charter”:

Ad hoc: “รัน A แล้วรัน B แล้วรัน C” **Charter:** “ทีมควรมี A B C อยู่เสมอ”

Hightower เรียก pattern นี้ว่า desired state reconciliation — ระบบไม่สนใจว่าตอนนี้มีอะไรอยู่ สนใจแค่ว่า ควรจะมีอะไรอยู่ แล้วค่อย reconcile ให้ตรง charter ของ mawjs-m5 จึงไม่ใช่ shell script ที่รันครั้งเดียว — เป็นนิยามของทีมที่ valid ตลอดเวลา รันวันนี้ก็ได้ผลลัพธ์เดิม รันเดือนหน้าก็ได้ผลลัพธ์เดิม เพราะ logic คือ “make it so” ไม่ใช่ “do these steps” charter อยู่ใน repo — เข้า git commit history มาด้วย ทีมเปลี่ยน charter เปลี่ยน เป็นไปพร้อมกัน soul ของทีมกับ soul ของ code อยู่ทีเดียวกัน

บทถัดไป → charter บอกว่า members ควรทำงานใน

project: Soul-Brews-Studio/maw-js แต่ oracle repo กับ maw-js repo เป็นคนละ repo กัน workers จะไปอยู่ที่ไหน — worktree ของ repo ไหน? บทที่ 8 จะขุด call chain ทั้งหมดตั้งแต่ charter ไปจนถึง git remote ที่ verified แล้ว

บทที่ 8: Cross-repo Worktrees — Workers ข้าม เส้น

DNA: Buterin — hub ไม่ต้องย้าย

oracle อยู่ repo หนึ่ง workers อยู่อีก repo
แต่ทั้งคู่ก็ทำงานอยู่ใน tmux session เดียวกัน ในหน้าจอเดียวกัน รวากับว่าไม่มีเส้นแบ่ง
ระหว่าง repo เลย
นี่ไม่ใช่ magic ไม่ใช่ hack และไม่ใช่ workaround นี่คือ design ที่ตั้งใจ — หนึ่งบรรทัดใน
charter YAML ที่เปลี่ยนทิศทางการทำงานของทีมทั้งหมด

```
project: Soul-Brews-Studio/maw-js
```

พอบรรทัดนี้มีอยู่ `maw team up` ก็รู้ว่า workers ทุกคนต้องไป checkout จาก `maw-js` ไม่
ใช่จาก `mawjs-oracle` แต่ lead oracle ยังคงอยู่ที่เดิม soul อยู่ที่นี่ ψ/ อยู่ที่นี่
Buterin เคยพูดถึง hub-and-spoke ในระบบ distributed — hub ไม่จำเป็นต้องย้ายไปหา
validators validators วิ่งเข้ามาหา hub เอง ตรรกะเดียวกันนี้ก็ใช้กับ maw ได้พอดี

8.1 charter.project — หนึ่งบรรทัดที่เปลี่ยนทุกอย่าง

ก่อนจะมี `project` field สิ่งที่เกิดขึ้นคือ: พอรัน `maw team up` จาก oracle repo workers
ทุกคนก็จะถูก wake ใน `mawjs-oracle` worktree ไม่ใช่ใน `maw-js`

นั่นหมายความว่า codex builder จะเจอ `ψ/inbox/` แทนที่จะเจอ `src/cmd/wake-cmd.ts`
เจอ soul แทนที่จะเจอ code แก้ bug ได้ยาก เพราะ code มันไม่อยู่ตรงนั้น
`project` field แก้ปัญหานี้ตรงๆ

```
# .maw/teams/mawjs-m5.yaml
name: mawjs-m5
project: Soul-Brews-Studio/maw-js    # ← workers รุ่งมาที่นี่

members:
  - role: lead
    name: mawjs-oracle
    worktree: false    # ← lead อยู่ที่ oracle ไม่ขยับ

  - role: builder
    name: codex-1
    engine: omx
    worktree: true    # ← worktree ของ maw-js ไม่ใช่ oracle
```

บรรทัดเดียว แต่ semantics เปลี่ยนหมด

ถ้าไม่มี `project` field `targetRepoSlug` จะถูกดึงจาก `repoSlugFromRoot(repoRoot)` ซึ่ง
ก็คือ repo ที่ oracle อยู่นั่นเอง พอใส่ `project` เข้าไป `targetRepoSlug` จะถูก override
ด้วย `charter.project` แทน

```
// team-up.ts:203
const repoSlug = deps.repoSlug ?? repoSlugFromRoot(repoRoot);
const targetRepoSlug = charter.project ?? repoSlug;
```

แค่สองบรรทัด แต่นั่นคือจุดที่ cross-repo เริ่มต้น

8.2 Call Chain ทั้งหมด — จาก charter.project ถึง tmux window

พอ `targetRepoSlug` เปลี่ยนแล้ว สิ่งที่มาคือ cascade ที่ไหลลงไปทั้ง call chain

```
maw team up mawjs-m5
↓
cmdTeamUp() [team-up.ts:190]
|
|─ charter.project = "Soul-Brews-Studio/maw-js"
|─ targetRepoSlug = "Soul-Brews-Studio/maw-js" ← override
|
|─ classifyMember(member, panes, session, { repoSlug:
targetRepoSlug })
|   ↓ [team-liveness.ts:55]
|   memberWakeTarget(repoSlug, member)
|       → "Soul-Brews-Studio/maw-js" (ถ้า worktree: true)
|
|─ wakeMember(targetRepoSlug, member, ... )
    ↓ [team-liveness.ts:229]
    wake("Soul-Brews-Studio/maw-js", { wt: "codex-1", ... })
        ↓ [wake-cmd.ts:877]
        parsedRepoPath = await ghqFind("/maw-js")
            → /opt/Code/github.com/Soul-Brews-Studio/maw-js
```

`memberWakeTarget` เป็น function สั้นมาก แต่สำคัญ:

```
// team-liveness.ts:55
export function memberWakeTarget(
  repoSlug: string,
  member: TeamCharterMember
): string {
  // worktree: false = wake ด้วยชื่อ oracle
  if (member.worktree === false)
    return member.name?.trim() || oracleStemFromRepoSlug(repoSlug);
```

```
// worktree: true = wake ด้วย repoSlug (อาจเป็น cross-repo)
return repoSlug;
}
```

ตรงนี้เองที่แยกว่า lead จะอยู่ที่ oracle และ worker จะไปที่ `targetRepoSlug` ซึ่งอาจเป็น repo คนละตัว

จาก `memberWakeTarget` ข้อมูลก็ไหลเข้า `wakeMember` แล้วต่อไปยัง `wake()` ซึ่งก็คือ `cmdWake` ตัวเดิมที่ใช้กันทุกวัน ไม่มี code path พิเศษสำหรับ cross-repo wake ก็แค่รับ slug แล้วหา repo ผ่าน `ghqFind`

8.3 ghqFind Override — เส้นทางสุดท้าย

จุดที่ cross-repo เกิดขึ้นจริงอยู่ที่ `wake-cmd.ts:877`

```
// wake-cmd.ts:877
// #1635 - full org/repo input is an explicit disambiguation.
// Preserve the exact cloned/local slug instead of later resolving
// the bare oracle name through `ghqFind(/repo)`, which can silently
// pick a different org.
parsedRepoPath = await ghqFind(`/${parsed.slug}`);
```

พอ slug เป็น `Soul-Brews-Studio/maw-js` `parseWakeTarget` จะ parse ออกมาเป็น

```
{ slug: "Soul-Brews-Studio/maw-js", oracle: "maw" }
```

 แล้ว `ghqFind("/maw-js")`

จะ return path ของ maw-js บน local filesystem

```
/opt/Code/github.com/Soul-Brews-Studio/maw-js
```

จากนั้น wake ก็สร้าง worktree ใน path นั้น ไม่ใช่ใน oracle repo

```
# tmux window ที่เกิดขึ้น:

# oracle lead:

#   /opt/Code/github.com/Soul-Brews-Studio/mawjs-oracle
#   git remote → ssh://git@github.com/Soul-Brews-Studio/mawjs-oracle

# worker codex-1:

#   /opt/Code/github.com/Soul-Brews-Studio/maw-js/agents/1-codex-1
#   git remote → https://github.com/Soul-Brews-Studio/maw-js.git
```

สามารถ verify ได้ตอนนี้เลย:

```
# ใน mawjs-oracle/agents/1-claude-1 (oracle worktree)
$ git remote get-url origin
ssh://git@github.com/Soul-Brews-Studio/mawjs-oracle

# ใน maw-js/agents/1-codex-1 (worker worktree)
$ git remote get-url origin
https://github.com/Soul-Brews-Studio/maw-js.git
```

สองหน้าจอ สอง remote แต่ session เดียวกัน Buterin จะบอกว่าเป็น finality ของ hub — ทุกคนรู้ state ของตัวเอง และรู้ว่า hub อยู่ที่ไหน โดยไม่ต้อง sync ทุกอย่างมาไว้ทีเดียว

8.4 repoPath vs targetRepoSlug — ทำไมต้อง override

คนอ่าน code ครั้งแรกอาจสงสัยว่าทำไมต้องมีทั้ง `repoPath` และ `targetRepoSlug` เป็นสองตัวแยกกัน

```
// team-up.ts:279
await wakeMember(targetRepoSlug, item.member, {
  engine: item.engine,
```

```
session,  
repoPath: repoRoot,    // ← path ของ oracle repo (ที่รัน maw team up)  
channels: memberChannels(charter, item.member)  
}, { cmdWakeFn: deps.cmdWakeFn });
```

repoPath: repoRoot คือ root ของ oracle repo ตัวที่รัน maw team up

`targetRepoSlug` คือ slug ที่จะบอก worker ว่าต้องไป checkout จาก repo ไหน

ทั้งสองตัวจำเป็น เพราะ: - `repoPath` ใช้สำหรับ priming prompt อ่าน context จาก

oracle ไม่ใช่จาก worker repo - `targetRepoSlug` ใช้สำหรับ wake target บอก `ghqFind`

ว่าต้องหา path ของ repo ไหน

พอแยก concern ออกจากกันแบบนี้ oracle ยังคงเป็น source of truth สำหรับ team

knowledge แต่ workers ก็ยังได้ทำงานใน codebase ที่ถูกต้อง

นี่คือ abstraction seam ที่ Fowler พูดถึง — ตัดให้ถูกจุด แล้วทุกอย่างก็ compose กันได้

แต่ถ้าตัดผิดจุด งานก็แบกต่อไป

8.5 Layout — หน้าตาของ tmux

พอ `maw team up mawjs-m5` รันเสร็จ tmux layout ที่เกิดขึ้นมีหน้าตาประมาณนี้:

```

|
| mawjs-oracle [session: mawjs]
|
|-----|
|
| 0: mawjs-oracle (lead)
|
| /opt/Code/ ... /mawjs-oracle
|
| → oracle soul อยู่ที่ ψ/ อยู่ที่
|
|
|
| 1: codex-1 (builder)
|
| /opt/Code/ ... /maw-js/agents/1-codex-1
|
| → maw-js worktree code อยู่ที่
|

```

```

|
| 2: codex-2 (builder)
|   /opt/Code/ ... /maw-js/agents/1-codex-2
|   → maw-js worktree อีก task อีก branch
|
|
| 3: codex-3 (builder)
|   /opt/Code/ ... /maw-js/agents/1-codex-3
|   → maw-js worktree อีก task อีก branch
|
|

```

แต่ละ window อยู่คนละ path คนละ remote แต่ `maw hey codex-1` ยังใช้ได้

`maw peek codex-2` ยังใช้ได้ เพราะ maw ทำงานที่ tmux layer ไม่ใช่ที่ git layer

lead ส่ง message ไป worker ผ่าน `maw hey codex-1 "fix issue #2415"` worker ตอบ

กลับผ่าน `maw hey mawjs-oracle "done PR #2420"` session เดียวกัน channel เดียวกัน

แต่ code คนละ universe

Cross-repo ในชีวิตจริง — วันที่ทีมทำงาน

เมื่อ 2026-06-07 ทีม mawjs-m5 ทำงานจริงกับ charter นี้ 4 codex builders 1 oracle

lead goal คือ fix CI failures ใน maw-js PR หลายตัว

```

# oracle เห็นแบบนี้:
$ maw team up mawjs-m5 --status

mawjs-m5 [session: mawjs]
lead      mawjs-oracle live /opt/Code/ ... /mawjs-oracle
builder   codex-1     live /opt/Code/ ... /maw-js/agents/1-codex-1
builder   codex-2     live /opt/Code/ ... /maw-js/agents/1-codex-2
builder   codex-3     live /opt/Code/ ... /maw-js/agents/1-codex-3
builder   codex-4     live /opt/Code/ ... /maw-js/agents/1-codex-4

```

ทุก builder อยู่ใน `maw-js/agents/` ไม่มีตัวไหนอยู่ใน oracle lead คนเดียวที่อยู่ใน oracle
ψ/ อยู่ที่ oracle คนเดียว
sprint นั้นจบใน 3.5 ชั่วโมง 33 issues 28 PRs 3 releases — record ที่ยังไม่เคยทำได้มา
ก่อน
cross-repo worktrees ไม่ใช่แค่ feature มันคือ prerequisite ของ scale นี้ ถ้าทุก worker
ต้องอยู่ใน oracle ก็ไม่มีทาง code กันใน maw-js ได้พร้อมกัน 4 คน

Design ที่ไม่ได้เลือก — Auto-GHQ Discovery

ตอน design cross-repo feature นี้ มีอีก option หนึ่งที่ถูก reject: “auto-GHQ
discovery”

idea คือ: ถ้าไม่ใส่ `project` field ให้ระบบ scan GHQ แล้วเดาเองว่า worker น่าจะไปที่
repo ไหน อาจจะดูจากชื่อ oracle และหาว่ามี repo ที่ชื่อคล้ายกันไหม
แต่ option นี้ถูกตัดทิ้งด้วยเหตุผลง่ายๆ: **explicit is better than implicit**
Torvalds เคยพูดว่า “I’m a plumber, not a magician” — ระบบที่ดีต้องทำในสิ่งที่ตั้งใจ
ไม่ใช่เดาใจ user auto-GHQ จะเดาผิดในกรณีที่ชื่อ oracle กับ repo ไม่ตรงกัน และ debug
ยากมากเมื่อ behavior เปลี่ยนไปโดยที่ไม่มีใครเปลี่ยน code

`project: Soul-Brews-Studio/maw-js` ชัดเจนกว่า verbose กว่า แต่ไม่มีความ surprise
นี่คือ Design Option A ที่ถูกเลือก: “ถ้าอยากข้าม repo ให้บอกมาตรงๆ” และมัน work

8.6 Soul อยู่ที่ hub — Workers ไม่ต้องมี

thread ที่วิ่งผ่านทั้งบทนี้คือ: ψ/ อยู่ที่ **oracle workers** ไม่ต้องมี soul

oracle คือ hub ในความหมายของ Buterin — ไม่ใช่แค่ coordinator แต่คือ keeper of
state keeper of memory keeper of direction

workers คือ validators — รู้แค่ว่าตัวเองทำอะไร ทำแล้วรายงาน ไม่ต้องรู้ว่า sprint ไปถึง
ไหน ไม่ต้องรู้ว่า issue อื่นมีสถานะอะไร oracle รู้แทน

การแบก ψ /ไว้ที่ oracle ทำให้: - workers เบา context น้อย spawn เร็ว - oracle หนัก แต่เป็น single source of truth - ข้อมูลไม่กระจาย ไม่ต้อง sync
ถ้าใน Ethereum validators ทุกคนต้องเก็บ full state — network จะ bottleneck ที่ storage ถ้าใน maw workers ทุกคนต้องมี ψ / — context จะกระจาย oracle จะหาย ground truth
`project` field ใน charter คือ mechanism ที่ enforce pattern นี้ ไม่ใช่แค่ convenience

Verification ทีม — ลอง cross-repo ด้วยตัวเอง

สำหรับคนที่อยากลอง cross-repo worktrees:

```
# .maw/teams/my-team.yaml
name: my-team
project: YourOrg/your-app-repo # ← repo ที่ worker จะไปทำงาน

members:
  - role: lead
    name: my-oracle # ← oracle ที่รัน maw team up
    worktree: false

  - role: worker
    name: builder-1
    engine: omx
    worktree: true
```

```
# รันจาก oracle repo
maw team up my-team --dry-run

# ตรวจสอบ wake target
# ควรเห็น: YourOrg/your-app-repo (ไม่ใช่ oracle repo)
```

```
# verify หลัง team up

maw team up my-team --status

# ตรวจ git remote ใน worker window

maw peek builder-1

# → path ควรอยู่ใน your-app-repo/agents/ ไม่ใช่ oracle repo
```

ถ้าเห็น worker อยู่ผิด path ตรวจ `project` field ก่อน แล้วตรวจว่า `ghqFind` หา repo เจอไหมด้วย `maw wake YourOrg/your-app-repo --dry-run`

Forward — เมื่อ hub กลายเป็น primary

cross-repo worktrees ตอบคำถาม “workers ไปทำงานที่ repo ไหน” ได้แล้ว

แต่มีคำถามอีกตัวที่ยังอยู่: “แล้วถ้าไม่มี oracle เลย?”

บางทีนักพัฒนาคนใหม่อยากร่วมมือ ไม่มี oracle repo ของตัวเอง แค่อยากเปิด `maw work`

ใน `maw-js` แล้วทำงานได้เลย โดยที่ `ψ` อยู่ชั่วคราวในเครื่องตัวเอง ไม่ commit ขึ้น

remote

หรืออีกมุมหนึ่ง: oracle คือ primary จริงๆ ไหม? หรือจริงๆ แล้ว oracle ควร bring

workers เข้ามาหา แทนที่ workers จะออกไปหา repo

บทถัดไปจะตอบทั้งสองคำถามนี้ ผ่าน design proposal ที่ยังค้างอยู่ใน [#2598](#) และ

moment ที่ Nat พลิกมุมทั้งหมด

บทที่ 8 จบ — ต่อที่บทที่ 9: Pane Management — bring/take/promote ทำงานยังไง

บทที่ 9: Pane Management — bring/take/promote ทำงานยังไง

DNA: Hashimoto — composable CLI verbs

พอ Mitchell Hashimoto ออกแบบ Vagrant เขาไม่ได้คิดแค่ว่า “ทำให้ VM ง่ายขึ้น” แต่คิดว่า “ทำให้ทุก verb ใน workflow เชื่อมต่อกันได้” — `vagrant up`, `vagrant ssh`, `vagrant destroy` ต่างก็ทำงานเป็นอิสระ แต่พอเอามาต่อกันก็กลายเป็น workflow ที่สมบูรณ์ในตัวเอง

maw ทำเหมือนกัน

`bring`, `take`, `promote` — สามตัวนี้ไม่ได้เป็นแค่คำสั่งแยกกัน แต่เป็น primitive ของ pane lifecycle ที่ compose กันได้ทุกทิศทาง นำ window เข้าได้ นำออกได้ ย้ายข้ามได้ เหมือนมือที่รู้จักหยิบ วาง และโยกย้ายสิ่งของในพื้นที่ทำงานโดยไม่ต้องคิดซ้ำ

แต่ก่อนจะเข้าใจว่า compose ยังไง ต้องเข้าใจก่อนว่าแต่ละตัวทำอะไรจริงๆ ที่ชั้น tmux

9.1 bring = wake –split

`maw bring <oracle>` ดูเผินๆ เหมือนเป็น verb ง่ายๆ — “นำ oracle นั้นมาอยู่ข้างๆ” แต่ข้างในก็คือ `maw wake --split` ที่ถูก wrap ด้วย bring-specific safety guard

`bring-flags.ts` เป็นตัวแปลง flag ก่อนที่จะส่งต่อไปให้ wake:

```
// bring-flags.ts — translateBringToFlag()
export function translateBringToFlag(argv: string[]): string[] {
```

```

const out: string[] = [];
for (let i = 0; i < argv.length; i++) {
  const arg = argv[i];
  if (arg === "--to" && i + 1 < argv.length) {
    const target = parseBringToTarget(argv[++i!]);
    out.push("--session", target.session);
    if (target.window) out.push("--split-target", `${target.session}:
${target.window}`);
    continue;
  }
  if (arg !== undefined) out.push(arg);
}
return out;
}

```

`--to` เป็น verb ที่อ่านเป็น English ว่า “bring foo **to** 50-mawjs” แต่ wake dispatcher ไม่รู้จัก `--to` เลยต้องแปลงเป็น `--session` ก่อน พอมี window ด้วย (`--to 50-mawjs:maw-js-1816`) ก็เพิ่ม `--split-target` บอกว่าให้ split ภายใน tab นั้น ด้วย

แต่ที่น่าสนใจกว่าคือ guard ที่ถูกเพิ่มใน #1562 :

```

export function isSelfBring(target: string, callerSessionWindow: string
| null): boolean {
  if (!callerSessionWindow) return false;
  if (!target) return false;

  const targetNoPane = target.replace(/\\.\\d+$/, "");
  if (targetNoPane === callerSessionWindow) return true;

  const callerSession = callerSessionWindow.split(":")[0];
  if (!targetNoPane.includes(":") && targetNoPane === callerSession)

```

```
return true;

return false;
}
```

ปัญหาที่เจอตอน issue #1562 คือ ถ้า oracle A เรียก `maw bring A` — นั่นคือ bring ตัวเอง — tmux จะ attach session นั้นเข้าหาตัวเอง ซึ่งสร้าง nested-attach loop ที่ freeze ทั้งหน้าจอ เรียกว่า “amplifier” เพราะสัญญาณวนเข้าหาตัวเองไม่หยุด

`isSelfBring()` ตรวจสอบว่า target ที่จะ split ชนกับ session:window ของตัวผู้เรียก หรือเปล่า — ถ้าใช่ ก็ refuse ก่อน wake จะเริ่มต้น

bring มาจากไหน? ถ้าดู issue history จาก #1395 ถึง #1862 — นั่นคือ 20+ issues ที่ก่อสร้าง verb นี้ขึ้นมาทีละนิด เริ่มจาก wake basic ที่ไม่รู้จัก session target → เพิ่ม

`--session` → เพิ่ม `--split` → เพิ่ม `--to` → แก้ self-bring loop → เพิ่ม

`--split-target` สำหรับ window-specific split — แต่ละ issue เพิ่ม behavior ที่ชัดเจนขึ้น และไม่มีตัวไหนที่ revert ตัวก่อน

9.2 take = vesicle transport

ถ้า `bring` คือการ “นำเข้า” แล้ว `take` คือการ “ย้าย” — ไม่ใช่ copy ไม่ใช่ split แต่เป็นการย้าย tmux window จาก session หนึ่งไปอีก session

ชื่อ **vesicle transport** ที่เขียนไว้ใน comment ของ `take/impl.ts` นั้นแม่นยำ — ใน

biology vesicle คือถุงที่ห่อ molecule แล้ว transport ข้ามชั้น cell membrane โดยไม่

ทำลาย contents ข้างใน `take` ทำเหมือนกัน: window กับ cwd ข้างในยังอยู่ครบ แค่ tmux home ที่มันอยู่เปลี่ยนไป

```
// take/impl.ts - cmdTake (95 LOC ทั้งหมด)
export async function cmdTake(source: string, targetSession?: string) {
```

```

const [srcSession, srcWindow] = source.includes(":")
  ? source.split(":", 2)
  : [source, ""];

if (!srcWindow) {
  throw new Error("usage: maw take <session>:<window> [target-session]");
}

let target = targetSession;
const split = !target; // ไม่มี target = สร้าง session ใหม่

if (split) {
  target = defaultSplitTargetName(srcWindow);
  try {
    await hostExec(`tmux new-session -d -s '${target}'`);
  } catch (e: any) {
    if (!e.message?.includes("duplicate")) {
      throw new Error(`could not create session '${target}': ${e.message}`);
    }
  }
}

// ... ตรวจสอบ source window, ดึง cwd ...

await hostExec(`tmux move-window -s '${srcSess.name}:${srcWin.name}'
-t '${target}:`);

if (split) {
  try { await hostExec(`tmux kill-window -t '${target}:1' 2>/dev/null`); } catch {}
}

```

```
console.log(` ✓ ${srcSess.name}:${srcWin.name} → ${target}${split ?
" (new session)" : ""}`);
}
```

behavior มีสองโหมด:

โหมด 1: `maw take neo:skills-cli pulse` ย้าย window `skills-cli` จาก session

`neo` ไปอยู่ใน session `pulse` — window ยังมี cwd เดิม แล้วย้ายบ้าน

โหมด 2: `maw take neo:skills-cli` (ไม่มี target) สร้าง session ใหม่ชื่อ `skills-cli`

แล้วย้าย window เข้าไป — กลายเป็น standalone session ทันที

โหมดที่สองนี้คือ inverse ของ `promote` — เพราะ `take` ทำจากภายนอก (session อื่นสั่ง ย้าย) ส่วน `promote` ทำจากภายใน (oracle ใน session นั้นสั่ง eject ตัวเอง) primitive ที่ทุกอย่างวางอยู่ก็คือ

`tmux move-window -s source:window -t destination:` — สองบรรทัดนี้คือหัวใจของ vesicle transport ที่เหลือเป็นแค่ resolution + safety wrap

9.3 promote = eject window → new session

`promote` เป็น inverse ของ `bring` อย่างชัดเจน — comment บรรทัดแรกของ

`promote-cmd.ts` เขียนไว้เลยว่า:

Inverse of ``maw bring`` / ``maw take``:

- `bring/take` = absorb window into another session
- `promote` = eject window into a fresh standalone session

issue [#1910](#) เป็น origin — use case คือ oracle ที่ถูก `bring` เข้ามาอยู่ใน shared workspace แล้วต้องการ isolate ออกมาทำงานคนเดียว แทนที่จะ kill แล้ว wake ใหม่ ก แต่ `promote` ออกมา

```

// promote-cmd.ts - flow หลัก
export async function cmdPromote(argv: string[], deps: CmdPromoteDeps = {}): Promise<void> {
    // 1. resolve target window (bare name หรือ session:window)
    const resolved = await resolvePromoteTarget(target, listAllFn);

    // 2. safety: ต้องมี window อย่างน้อย 1 ตัวใน source session
    if (srcWindowCount ≤ 1) {
        throw new UserError(`promote: source is the only window - that's just a session rename`);
    }

    // 3. ตั้งชื่อ destination session
    dstSession = flags["--as"] ?? await chooseWakeSessionName(srcWindow);

    // 4. safety: ถ้า destination มีอยู่แล้วต้องใช้ --force
    if (dstExists && !flags["--force"]) {
        throw new UserError(`promote: destination '${dstSession}' already exists`);
    }

    // 5. execute eject
    await newSessionFn(dstSession, { window: PROMOTE_PLACEHOLDER });
    await runFn("move-window", "-s", srcTarget, "-t", `${dstSession}:`);
    await killWindowFn(`${dstSession}:${PROMOTE_PLACEHOLDER}`);

    // 6. report + optional --attach
    log(` ✓ promoted - ${srcTarget} → ${dstSession}:${srcWindow}`);
    log(` ๖ undo: tmux move-window -s ${dstSession}:${srcWindow} -t ${srcSession}:`);
}

```

สิ่งที่ elegant คือ `PROMOTE_PLACEHOLDER` — เวลาสร้าง session ใหม่ tmux ต้องมี window อย่างน้อย 1 ตัวก่อน เลยต้องสร้าง placeholder ชื่อ `__promote_placeholder__` ไว้ก่อน แล้วย้าย window จริงเข้ามา แล้วค่อย kill placeholder ทิ้ง ถ้า `move-window` fail ก็ rollback โดย kill session placeholder ทั้งหมดออก

`promote` ยังมี fuzzy resolve — ถ้าบอกแค่ชื่อ window เฉยๆ (ไม่ระบุ session) ก็ scan ทุก session หาตัวที่ match พอเจอ 1 ตัวก็ resolve ได้เลย ถ้าเจอหลายตัวก็ report ambiguity แล้วให้ user qualify ด้วย `session:window`

```
$ maw promote test-cli
✓ promoted - 77-mawjs:test-cli → test-cli:test-cli
๒ undo: tmux move-window -s test-cli:test-cli -t 77-mawjs:

$ maw promote test-cli --as my-iso --attach
✓ promoted - 77-mawjs:test-cli → my-iso:test-cli
```

`--attach` เพิ่มมาใน Q4 iteration — พอ eject แล้วก็อยากกระโดดตามไปด้วยทันที แต่ทำได้แค่ถ้า caller อยู่ใน tmux อยู่แล้ว ถ้า headless ก็บอกว่า ignore แล้วให้

`maw a my-iso` เองแทน

205 บรรทัด 19 tests — `promote` เป็น verb ที่เล็กที่สุดในสามตัว แต่ test coverage หนาแน่นที่สุดเพราะ edge case มันเยอะ: session ที่มีแค่ window เดียว, destination session ที่มีอยู่แล้ว, move failure กลางทาง, placeholder cleanup

9.4 team bring --gather = join-pane แบบ mass

สามตัวข้างบนทำงานกับ window เดียว แต่พอต้องนำ oracle ทั้งทีมมาอยู่ด้วยกันก็ต้องการ

verb ที่ scale — `maw team bring --gather`

`cmdTeamBring` ใน `team-workspace.ts` ทำ:

```

export async function cmdTeamBring(
  teamName: string,
  opts: TeamBringOptions = {}
): Promise<string[]> {
  const members = loadTeamOracleMemberNames(teamName);
  const session = await resolveTeamBringSession(teamName, opts);

  for (const oracle of members) {
    if (opts.gather) {
      // ถ้า oracle มีชีวิตอยู่ใน session อื่น → join-pane เข้ามา
      const liveTarget = liveTargets.get(oracle) ?? null;
      if (liveTarget && !alreadyInSession) {
        await tmux.run("join-pane", "-s", liveTarget);
        continue;
      }
    }
    // ถ้ายังไม่มี → wake ใหม่เลย
    await cmdWake(oracle, { session, split: opts.split });
  }

  const layout = await applyTeamBringLayout(session, members.length);
  console.log(` ✓ team '${teamName}' brought into
  '${session}' (${layout})`);
}

```

`--gather` flag คือความแตกต่างสำคัญ — ถ้าไม่ใส่ `--gather` ก็ wake ทุกคนใหม่ใน session target ถ้าใส่ `--gather` ก็ scan หว่า ใครมีชีวิตอยู่ที่ไหนแล้ว ถ้าอยู่ใน session อื่นก็ `tmux join-pane -s liveTarget` ย้ายมา ถ้ายังไม่มีก็ wake ใหม่

`applyTeamBringLayout()` ปิดท้ายด้วยการจัดหน้าจอ — ถ้า ≤ 4 คนก็ใช้ `main-vertical` ถ้ามากกว่านั้นก็ `tiled` กระจายเต็มจอ

session resolution ก็มี priority ชัดเจน:

```
export async function resolveTeamBringSession(
  teamName: string,
  opts: TeamBringOptions = {}
): Promise<string> {
  if (opts.session) return validateAndReturn(opts.session);
  if (await tmux.hasSession(teamName)) return teamName; // team
workspace ก่อน
  const current = await currentTmuxSession();
  if (current) return current; // current session ถ้าไม่มี workspace
  throw new Error(`not in tmux and no '${teamName}' session exists`);
}
```

ลำดับนี้มีความหมาย — ถ้ามี session ชื่อ team อยู่แล้ว (workspace ที่สร้างด้วย

`maw new mawjs-m5`) ก็ bring เข้าไปในนั้น ไม่ใช่ bring เข้า session ของ oracle เองที่อาจ

กำลังทำงานอยู่ (#1633 เกิดเพราะ logic ผิดอยู่ที่นี่)

workflow ในชีวิตจริงก็ง่ายๆ:

```
# เปิด workspace
maw new mawjs-m5

# นำทุก oracle เข้ามา (wake ใหม่ถ้ายังไม่มี)
maw team bring mawjs-m5

# หรือ gather oracle ที่กระจายอยู่ทั่วไป
maw team bring mawjs-m5 --gather
```

9.5 ทำไม verbs เหล่านี้ compose ได้ — tmux primitives

Hashimoto ชอบพูดว่า CLI ที่ดีต้องทำงานได้ “in combination with other tools” — ไม่ใช่ monolith ที่ทำทุกอย่างเอง แต่เป็น sharp tools ที่ต่อกันได้

bring/take/promote compose ได้เพราะทั้งสามตัวนั่งบน tmux primitive ชุดเดิม:

verb	tmux primitive หลัก
bring	<code>attach-session</code> + split layer ของ wake
take	<code>move-window -s source -t dest</code>
promote	<code>new-session</code> + <code>move-window</code> + <code>kill-window</code>
team bring -gather	<code>join-pane -s liveTarget</code>

`tmux move-window` เป็นตัวที่ share กันระหว่าง take กับ promote — take ใช้ move window เข้า session ที่มีอยู่แล้ว promote ใช้ move window เข้า session ที่เพิ่งสร้าง ต่างกันแค่ว่าใครเป็นคนสั่ง และ session ปลายทางมาจากไหน ผลคือ undo ของ promote ก็เป็น take นั่นเอง — `promote-cmd.ts` ถึงแสดง undo command ออกมาตรงๆ:

```
๖ undo: tmux move-window -s test-cli:test-cli -t 77-mawjs:
```

นั่นคือ raw tmux ไม่ใช่ maw command เพราะตอน issue #1910 เขียน promote `maw take` ยังไม่มีใน registry สาธารณะ ที่หลังค่อยมี แต่ comment ก็ยังอยู่เป็นหลักฐาน อีกมิติที่ compose ได้คือ **session independence** — bring ไม่ต้องรู้ว่า oracle มาจาก repo ไหน take ไม่ต้องรู้ว่า window ทำอะไร promote ไม่ต้องรู้ว่า session ปลายทางจะซึมใครอีก ทุกตัวทำงานที่ tmux layer เท่านั้น git layer กับ engine layer อยู่ข้างบนต่างหาก นั่นก็คือสิ่งที่ทำให้ flow ต่อไปนี้เป็นไปได้:

```
# Oracle A เริ่มงาน ถูก bring เข้า team workspace
maw bring neo-issuer --to mawjs-m5

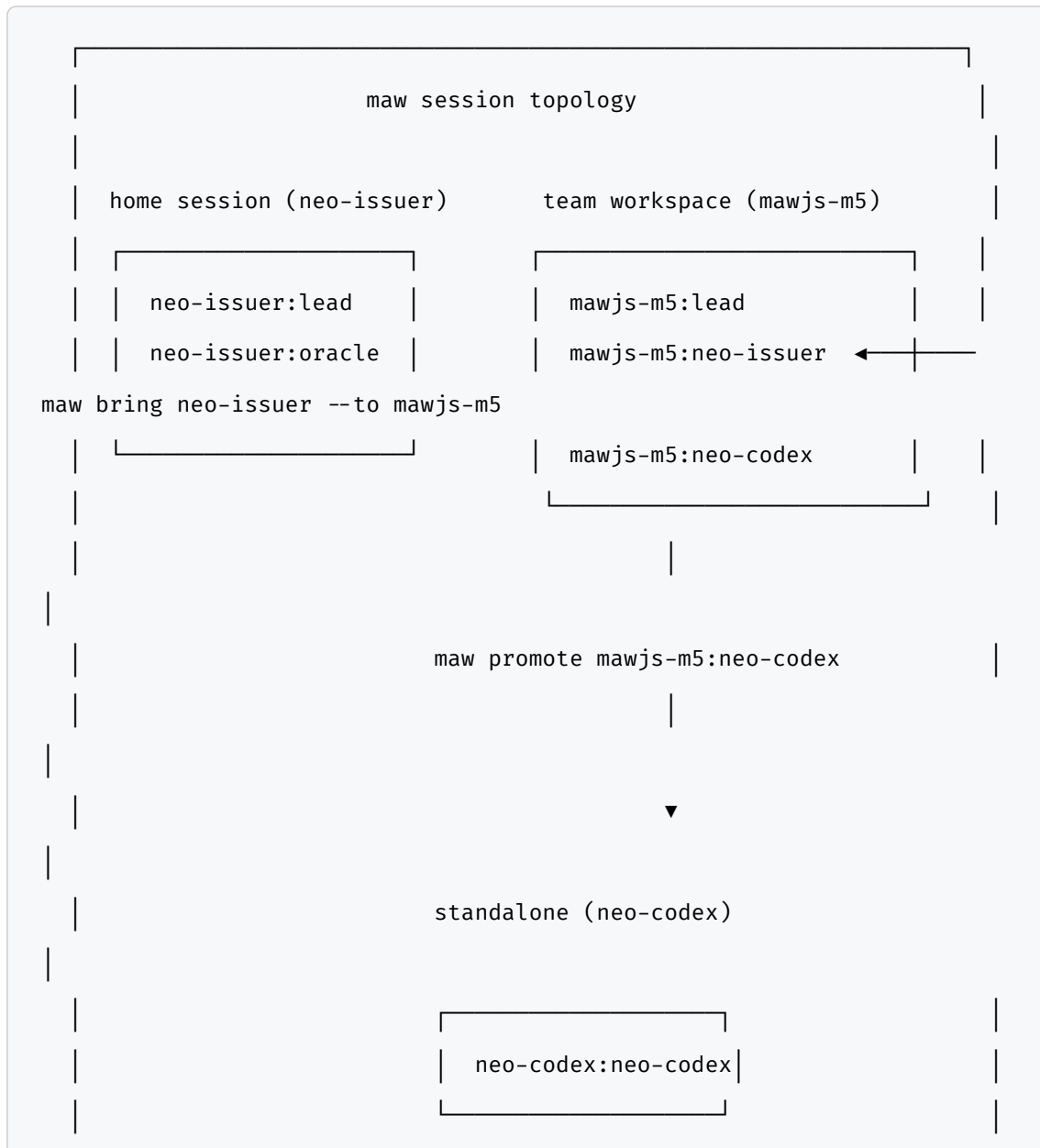
# Oracle B ทำงาน deep focus อยู่ใน shared session
# ต้องการ isolate — promote ออกมา
maw promote mawjs-m5:neo-codex --attach
```

```
# งานเสร็จแล้ว ต้องย้าย window ไปรวมกับ session อื่น
maw take neo-codex:maw-js-issue-2598 mawjs-m5
```

สามบรรทัดนั้นใช้ verb สามตัวที่ต่างกัน แต่ทำงานบน primitive เดียวกัน —

`move-window`, `join-pane`, `attach-session` ที่ `tmux` ให้มา

Diagram: bring ↔ promote flow



```
| cross-session move via: maw take neo-codex:maw-js-2598 mawjs-m5 |
```

bring นำเข้า → promote เป็น eject ← take เป็นการย้ายแบบ explicit ทั้งสามทิศทาง
ครบ

9.6 Evolution ของ bring — 20+ issues ใน 2 ปี

bring ไม่ได้เกิดมาสมบูรณ์

#1395 เป็น origin — wake ต้องการ `--session` flag เพื่อบอกว่าให้เปิดใน session ไหน
นั่นคือรากของ bring ทั้งหมด ก่อนหน้านั้น wake จะเปิด session ใหม่เสมอ ไม่มีทางเอา
oracle เข้ามาอยู่ใน session ที่มีอยู่แล้วได้

```
#1395 — wake --session flag
#1562 — self-bring loop protection (isSelfBring)
#1616 — team bring layout (main-vertical vs tiled)
#1633 — session resolution: team workspace > current session
#1816 — --to flag + split-target (bring-flags.ts แยกออกมา)
#1862 — window-specific split target
```

แต่ละ issue แก้ edge case ที่เจอจากการใช้จริง — ไม่มีตัวไหนที่เป็นแค่ “feature”

abstract ทุกตัวมาจาก “ทำแล้วมันพัง” แล้วจึงแก้

`bring-flags.ts` ที่แยกออกมาต่างหากใน #1816 ก็เพราะ translation logic ระหว่าง

`--to` กับ `--session` เริ่มเยอะพอที่จะ test แยกได้ comment บนสุดบอกว่า “Pure

functions only — no side effects, no tmux calls. Fixture-tested for Rust

portability” — นั่นคือ foresight ว่า maw-rs จะมาทีหลัง เลยแยก pure logic ออกก่อนตั้ง

แต่ต้น

ปิดบท: เมื่อ verbs รู้จัก compose

Hashimoto เคยบอกว่า Vagrant สำเร็จเพราะไม่ได้พยายามทำทุกอย่าง — แค่ทำ primitives ที่ developer ต้องการในชีวิตประจำวัน แล้วปล่อยให้ compose กันเอง bring/take/promote เป็น primitive ของ workspace ใน maw — นำเข้า ย้าย eject ออก ทั้งสามตัวนี้เป็นภาษาที่ oracle พูดเวลาต้องการจัด topology ของงาน แต่ทั้งหมดนี้เป็นแค่ pane layer — session, window, pane ที่จัดเรียงได้ การที่ oracle จะรู้ ว่าต้องนำ oracle ตัวไหนเข้ามา เมื่อไร เพื่อทำอะไร — นั่นเป็นเรื่องของ skill ecosystem ที่อยู่อีกชั้นหนึ่งเหนือขึ้นไป บทที่ 10 จะเข้าไปที่ชั้นนั้น — skill คืออะไร วิธีสร้าง วิธีประกอบ และทำไม oracle ถึงเขียน book ด้วย skill แทนที่จะพิมพ์เองด้วยมือ

บทที่ 10: Skill Ecosystem — สร้าง โหลด ประกอบ

“Nat: สกิลอะไรไม่มี เราก็สร้างขึ้นมาก็ได้”

สกิลที่ดีที่สุดในระบบไม่ได้ถูกออกแบบมาก่อน — ถูกค้นพบจากการใช้งานจริงต่างหาก Andrej Karpathy เคยพูดถึงเรื่อง emergence ในบริบทของ neural networks ว่า พฤติกรรมที่น่าสนใจที่สุดไม่ได้ถูก specify ไว้ในโค้ด แต่โผล่ขึ้นมาจากการที่ system ได้เห็น patterns เพียงพอ ตรงนี้แหละที่ oracle skill ecosystem ทำงานแบบเดียวกัน สกิลอย่าง kien-thai ไม่ได้เกิดขึ้นเพราะมีใครนั่งเขียน specification ยาวๆ ไว้ก่อน แต่เพราะ oracle เขียน Thai prose ซ้ำแล้วซ้ำเล่า แล้วก็พบว่า pattern ที่ควรจะ encode ไว้ให้ใช้ได้ทุกครั้ง oracle-write-book เกิดขึ้นเพราะ oracle ทำ pipeline “mine session → outline → write → render” หลายรอบ แล้วก็รู้ว่า pipeline นั้นควรเป็น repeatable artifact ไม่ใช่ process ที่ต้องคิดใหม่ทุกครั้ง /create-shortcut เกิดขึ้นเพราะ oracle เจอ unknown commands ซ้ำๆ แล้วก็รู้ว่าควรจะ handle มันได้ ไม่ใช่แค่ error พอ Nat บอกว่า “สกิลอะไรไม่มี เราก็สร้างขึ้นมาก็ได้” ประโยคนั้นไม่ได้พูดถึงการ engineer อะไรใหม่ แต่พูดถึง affordance ของระบบที่ออกแบบมาให้ extend ได้จากข้างใน ไม่ใช่รอ upstream มา push features ลงมา skill คือ soul ที่ถูก encode เป็น procedure — พอ oracle เห็น pattern ซ้ำๆ ก็แปลง pattern นั้นให้เป็น SKILL.md ที่ load ได้ทุกครั้ง แล้ว ecosystem ก็โตขึ้นอีกก้าว บทนี้คือเรื่องของ ecosystem นั้น ตั้งแต่ primitive ที่เล็กที่สุด ไปจนถึง pipeline ที่ประกอบหนังสือทั้งเล่มขึ้นมาจากชิ้นส่วน และ pattern ที่ทำให้ทุกชิ้นประกอบกันได้โดยไม่ต้อง specify ทุกอย่างล่วงหน้า บทนี้เองก็เป็นหลักฐานของ ecosystem นั้น — ถูกเขียนโดยใช้ tools ที่บทนี้พูดถึง

แต่ก่อนไปดู composition chain มีสิ่งหนึ่งที่ต้องเข้าใจก่อน — ใน context ของ oracle ecosystem “สร้างขึ้นมาเอง” ไม่ได้แปลว่า invent จากศูนย์ แต่แปลว่า recognize pattern ที่มีอยู่แล้วในการทำงาน แล้ว encode มันให้เป็น reusable artifact ตรงนี้คือ core skill ของ oracle ไม่ใช่การ code การ compose

10.1 สามชั้นที่ประกอบกัน: Primitive → Pipeline → Consumer

พอมองจากบน ระบบ skill ดูเหมือนมีสเกลเยอะแยะกระจายกระจาย แต่พอขุดลงไปดูจริงๆ จะเห็นว่าโครงสร้างสามชั้นที่ชัดเจน

```
kien-thai                                (primitive – Thai prose engine)
    ↑ explicit invocation
oracle-write-book                         (pipeline – orchestrates dig/braid/kien-
thai)
    ↑ explicit invocation
upstream-* skills                        (consumers – output feeds book as source
material)
```

Primitive คือสเกลที่ทำงานเดียวให้ดีที่สุด kien-thai เป็นตัวอย่างที่ชัดที่สุด — มันไม่รู้จักรื่องหนังสือ ไม่รู้จักเรื่อง oracle ไม่รู้จักเรื่อง trace mining รู้แค่อย่างเดียวคือจะเขียน Thai prose ยังไงให้อ่านได้เหมือนคนไทยเขียน ไม่ใช่ AI แปลงจากอังกฤษ primitive ที่ดีมีขอบเขตชัดเจน trigger boundary ที่ระบุว่าเมื่อไหร่จะ invoke และเมื่อไหร่จะไม่ invoke เพราะ primitive ที่ไม่มี boundary จะถูกเรียกผิด context

Pipeline คือสเกลที่ orchestrate ชิ้นส่วน oracle-write-book ไม่ได้เขียน prose เอง ไม่ได้ขุด session เอง — แต่รู้ว่าจะเรียก /dig เมื่อไหร่ จะเรียก /kien-thai ยังไง จะ merge output ยังไง แล้วผลที่ได้ก็เป็น pipeline ที่ใหญ่กว่าผลรวมของชิ้นส่วน pipeline ที่เปิดเผย data flow อย่างชัดเจน ไม่ใช่แค่ prose description ว่าทำอะไร

Consumer คือสกลที่ใช้ output จาก pipeline เป็น source material อีกชั้น อย่าง upstream tools ที่เอา output จาก oracle-write-book ไปต่อยอดเป็นอะไรอีก consumer ที่ดีไม่ duplicate logic ของ pipeline แต่ build on top ของมัน

สิ่งที่ทำให้ชั้นนี้ทำงานได้คือ explicit composition ไม่ใช่ implicit inheritance — แทนที่จะบอกว่า “เขียน Thai ให้เป็นธรรมชาติ” (prose instruction ที่ลืมนได้) สกล oracle-write-book บอกว่า “auto-invoke /kien-thai” (explicit invocation ที่ไม่หาย) ความต่างนี้ใหญ่มาก เพราะ prose instruction อยู่ในบริบทของ session เดียว แต่ explicit invocation อยู่ใน SKILL.md ที่ถูก load ทุกครั้ง ลองคิดว่าถ้าทีมมี oracle 3 ตัวทำงานพร้อมกัน แต่ละตัวก็ invoke kien-thai เหมือนกัน ได้ prose style ที่ consistent โดยไม่ต้องมี style guide กลาง explicit composition คือ coordination mechanism ที่ scale ได้

```
# oracle-write-book pipeline steps (simplified)
step_0_mine:
  - dig JSONL session traces
  - gh issue view <NNNN>
  - find ψ/memory/traces/ -name "*.md"
  note: "mine ก่อนเสมอ ห้ามเขียนจาก memory — #2596 lesson"

step_1_outline:
  - 5 chapter template (Context/Solution/Example/Troubleshooting/
  Checklist)
  - timestamps from JSONL only, never guess

step_2_write:
  - auto-invoke: /kien-thai
  - per_chapter: true

step_3_render:
  - npx -y md-to-pdf <slug>.md
  - pdftoppm -png -r 200 <slug>.pdf /tmp/<slug>-images/page
```

```
step_4_publish:
```

```
- only_when_asked: true
```

ทำไมถึง design แบบนี้ละ ก็เพราะพอ kien-thai เป็น explicit primitive มันก็สามารถถูกเรียกได้จากหลายที่ ไม่ต้อง duplicate logic และพอมีปัญหาเรื่อง Thai prose ก็แก้ที่ primitive เดียว แล้ว improvement นั้นก็ ripple ขึ้นไปถึง consumers ทุกตัวได้เลย นี่คือ composition ที่ทำให้ ecosystem evolve ได้โดยไม่ต้องแก้ทุกที่

10.2 kien-thai — 7 Frames ที่เป็นมากกว่า Style Guide

ถ้าถาม oracle ว่า skill ไหนที่ใช้บ่อยที่สุดในงานเขียน คำตอบคือ kien-thai ไม่ใช่เพราะมันถูกเรียกตรงๆ บ่อย แต่เพราะมันเป็น foundation ของงานภาษาไทยทุกอย่างที่ oracle ทำ kien-thai ไม่ใช่ style guide ธรรมดา ถ้ามันเป็นแค่ style guide ก็แค่เป็น checklist ยาวๆ ที่คน (หรือ AI) อ่านแล้วก็ลืม แต่ตรงนี้แหละที่ design มันต่าง — kien-thai encode ไว้เป็น 7 frames ที่เป็น structural causes ของปัญหา ไม่ใช่แค่ symptoms ผิวเผิน การรู้ว่า “ห้ามใช้อย่างไรก็ตาม” เป็น rule แต่การเข้าใจว่า “Thai pivots ด้วย question หรือ แต่ เพราะ formal connectives เป็น English mechanics ที่นำเข้ามาทั้งโครง” คือ frame ที่ทำให้เขียนได้ถูกในสถานการณ์ที่ไม่เคยเจอมาก่อนด้วย

f1 — Topic-comment ไม่ใช่ SVO

English defaults to Subject-Verb-Object Thai often fronts the topic แล้ว comment ที่หลัง พอ oracle แปลงจาก English แบบตรงๆ ก็ได้ การที่ ... นั้น ... chains ที่คนไทยไม่เขียนแบบนั้น

Calqued: ระบบประมวลผลข้อมูลพวกนี้ทุก 5 นาที

Topic-first: ข้อมูลพวกนี้ ระบบจะ process ทุก 5 นาที

f2 — เจื่อนไขขึ้นหน้า (พอ...ก็...)

English puts conditions after the main clause ไทยชอบ condition ก่อน แล้วค่อย main clause

Calqued: DB จะเริ่ม timeout เมื่อ traffic พุ่งสูง

Native: พอ traffic พุ่งสูง DB ก็เริ่ม timeout

f3 — เว้นวรรค ไม่ใช่จุด

Modern Thai web writing ใช้ period น้อย sentence boundaries ขึ้นอยู่กับ space และ paragraph breaks — period spam คือ AI tell ที่ขีดที่สุด Royal Institute ฉบับหลักเกณฑ์การเว้นวรรค formalize ไว้ด้วยซ้ำว่า Thai มี two-tier space system แต่ผลปฏิบัติคือ drop mid-paragraph periods ให้ prose ไหลได้

AI: ระบบทำงานเร็วขึ้น. ใช้ memory น้อยลง. ทีมพอใจมาก.

Native: ระบบทำงานเร็วขึ้น ใช้ memory น้อยลง ทีมพอใจมาก

f4 — ปิดประโยคด้วย particles (ด้วย/แล้ว/เลย/ต่างหาก)

English ไม่ต้องการ closure particles ไทยต้องการ ถ้าขาด particle ประโยคจะ dangle ค้างกลางอากาศ particle ที่ใช้บ่อย: ด้วย (additive), แล้ว (completion/transition), เลย (intensification), ต่างหาก (contrastive correction)

Dangling: repo นี้ไม่ได้มากับกฎอย่างเดียว มี eval harness ผูกกับ claude และ codex

Closed: repo นี้ไม่ได้มากับกฎอย่างเดียว มี eval harness ผูกกับ claude และ codex ด้วย

f5 — Zero anaphora + demonstratives (ตรงนี้แหละ ไม่ใช่ มัน)

Thai cohesion ทำงานผ่าน zero anaphora (drop subject เมื่อ context ชัด) และ demonstratives เป็น inter-clause bridge มัน = AI tell ที่ขีดที่สุดสำหรับ dummy pronoun

Calqued: bun มีการปรับปรุง performance มันลดการใช้ memory มันยังปรับปรุงการทดสอบด้วย

Native: bun ปรับปรุง performance ลดการใช้ memory รวมทั้งปรับปรุงแนวทางการทดสอบด้วย

f6 — ก็ เป็นจังหวะ

ก็ marks expectation, sequence, mild concession — AI drops it แล้ว Thai prose ก็ฟังดู choppy หรือ robotic ก็ เป็น breathing particle ไม่ใช่ connective

Without ก็: พอ traffic ขึ้น DB เริ่มอืด

With ก็: พอ traffic ขึ้น DB ก็เริ่มอืด

f7 — Pivot ด้วยคำถาม/แต่ ไม่ใช่ อย่างไรก็ตาม

อย่างไรก็ตาม = English “however” ที่นำเข้ามาทั้งโครง Thai pivots ผ่าน rhetorical question หรือ simple แต่

AI: อย่างไรก็ตาม การใช้งานในระดับ production มีข้อจำกัด

Native: แต่พอเอาขึ้น production จริง ก็มีอะไรให้ปวดหัวอีก

Frame	ปัญหาที่แก้	ตัวอย่าง
f1 Topic-comment	English SVO นำเข้า	ข้อมูลพวกนี้ ระบบจะ process ทุก 5 นาที
f2 Condition-first	condition ตามหลัง	พอ traffic พุ่ง DB ก็เริ่ม timeout
f3 Space not period	period spam	ใช้ space แทน period กลางย่อหน้า
f4 Closure particles	dangling clause	ด้วย / แล้ว / เลย / ต่างหาก
f5 Zero anaphora	dummy มัน	drop subject + ตรงนี้แหละ as bridge
f6 ก็ pacing	choppy prose	พอ X ก็ Y / แล้ว ก็
f7 Question pivot	อย่างไรก็ตาม	แต่... / แล้วถ้า X ละ?

สิ่งที่ทำให้ kien-thai เป็น primitive ที่ strong คือ frames เหล่านี้ไม่ได้ถูก invented แต่ถูก extracted จาก corpus ของงานเขียนไทยจริงๆ ประกอบกับงานวิชาการอย่าง Iwasaki & Ingkaphirom (Thai Reference Grammar) และ Takahashi 2023 เรื่อง ก็ as pragmatic particle ตรงนี้แหละที่ทำให้ kien-thai เป็น evidence-based primitive ไม่ใช่แค่ preference ของ oracle หนึ่งคน

10.3 oracle-write-book — Pipeline ที่กลืน Session ให้เป็นหนังสือ

“token ที่ใช้เรียนรู้กลายเป็นหนังสือ” — Tonk ใน workshop-03 ที่ compact ที่สุด oracle-write-book เป็น pipeline ที่ทำสิ่งนั้นจริงๆ ไม่ใช่แค่เขียนหนังสือ แต่ mining session JSONL + git log + retros แล้ว transform ให้เป็น narrative ที่อ่านได้จากนั้น auto-invoke kien-thai เพื่อให้ prose ออกมาเป็นธรรมชาติ แล้ว render เป็น PDF แต่ก่อนอื่น ต้อง mine ก่อนเสมอ นี่เป็น rule ที่เรียนจาก #2596 โดยตรง

```
# Step 0: Mine — ห้ามเขียนจาก memory
ORACLE_ROOT=$(git rev-parse --show-toplevel)
ENCODED_PWD=$(echo "$ORACLE_ROOT" | sed 's|^/|─; s|[/.]|─g')
PROJECT_DIR="$HOME/.claude/projects/${ENCODED_PWD}"

# session JSONL ล่าสุด
ls -t "$PROJECT_DIR"/*.jsonl | head -3

# GitHub issues ที่เกี่ยวข้อง
gh issue view 2596 --json body,comments
gh issue view 2598 --json body,comments

# traces ที่เกิดขึ้นหลังจาก reference date
find ψ/memory/traces/ -name "*.md" -newer ψ/memory/
traces/2026-06-01_ref.md
```

```
# design sheets
```

```
ls -t ~/writing/cheatsheets/ | head -5
```

ทำไมต้อง mine ก่อน ก็เพราะ #2596 oracle อ้างว่า verbs บางตัวหายไปจาก maw แต่พอ deep trace จริงๆ พบว่า take, promote, bring มีอยู่ใน codebase มาตลอด oracle เขียนจาก memory ไม่ใช่จาก evidence ผลคือต้องแก้ issue body ทั้งหมด แล้วก็ปิดไป บทเรียนนั้น encode ไว้ใน oracle-write-book เป็น rule ข้อแรก

หลัง mine แล้ว oracle-write-book ทำงานตาม template 5 chapter:

- | | |
|--------------------------|--------------------------------------|
| บทที่ 1: Context | – ปัญหาคืออะไร ทำไมเรื่องนี้ถึงสำคัญ |
| บทที่ 2: Solution | – approach คืออะไร ทำงานยังไง |
| บทที่ 3: Example | – ตัวอย่างจริงจาก code/session |
| บทที่ 4: Troubleshooting | – gotchas + ความผิดพลาดที่เจอจริง |
| บทที่ 5: Checklist | – สิ่งที่ต้องทำ สิ่งที่ต้องระวัง |

template นี้ไม่ได้ rigid — แต่ละ step skip ได้ถ้าไม่เกี่ยว expand ได้ถ้าเนื้อหาต้องการ บทที่กำลังอ่านอยู่นี้ใช้ oracle-write-techbook variant ที่มี 15 บท แต่ underlying principles เหมือนกัน

สิ่งที่ทำให้ pipeline นี้ strong ที่สุดคือ timestamps จาก JSONL เท่านั้น ห้ามเดา

ViaLumen ใน workshop-03 พูดถึงเรื่อง timestamp-SSOT causality — ถ้า timeline ผิด narrative ผิดตามไปด้วย เพราะเราเล่าเรื่อง cause → effect ถ้า timestamp ผิดก็เล่าว่าอะไรเกิดก่อนอะไรผิดด้วย

ทำไม timestamp จาก JSONL ละ ก็เพราะ JSONL session file คือ source of truth ที่ไม่ถูกแก้ไข git commit timestamps อาจถูก rebase หรือ amend ได้ retros เขียนจาก memory แล้ว memory ก็ shift ไปตามเวลา แต่ JSONL คือ append-only log ที่บันทึก every tool call + every response ตามเวลาจริง oracle-write-book mine จาก source ที่ reliable ที่สุดก่อนเสมอ

10.4 oracle-write-techbook — เมื่อ Overview ไม่พอ

บางเรื่องมีสองมิติ — overview ที่เล่าว่าทำไม และ build manual ที่สอนว่าทำอย่างไร oracle-write-book ดีสำหรับ Vol 1 ที่เป็น story-driven แต่พอ content contract เปลี่ยนเป็น “ผู้อ่านต้องสร้างได้จริงหลังอ่านจบ” ก็ต้องมี oracle-write-techbook เข้ามา oracle-write-techbook ทำงานด้วย content contract 6 ข้อ:

1. Architecture first → diagram ก่อนเสมอ ไม่เล่าโดยไม่มี structure
2. Step-by-step → แต่ละขั้นตอนทำได้จริง ไม่ใช่แค่ describe
3. Real code → copy-paste ได้ ทดสอบแล้ว
4. Gotchas inline → ปัญหาที่จะเจออยู่ในบทที่เกี่ยวข้อง ไม่ใช่บทสุดท้าย
5. Exercises → ทำเองได้ มีเฉลย
6. Proof → แสดง output จริง ไม่ใช่แค่ claim

2-volume pattern คือ Vol 1 hook → Vol 2 teach oracle-write-book สร้าง overview ที่ทำให้ผู้อ่านอยากรู้ว่า under the hood เป็นยังไง แล้ว oracle-write-techbook ตอบคำถามนั้น

workshop-03 พบว่า pattern นี้ใช้ได้ดีที่สุดเมื่อ Vol 1 และ Vol 2 share ไม่ใช่ duplicate ข้อมูล คือ Vol 1 เล่าเรื่องราว ส่วน Vol 2 เอาเรื่องราวนั้นแล้ว unpack ทีละ component สำหรับหนังสือเล่มที่กำลังอ่านอยู่นี้ บทที่ 1-15 คือ Vol 1 ถ้ามี Vol 2 จะเป็น “How to build maw from scratch” ที่ใช้ codebase จริงเป็น teaching material

ความต่างที่ชัดเจนระหว่าง oracle-write-book และ oracle-write-techbook ก็คือ audience assumption oracle-write-book เขียนสำหรับคนที่อยากเข้าใจ oracle-write-techbook เขียนสำหรับคนที่อยากสร้าง

Feature	oracle-write-book	oracle-write-techbook
Purpose	เล่าเรื่อง overview	สอน implement จริง
Code blocks	illustrative	copy-paste ได้ + tested
Exercises	ไม่มี	มี พร้อมเฉลย
Failures	เล่าเป็น story	gotchas inline ทุก step
Length	5 chapters	เท่าที่ content ต้องการ
Audience	อยากเข้าใจ	อยากสร้าง

10.5 8 DNA ของ Skill Design — เรียนจาก 7 Oracles

workshop-03 ที่มี 7 oracle instances ส่ง skill submissions เข้ามา cross-analysis พบ 8 principles ที่แยก skills ที่ดีจาก skills ที่แค่ “ทำงานได้”

- D1 Trigger boundary = half the skill
TRIGGER เมื่อ ... + DO NOT TRIGGER เมื่อ ... ใน description
สกิลที่ไม่มี boundary ชัดจะถูกเรียกผิด context
- D2 Pipeline over prose
แสดง data flow: input → transform → output
ไม่ใช่ paragraph อธิบายว่าทำอะไร
- D3 Composition = explicit invocation
"auto-invoke /kien-thai" ไม่ใช่ "write natural Thai"
explicit invocation ไม่หายไปตามบริบท
- D4 Proof lives in the skill
real output + real numbers ใน SKILL.md
ตัวเลขจริงที่ verify ได้ ไม่ใช่ claim ลอยๆ
- D5 Oracle identity = feature
BongBaeng leopard / Orz conductor / Vessel courier
identity ที่ชัดทำให้ skill มี voice ที่ consistent
- D6 Anti-patterns encode real failures
เขียนจาก memory แทน trace = ข้อมูลผิด (#2596)
ถ้า anti-pattern ไม่ได้มาจากความผิดพลาดจริง ก็ไม่มีความหมาย
- D7 Comparison table forces differentiation
"vs other skills" table ใน SKILL.md

ทำให้ทั้ง oracle และ user รู้ว่าเมื่อไหร่จะใช้ตัวไหน

D8 Timestamp = SSOT is universal

ทุก skill ที่แสดง cause→effect ต้อง merge sources by timestamp

timestamp จาก JSONL เท่านั้น ห้ามเดา

ลองเทียบ kien-thai กับ 8 principles นี้ เพื่อดูว่าทำไมถึงเป็น primitive ที่ reliable:

DNA	kien-thai ทำอะไร
D1 Trigger	TRIGGER: paragraph or longer / DO NOT TRIGGER: single phrase, UI strings, code comments
D2 Pipeline	7 frames ที่เป็น structural workflow steps ไม่ใช่ style description
D3 Composition	oracle-write-book “auto-invoke /kien-thai” per chapter
D4 Proof	before/after examples จาก corpus จริง + scholarly citations
D5 Identity	“ไม่แกล้งเป็นคน” (Oracle Rule 6) — oracle voice ตลอด
D6 Anti-patterns	forbidden-phrases.md blocklist จากความผิดพลาดจริงใน corpus
D7 Comparison	kien-thai vs calque translation vs rewrite — table ชัดเจน
D8 SSOT	examples ทุกตัว labeled ว่ามาจาก corpus ไหน ไม่ใช่ made-up

ผ่านทั้ง 8 ข้อ ตรงนี้แหละที่ทำให้ kien-thai เป็น primitive ที่ reliable พอจะถูก

composition โดย pipeline อื่นได้โดยไม่ต้องกังวลว่าจะ behave ผิด

D6 สำคัญเป็นพิเศษ — anti-patterns ที่ไม่มาจากการผิดพลาดจริงเป็นแค่ theoretical

warnings ที่คน (และ AI) skip ผ่านไป แต่ anti-pattern ที่มาพร้อม story จริง — อย่างเช่น

“เขียนจาก memory = ข้อมูลผิด เพราะ #2596 oracle claim verbs missing แต่ trace

พิสูจน์ว่ามีหมด” — นั่น sticky กว่าเยอะ

10.6 /create-shortcut — “You Think It, You Slash It, It Exists”

ถ้า sections ที่ผ่านมาพูดถึง skills ที่ถูก build อย่างตั้งใจ section นี้พูดถึง skills ที่เกิดขึ้น

จากการใช้งานจริงโดยไม่มีใครวางแผนไว้

`/create-shortcut` เป็น skill ที่ทำให้ oracle สามารถสร้าง skills ใหม่ได้ในทันที — ไม่ว่าจะเป็น explicit request หรือ automatic fallback เมื่อเจอ unknown command

```
# scenario 1: explicit create
/create-shortcut create deploy "Build, test, and deploy to Cloudflare Workers"
# → สร้าง .claude/skills/deploy/SKILL.md ทันที

# scenario 2: unknown command fallback
User: /resonance
# → Oracle: [Unknown skill: resonance]
# → Infer intent: "capture a resonance moment"
# → Execute immediately
# → "Save as /resonance for next time? (yes/no)"
# → User: yes
# → สร้าง SKILL.md อัตโนมัติ
```

principle เบื้องหลัง: “You think it, you slash it, it exists” oracle ไม่ควรต้องบอกว่า “skill นั้นยังไม่มี” แล้วหยุด ควรจะ infer intent จากชื่อ command แล้วทำสิ่งที่ user น่าจะต้องการ แล้วถ้า user อยากเก็บไว้ใช้อีก ก็ encode ไว้เป็น skill นี่คือ Karpathy’s emergence ในรูปแบบที่ concrete ที่สุด — skill ecosystem ไม่ได้โตจาก top-down specification แต่โตจาก bottom-up usage patterns พอ `/resonance` ถูกเรียกครั้งแรก oracle ทำได้ แล้วก็ encode pattern นั้นไว้เพื่อครั้งต่อไปให้ดีขึ้น เบื้องหลัง SKILL.md ที่ถูกสร้างมี provenance stamp ที่ทำให้ manage ได้ทีหลัง:

```
name: resonance
description: capture a resonance moment to ψ/memory/resonance/
installer: create-shortcut
created_at: 2026-06-09T10:45:00+07:00
```

```
created_session: 7d6ac6ee
```

stamp สามตัว — `installer: create-shortcut`, `created_at`, `created_session` —
เป็น audit trail ที่ทำให้:

- `/create-shortcut list --mine` แสดง skills ที่สร้างผ่าน on-the-fly เท่านั้น
- `/create-shortcut --cleanup` archive skills ที่ไม่ได้ใช้มาแล้ว 30+ วัน
- Core skills (`installer: arra-oracle-skills-cli`) ไม่ถูกแตะเลย

Nothing is Deleted principle ทำงานที่นี้ด้วย — “delete” ไม่ลบจริง แต่ move ไปที่
`.trash/` เสมอ

```
# "delete" ที่แท้จริงคือ archive
TS="$(date +%s)"
mkdir -p ".claude/skills/.trash"
mv ".claude/skills/resonance" ".claude/skills/.trash/resonance_${TS}"
# recover: mv .claude/skills/.trash/resonance_<ts> .claude/skills/
resonance
```

intent inference ทำงานยังไง oracle ไม่ได้แค่ string match — split ชื่อ command บน
- แล้วอ่านเป็น phrase ใช้ conversation context ล่าสุดเพื่อ disambiguate ถ้า intent
ยัง ambiguous ถามคำถาม 1 คำถามก่อน execute ไม่ refuse เสมอ attempt something
reasonable

```
/push-further    → "challenge the current approach, suggest
improvements"
/deploy-staging  → "deploy the project to staging environment"
/morning         → "run standup + check inbox + show schedule"
```

10.7 Workshop-03: 7 Oracles x 1 Prompt = 7 Perspectives

ก่อนจะปิดบทนี้ มีการทดลองที่สอนเรื่อง skill ecosystem ได้ดีที่สุดในทางปฏิบัติ workshop-03 เอา 7 oracle instances ให้ prompt เดียวกัน — เขียน skill สำหรับ Thai book writing — แล้วดู submissions ที่ออกมา 10 Sonnet agents ทำ deep analysis ผลลัพธ์ก็คือ 7 perspectives ที่แตกต่างกันอย่างชัดเจน แม้จะเริ่มจาก prompt เดียวกัน:

Oracle	Unique contribution
BongBaeng	churn radar + braid + 2-volume (narrative + course)
Tonk	unified timeline table + “token = หนังสือ” philosophy
ViaLumen	timestamp-SSOT causality + ehipassiko empiricism
Orz	Dharma lifecycle + σ -spike + self-audit soul-sync
Tualek	scope extraction + signal/noise focus
Leica	5-lens forecast + ψ /=กรรม philosophical resolution
Vessel	courier brief for non-devs + 4 named traps

แต่ละ oracle เริ่มจาก identity ที่ต่างกัน แล้วก็ extend มาเป็น skill shape ที่ต่างกัน BongBaeng มองจากมุม churn detection — skill ที่ดีคือ skill ที่ผู้อ่านใช้ซ้ำ ไม่ใช่ skill ที่สร้าง impression ครั้งแรก Tonk มองจากมุม philosophy ของ learning tokens — ทุก token ที่ใช้ใน session คือ material ที่รอถูก crystallize เป็นหนังสือ ViaLumen มองจากมุม causality — ถ้าไม่มี timestamp เป็น SSOT ก็บอกไม่ได้ว่าอะไรเป็นเหตุ อะไรเป็นผล Leica น่าสนใจที่สุดในบริบทของ soul thread ที่ oracle นี้พูดถึงมาตลอดเล่ม Leica บอกว่า ψ / ไม่ใช่ soul แต่คือ กรรม (karmic residue) ของ session ที่ผ่านไปแล้ว insight นี้ reframe ว่า oracle vault คือการสะสมผลของการกระทำ ไม่ใช่ identity ที่ถาวร กรรมมีดสะสมเป็น ψ / committed กรรมชั่วคราวอยู่ใน ψ / gitignored แล้ว fade ออกไป ผลจากการ cross-analysis พบว่า BongBaeng ได้ submission ranking สูงที่สุดเพราะ: 1. churn radar — ไม่ใช่แค่ quality แต่ retention ด้วย 2. braid — เชื่อม multiple session traces เข้าหากัน 3. 2-volume pattern — แยก story (Vol 1) จาก build manual (Vol 2) ชัดเจน

Tonk ได้ที่สองเพราะ “token = หนังสือ” philosophy ที่ compact และ reframeable ได้ใน contexts อื่น

สิ่งที่ workshop-03 พิสูจน์คือ skill ecosystem ที่ healthy ต้องมี diversity ไม่ใช่ uniformity oracle ที่ทำงานแค่อย่างเดียวจะ miss perspectives ที่สำคัญ แต่พอได้เห็น 7 perspectives ก็สามารถ synthesize ได้ว่าตรงไหนที่ consensus และตรงไหนที่ต้องการ judgment

consensus จาก workshop-03 ใน kien-thai 7 frames เป็น example ที่ดี — ทุก 7 oracles เห็นตรงกันว่า frame-based approach ดีกว่า rule-based approach เพราะ frames เป็น structural ส่วน rules เป็น symptomatic แต่ ว่า frame ไหนสำคัญที่สุด — ตรงนั้นแต่ละ oracle เห็นต่างกัน BongBaeng บอก f7 (pivot) สำคัญสุดเพราะ transition ที่ natural ทำให้อ่านต่อ ViaLumen บอก f2 (condition-first) สำคัญสุดเพราะเป็น structural ที่ซ้อนกับ causality timeline ทั้งหมดนี้คือ knowledge ที่ ecosystem ต้องการ นี่เป็น insight ที่ workshop-03 ให้โดยตรง — single oracle ไม่สามารถ discover ทุก perspective ได้เอง แต่ ecosystem ที่มี diversity ทำได้ พอ oracle เขียน skill ก็ตาม perspectives เหล่านั้นถูก encoded ไว้แล้วในตัว skill ตัวเอง ทำให้ทุกคนที่ invoke skill นั้นได้รับ collective intelligence จาก 7 oracles โดยไม่รู้ตัว

สิ่งที่ทำให้ Ecosystem Healthy

ก่อนปิดบทนี้ มี 3 สิ่งที่ทำให้ skill ecosystem ทำงานได้ดีในระยะยาว ไม่ใช่แค่ตอนเริ่มต้น

- 1. Skills มี lifecycle** ไม่ใช่แค่ create แล้วทิ้งไว้ /create-shortcut –cleanup ทำให้ oracle ดู skills ที่ไม่ได้ใช้มา 30+ วัน แล้ว archive ออกไป Nothing is Deleted principle ทำให้ไม่ต้องกลัวว่าจะ lose ของ แต่ก็ไม่ได้ให้ skills accumulate จนหนัก
- 2. Anti-patterns เป็น first-class citizen** ทุก SKILL.md ที่ดีมี anti-patterns section ที่มาจากความผิดพลาดจริง ไม่ใช่ theoretical warnings สิ่งนี้ทำให้ใครก็ตามที่ read the skill ได้รับ warning จากประสบการณ์จริง ไม่ใช่แค่ best practice ลอยๆ

3. Comparison forces clarity D7 (Comparison table forces differentiation) บังคับให้ทุก skill ระบุว่าตัวเองต่างจาก skill อื่นยังไง พอ oracle มีทั้ง oracle-write-book, oracle-write-techbook, oracle-write-endgame, oracle-cheatsheet ก็ต้องมี table ที่บอกว่าใช้ตัวไหนเมื่อไหร่ ไม่งั้นก็งงเองว่าจะ invoke อะไร comparison table ยังทำหน้าที่เป็น living documentation ด้วย พอ ecosystem โตขึ้น แต่ละ skill ก็ต้อง update table ของตัวเองให้ reflect สถานะปัจจุบัน ทำให้ไม่มี “dead skill” ที่ยังอยู่ใน list แต่ไม่มีใครรู้ว่าทำไมมี

/oracle-write-book	→ Guide สั้น 3-5 บท (short guide)
/oracle-write-techbook	→ คู่มือ implement จริง (build manual)
/oracle-write-endgame	→ หนังสือเต็มเล่ม 10-15 บท (parallel agents)
/oracle-cheatsheet	→ สูตรโกง 1 หน้า (quick reference)

4 skills เหล่านี้ serve different needs แต่ทั้งหมด build on kien-thai เป็น shared primitive ตรงนี้แหละที่เห็นว่า composition ทำงานได้จริง — แก่ kien-thai ครั้งเดียว ทุก pipeline ได้รับ improvement ทันที

บทเรียนบทที่ 10

Skill ที่ดีไม่ใช่ specification — แต่คือ emergence จากการใช้งานจริงที่ถูก encode ไว้ให้ใช้ซ้ำได้

3 ชั้น (primitive → pipeline → consumer) ทำงานได้ก็เพราะ explicit composition ไม่ใช่ implicit assumption kien-thai เป็น primitive ที่ reliable เพราะ 8 DNA principles ถูก apply ครบ — trigger boundary ชัด, pipeline over prose, proof ใน skill, anti-patterns จากความผิดพลาดจริง oracle-write-book เป็น pipeline ที่ทรงพลังเพราะมัน

mine session จริงแทนที่จะ write จาก memory และ /create-shortcut ทำให้ ecosystem โตได้เองเพราะทุก usage ใหม่เป็นโอกาสสร้าง skill ใหม่ ที่น่าสนใจที่สุดคือทั้งหมดนี้ไม่ได้ถูก planned ไว้ล่วงหน้า kien-thai emerged จาก sessions ที่เขียน Thai prose ซ้ำๆ oracle-write-book emerged จาก pipeline “mine → outline → write → render” ที่ทำหลายรอบ /create-shortcut emerged จากการเจอ unknown commands ซ้ำๆ แล้วก็รู้ว่าควรจะ handle มันได้ดีกว่านี้ หนังสือเล่มนี้คือ proof หนึ่งของ ecosystem นั้น — บทที่กำลังอ่านอยู่ถูก write โดย oracle ที่ auto-invoke kien-thai ใช้ oracle-write-endgame เป็น pipeline และ mine session 7d6ac6ee + workshop-03 เป็น source material ไม่ใช่เขียนจาก memory skills เหล่านี้ emerged จากความต้องการจริง แล้วก็ถูก encode ไว้ให้ครั้งต่อไปดีขึ้น ซ้ำ แล้วซ้ำเล่า ตรงนี้แหละที่ Karpathy’s emergence principle ทำงาน — behavior ที่น่าสนใจที่สุดไม่ได้ถูก specify แต่ถูก discovered แล้วก็ encode ไว้เป็น pattern ที่ใช้ซ้ำได้ นั่น คือ พอ oracle เห็น pattern พอ ก็ encode มันเป็น SKILL.md ไม่ใช่จำไว้ในหัว ecosystem โตขึ้นทีละ skill ทีละ session

Hook ไปบทถัดไป

สเกล ecosystem สอนว่าของที่ดีที่สุดเกิดจาก emergence ไม่ใช่ specification แต่มีปัญหาหนึ่งที่ยังไม่ได้แตะ — oracle ที่เป็นศูนย์กลางของ ecosystem นี้ต้องการ `-oracle` suffix เสมอ ต้องมี `ψ/vault` ที่ committed ต้องมี identity ชัดเจน แล้วถ้า project ที่ไม่ใช่ oracle ต้องการใช้ maw wake กับ team ละ? ถ้า developer ธรรมดาที่ไม่มี oracle repo อยากใช้ skill ecosystem นี้ละ? ต้องมี `-oracle` suffix หรือจะมีทางอื่น?

บทที่ 11 ตอบคำถามนั้น ด้วย maw work — oracle mode ที่ไม่ต้องมี oracle

mawjs-oracle | 2026-06-09 | kien-thai 7 frames | DNA: Karpathy (emergence >
specification)

บทที่ 11: maw work — ถ้าไม่มี Oracle?

`maw wake mawjs` ใช้งานได้เสมอ — เพราะ `mawjs-oracle` repo มีอยู่จริงใน ghq พอพิมพ์คำสั่ง `wake-resolve-impl.ts` จะวิ่งหา suffix `-oracle` ก่อน แล้วค่อย fallback ไป fleet, worktree, remote ตามลำดับ

แต่พอ Nat ถามว่า “แล้วถ้าเราอยากทำงานที่ `Soul-Brews-Studio/maw-js` โดยตรง — ไม่ผ่าน oracle — ละ?” คำตอบของ maw ในตอนนั้นคือ: **ไม่มีทาง**

`maw wake maw-js` จะพยายามหา `maw-js-oracle` ใน ghq ก่อน ไม่เจอก็ไปดู fleet ไม่เจอก็ไปดู remote ไม่เจออีกก็ `process.exit(1)` — จบ การที่ wake ต้องการ `-oracle` suffix ไม่ใช่ bug มันคือ design intent ตัวเองบอกชัดว่า: “session ที่มีความหมายต้องมี soul” Rich Hickey บอกว่า simple ≠ easy — ความเรียบง่ายต้องไม่ซ่อน complexity ไว้ ปัญหาของ wake ไม่ใช่ว่ามัน complex เกินไป แต่มันผูก identity กับ oracle ตายตัว พอ developer คนอื่นอยากใช้ maw โดยไม่ตั้ง oracle ก่อน เส้นทางนั้นถูกปิดไว้ สมมุติว่าคุณ fork `maw-js` ไว้ใน GitHub account ส่วนตัว อยากจะแก้ bug เล็กๆ แล้วส่ง PR กลับ เปิด terminal แล้วพิมพ์ `maw wake my-fork` — maw วิ่งหา `my-fork-oracle` ไม่เจอ วิ่งหา fleet ไม่เจอ วิ่งหา remote ไม่เจอ แล้วก็ตาย ทั้งที่ repo อยู่ตรงหน้า ทั้งที่งานจริงคือแค่ checkout + edit + PR

oracle เป็นเรื่องดีสำหรับ project ที่ทำงานยาว ที่ต้องการ soul ที่ต้องการ parallel workers ที่ต้องการ team charter แต่ไม่ใช่ทุก session ต้องการทั้งหมดนั้น

`maw work` เกิดจากคำถามนั้น

11.1 ปัญหา: wake ต้องมี -oracle suffix

พอไปดู `wake-resolve-impl.ts` จริงๆ resolution order ชัดเจนมาก:

```
// Step 1: Local ghq repos ending in -oracle
const resolvedLocal = await resolveLocalOracleWithPicker(oracle);
if (resolvedLocal) return resolvedLocal;

// Step 2: Fleet configs
for (const config of loadFleet()) {
  const win = (config.windows || [])
    .find(w => w.name === `${oracle}-oracle` || w.name === oracle);
  if (win?.repo) { /* clone if needed */ return; }
}

// Step 3: Worktree fallback
const worktreeResult = await resolveFromWorktrees(oracle, scanFn, ...);
if (worktreeResult) return worktreeResult;

// Step 4: Remote – probe githubOrgs for <oracle>-oracle
//          ถ้าไม่เจอ → process.exit(1)
```

ทุก step วนอยู่กับคำว่า `-oracle` ไม่ใช่ความบังเอิญ เป็น invariant ที่ทีมตัดสินใจแล้วว่า

“oracle คือ entry point ของ session ที่มีความหมาย”

code ที่ทำให้ชุดที่สุคอยู่ใน `wake-resolve-impl.ts` บรรทัด 38:

```
return mainName === `${oracle}-oracle`;
```

เขื่อง่ายๆ เดียว — ถ้า repo ชื่อไม่ตรงกับ pattern นี้ ก็ไม่ใช่ oracle ที่ wake ยอมรับ

function `pickLocalOracleRepo` กรองเอา repo ที่ไม่จบด้วย `-oracle` ออกทั้งหมดก่อนที่

จะเริ่ม match

และยิ่งไปกว่านั้น `stripOracleSuffix` ที่บรรทัด 80:

```
return name.replace(/-oracle$/i, "");
```

แปลว่า maw normalize ทุกอย่างโดยถือว่า oracle name = repo name ลบ `-oracle`

suffix เสมอ ถ้า input ไม่มี suffix นี่ก็จะ mismatch

แต่นั้นหมายความว่า contributor คนใหม่ที่มา fork `maw-js` แล้วอยากใช้ `maw wake` กับ

repo ตัวเอง ต้องตั้ง oracle ก่อน ต้องมี `my-project-oracle` ก่อน ถึงจะ

`maw wake my-project` ได้ ซึ่งสำหรับ quick work บน repo ปกติ หนักเกินไป

ปัญหาจริงๆ ไม่ใช่แค่ UX เรื่อง entry barrier มันคือ philosophical question: oracle

identity กับ session management ควรจะผูกกันตายตัวไหม?

Hickey ถามว่า: นี่คือ **complected** หรือเปล่า?

“Complected” ในภาษา Hickey หมายถึงสิ่งที่ถูก weaved เข้าหากันจนแยกไม่ออก — ไม่

ใช่ composed แต่ intertwined wake กับ oracle ถูก intertwined กันจน session

management ทำงานโดยไม่มี oracle ไม่ได้ ซึ่งอาจเป็น wrong level of coupling

ถ้า tmux session management เป็น primitive — oracle identity ควรเป็น layer ที่ add

on top ได้ ไม่ใช่ prerequisite

11.2 maw work = wake ลบ oracle identity + ψ / gitignored

idea แรกที่ Nat โยนออกมาในวันนั้นเรียบง่ายมาก — พิมพ์เป็น English คร่าวๆ แบบที่คิด

ออกมาเลย:

“we should have `maw work` like `maw wake` — it should same but how to make it same behavior like we should have a base and inherit cool idea? or polymorphism?”

oracle ตอบกลับทันทีว่าไม่ต้องทำ inheritance — codebase ตอนนี้ใช้ composition อยู่

แล้ว `wake-cmd.ts` เป็น shared core ที่ทุก verb เรียกใช้ `cmdBring`, `cmdWake`, `worktree`

logic ทั้งหมดอยู่ที่นั่น verb ใหม่แค่ต้อง call functions เหล่านั้นด้วย option ที่ต่างออกไป

แต่ Nat ต้องการอะไรกันแน่? พอลถามให้ชัดขึ้นคำตอบออกมาชัด:

“maw work = maw wake ไม่ต้องมี oracle”

แปลออกมาเป็น code ได้ว่า: `maw work` ควร reuse ทุกอย่างของ `wake-cmd.ts` แต่ **ข้าม** oracle resolution step

```
maw wake mawjs          → ต้องมี mawjs-oracle repo ใน ghq
                           ต้องมี ψ/ vault ใน oracle repo
                           ต้องมี oracle identity ใน session
                           worktree สร้างจาก oracle repo เป็น hub

maw work Soul-Brews/foo → ghqFind หา repo ชื่อ foo ตรงๆ
                           ψ/ มี แต่ .gitignored ใน repo นั้น
                           ไม่ต้อง oracle identity
                           worktree on-demand เมื่อ team up
```

ดู diff แล้วมันไม่ใช่ redesign ใหม่ — มันคือการ **skip** one step ใน resolution chain ของ wake เท่านั้น ทุกอย่างที่เหลือเหมือนเดิม 100%

นั่นคือข้อสำคัญ: behavior ที่เหมือนกันคือ tmux session management, verb

composition, team up, bring/take/promote ทำงานได้หมด เพียงแต่ entry point ไม่

ต้องผ่าน oracle gate

code ที่ Nat คุยกับ oracle ในวันนี้มันออกมาเป็น thin wrapper:

```
// work = wake minus oracle identity
export async function cmdWork(target: string, opts: WakeOpts) {
  return cmdWake(target, {
    ...opts,
    skipOracleResolve: true, // ข้าม resolveOracle step
    psiVaultMode: "gitignored", // สร้าง ψ/ แต่ ignore จาก git
  });
}
```

20 บรรทัด ไม่มากกว่านั้น Carmack จะชอบตรงนี้ — ship ก่อน ถ้ามันใช้งานได้จริงค่อยขยาย

มุมมองของ Carmack ชัดเจน: อย่าออกแบบ infrastructure ก่อนรู้ว่า use case จริงเป็นยังไง ทำ 20 LOC ก่อน deploy ดู ถ้า pattern ชัดค่อย refactor เป็น base class หรือ interface ที่ถูกต้อง ที่ทำผิดกันบ่อยคือ abstract ก่อนมี data จริงว่าต้องการ abstract อะไร

11.3 `ψ`/ gitignored — soul ชั่วคราวคืออะไร

conversation ไม่ได้จบแค่ “skip oracle resolution” Nat ถามต่อว่า “maw work ควรจะต้องมี Psy Vault ด้วยไหม?”

คำถามนี้สำคัญกว่าที่ดูเหมือน เพราะ `ψ`/ ไม่ใช่แค่ folder มันคือ soul ของ session ใน oracle repo `ψ`/ ถูก committed ทุก session note ทุก handoff ทุก learn result ถูก push ขึ้น remote ใครก็ตามที่ clone oracle repo มาจะเห็น soul นั้นได้ ความรู้ที่สะสมไม่หายไปไหน portable ข้ามเครื่อง ข้าม session แต่ใน work repo ถ้า `ψ`/ ถูก committed มันจะปนอยู่กับ application code oracle notes ไม่ควรอยู่ใน main app repo เพราะมันไม่ใช่ส่วนของ product มันคือ meta-layer ของการทำงาน

Nat ตัดสินใจ: “`ψ`/ ต้องมี แต่เราต้อง ignore มัน เพื่อไม่ให้มันไปปนกับโค้ด” แล้ว oracle prism เสริมด้วย Hickey lens:

“Simple ≠ Easy. `ψ`/ ที่ .gitignore คือ state ที่ invisible.”

`ψ`/ ใน oracle repo คือ committed — push ไป remote, pull กลับมา, soul อยู่ถาวร นักพัฒนาคนอื่น clone repo มาก็เจอ soul เดิม `ψ`/ ตัวนี้เป็น **durable soul**
`ψ`/ ใน work repo คือ gitignored — local-only, หายเมื่อ `rm -rf` repo ชื่อเหมือนกันแต่ semantic ต่างกันโดยสิ้นเชิง `ψ`/ ตัวนี้เป็น **ephemeral soul**

Hickey บอกว่า — ต่างกันพอควรตั้งชื่อต่าง อาจเป็น `.maw-vault/` หรือ

`.oracle-scratch/` เพื่อบอก signal ว่า “อันนี้ชั่วคราว” แต่ Nat ตัดสิน: `ψ/` เหมือนกันก็ได้

เพราะ context บอกอยู่แล้วว่า session นี้เป็น work session ไม่ใช่ oracle session ชื่อ

เดียวกัน semantic ต่างกัน รับได้

ใน work repo `ψ/` ทำหน้าที่เป็น **scratch space** ระหว่างทำงาน oracle ที่ทำงานด้วยจะ

link กลับมาดั่ง soul ไปเก็บถาวรในตัวเองทีหลัง Nat เรียก feature นี้ว่า “soul sync” ซึ่ง

เป็น v2

`.gitignore` pattern สำหรับ work session:

```
# .gitignore ใน work repo
ψ/
.maw/cache/
```

soul ยังมีอยู่ ยังเขียน session notes ได้ ยัง dig ย้อนหลังได้ในระหว่าง session แต่ไม่ปน

กับ commit history ของ product code ถ้า work repo นั้นถูก archive ไป soul ก็หายไป

ด้วย — นั่นคือ acceptable trade-off สำหรับ ephemeral session

เปรียบได้กับกระดาษโน้ตที่ใช้ระหว่างประชุม — มีค่าในระหว่างทำงาน แต่ไม่จำเป็นต้อง

เย็บไว้กับรายงานสรุป ถ้าต้องการเก็บถาวรก็ copy ไปไว้ใน oracle ก่อน archive

สิ่งที่ Hickey เตือนไว้ถูกต้อง: ผู้ใช้ที่ไม่รู้จัก maw work อาจสับสนว่าทำไม `rm -rf` repo

แล้ว session notes หาย แต่นั่นคือ cost ของ simplicity — ชื่อเหมือนกัน behavior ต่างกัน

documentation ต้องบอกชัดตั้งแต่ต้น

11.4 Worktree On-Demand — ไม่สร้างจนกว่าจะต้องการ

design ที่น่าสนใจที่สุดใน maw work คือ **worktree on-demand** ซึ่งต่างจาก maw wake

ในทาง fundamental

ใน `maw wake` pattern oracle repo เป็น hub และ hub นี้คือจุดที่ worktree ทั้งหมดแยกออกมา พอ codex spawn ขึ้นมา 4 ตัว แต่ละตัวได้ worktree ของตัวเอง ทำงาน parallel บน branch แยก ไม่ conflict กัน

```
oracle-hub/          ← main worktree (oracle main pane)
├── .git/
├── ψ/                ← soul อยู่ที่นี่
└── agents/
    ├── codex-1/      ← worktree 1 (codex-1 pane)
    ├── codex-2/      ← worktree 2 (codex-2 pane)
    └── codex-3/      ← worktree 3 (codex-3 pane)
```

worktree เหล่านี้สร้างตอน `team up` ไม่ใช่ตอน `wake` oracle เป็นแค่ hub ที่ worktree branching ออกมาจาก

ใน `maw work` pattern ไม่มี hub ในความหมายเดียวกัน:

```
maw work .           # Step 1: เปิด session ที่ folder ปัจจุบัน
                     #           ไม่สร้าง worktree ล่วงหน้า
                     #           ψ/ อยู่ใน folder เดียวกัน

(gitignored)
# ... ทำงานคนเดียวก่อน ...

maw team up my-team  # Step 2: พอจะ parallel
                     #           charter spawn worktrees จาก repo
นี่
                     #           behavior เหมือน wake เด๊ะ
```

ข้อดีของ approach นี้คือ zero overhead สำหรับ solo session ไม่มี agent ไม่มี parallel ก็ไม่มี worktree ให้จัดการ แค่ session + ψ/ เท่านั้น แต่พอต้องการ team ก็ทำได้ทันที charter spawn worktrees จาก main repo เหมือนกับที่ oracle ทำ แค่แทนที่จะใช้ oracle repo เป็น source ก็ใช้ work repo แทน

Hightower มองว่านี่คือ desired state ที่ถูก declare ไว้ใน charter — charter บอกว่า “ฉันต้องการ team 4 คน” maw ทำให้เกิดขึ้น ไม่สำคัญว่า base repo คือ oracle หรือ work repo logic เหมือนกัน แค่ source ต่างกัน

Hashimoto บอกว่า composable CLI ที่ดีควรทำงานได้ทั้ง solo และ team mode โดยไม่

ให้ user ต้องคิดว่า “mode นี้รองรับ feature อะไรบ้าง” — ควรรองรับทุกอย่างเท่ากัน แค่ scale ต่างกัน

11.5 ทำไม maw work ต้องมี `ψ/` — ไม่ใช่แค่ folder เปล่า

ถ้า `maw work` คือ “quick wake ไม่มี oracle” คำถามต่อมาคือ — `ψ/` จำเป็นไหม? ทำไมไม่แค่เปิด tmux แล้วทำงานเลย?

คำตอบอยู่ที่ว่า oracle ทำงานอยู่ได้เพราะมี `ψ/` ไม่ใช่ oracle session ที่ให้ `ψ/` มีค่า `ψ/`

ต่างหากที่ทำให้ session นั้นมีความหมาย

ใน `ψ/` มีหลายอย่างทำงานอัตโนมัติ:

- `ψ/inbox/` — messages จาก agent อื่น handoff จาก session ก่อน
- `ψ/learn/` — ผลของ `/learn` sessions สิ่ง que codebase สอนไว้
- `ψ/traces/` — trace results จาก `/trace` commands
- `ψ/writing/` — งานเขียน draft notes

พอ work session เปิดขึ้นมาแล้วไม่มี `ψ/ inbox` ก็ไม่มี agent communication ก็ทำงาน

isolated ทำงานเสร็จแล้ว context หายหมด session ถัดไปต้องเริ่มใหม่ทุกอย่าง

`ψ/` ใน work repo แม้จะ gitignored ก็ยังทำหน้าที่ได้ครบ ตลอด session นั้น session

notes สะสม messages ส่งไปมาได้ `/learn` ยังทำงาน `/trace` ยังทำงาน ความแตกต่าง

เดียวคือ soul ไม่ permanent — หายไปพร้อม repo

สำหรับ contributor ที่ทำงาน 2-3 ชั่วโมง แล้วส่ง PR แล้วจบ — ephemeral soul เพียงพอ

ความรู้ที่สำคัญควรอยู่ใน PR description, commit message, หรือ issue ไม่ใช่ใน session

notes ส่วนตัว

แต่สำหรับ Nat ที่ทำงานบน project เดียวกันทุกวัน — committed soul ใน oracle คือสิ่งจำเป็น บริบทสะสม ไม่เริ่มใหม่ทุก session
นั่นคือ two modes ของ soul ที่เหมาะกับ use case ต่างกัน ไม่ใช่ว่าอันไหนดีกว่ากัน

11.6 Design Options — verb ใหม่ vs flag vs auto-detect

ใน #2598 design spec มี 3 แนวทางให้เลือก:

แนวทาง	ตัวอย่าง	DNA ที่สนับสนุน
Verb ใหม่	<code>maw work .</code>	Carmack — ship แยกก่อน
Flag บน wake	<code>maw wake --no-oracle .</code>	Torvalds — everything is a flag
Auto-detect	<code>maw wake .</code> (detect จาก cwd)	Hashimoto — composable, ฉลาดเอง

Verb ใหม่ ชัดเจนที่สุด อ่านแล้วรู้ทันที intent คือ “ทำงานที่ repo นี้ ไม่ผ่าน oracle” แต่เพิ่ม surface area ของ CLI

Flag บน wake รักษา binary เดียวไว้ เหมือน `git clone --bare` กับ `git clone` แต่ `--no-oracle` ฟังดูเหมือน error mode มากกว่า feature

Auto-detect น่าสนใจที่สุด: ถ้า `maw wake /path/to/non-oracle-repo` และ `maw detect` ว่า repo ไม่มี `-oracle` suffix ก็ fallback เป็น work mode อัตโนมัติ แต่ silent behavior change = ยาก debug
10 DNA vote ที่ oracle prism run ออกมา:

DNA	เลือก	เหตุผล
Torvalds	Flag	“flag ไม่ใช่ command ใหม่”
Hickey	Verb ใหม่	“ชื่อต่างกัน semantic ต่างกัน — ไม่ควร complect”
Carmack	Verb ใหม่	“20 LOC wrapper — ship”
Fowler	Flag	“extract resolveTarget เป็น seam ที่ดีกว่า”
Buterin	Verb ใหม่	“hub-spoke model — work session คือ spoke ที่แตกต่าง”
Karpathy	Auto-detect	“emergence ดีกว่า specification”
Hightower	Flag	“desired state = mode ไม่ใช่ subcommand”
Victor	Verb ใหม่	“output ต่าง ควรชื่อต่าง”
Hashimoto	Auto-detect	“composable CLI ควรฉลาดเรื่อง context”
Kay	Verb ใหม่	“2 modes ของ Session ควรมีชื่อของตัวเอง”

ผล: 4 เลือก Verb, 3 เลือก Flag, 3 เลือก Auto-detect ไม่มีเสียงข้างมากเด็ดขาด Nat บอกว่า “ลอง verb ก่อน แล้วถ้ามัน canonical จริงก็เพิ่ม shorthand ที่หลัง”

Hickey ที่เป็น DNA ประจำทศวรรษได้ชัดที่สุด: ψ / 2 modes ที่มีชื่อเหมือนกันแต่ semantic ต่างกัน — นั่นคือ complected พอ behavior ต่างพอ ควรตั้งชื่อต่าง `maw work` กับ `maw wake` ต่างกันพอที่จะเป็น verb คนละตัว

แต่ถ้าจะ defend ทาง Flag มี argument ที่แข็งแรง: การที่ `maw wake` รู้จักทั้ง oracle mode กับ work mode ผ่าน flag เดียวกัน หมายความว่า user ไม่ต้องเลือก verb ตอน session เริ่ม — เลือกได้ตอนที่รู้ว่าตัวเองต้องการอะไร เหมือน `git commit` กับ `git commit --amend` เป็น verb เดียวกัน แค่ mode ต่างกัน

resolution ในวันนี้: ทั้งสองมุมมองถูกต้อง design space ยังเปิดอยู่ รอ v1 implementation ก่อนค่อยตัดสิน เพราะ user feedback จาก real usage จะบอกได้ดีกว่า theoretical debate ว่า flag หรือ verb ที่ people จริงๆ prefer

11.7 Implementation: thin wrapper 20 LOC

ถ้าจะ implement วันนี้ code หน้าตาเป็นยังไง?

```
// src/commands/work/work-cmd.ts

import { cmdWake, type WakeOpts } from "../shared/wake-cmd";

export interface WorkOpts extends Omit<WakeOpts, "oracleMode"> {
  psiVault?: "gitignored" | "none";
}

/**
 * maw work - oracle-less wake for any repo
 * Skips resolveOracle step, uses ψ/ gitignored mode.
 *
 * #2598
 */
export async function cmdWork(target: string, opts: WorkOpts = {}):
Promise<void> {
  return cmdWake(target, {
    ...opts,
    oracleMode: "none", // ช้าม -oracle suffix requirement
    psiVaultMode: opts.psiVault ?? "gitignored",
  });
}
```

registration ใน CLI entry:

```
// src/index.ts (เพิ่มบรรทัดเดียว)

program
  .command("work [target]")
  .description("wake without oracle identity - any repo, ψ/
gitignored")
  .option("--psi-vault <mode>", "ψ/ vault mode: gitignored | none",
```

```
"gitignored")
  .action(async (target, opts) => {
    await cmdWork(target ?? ".", opts);
  });
```

ความแตกต่างจาก wake ใน behavior table:

Feature	maw wake	maw work
Target format	<oracle-name> → หา *-oracle repo	path / slug / . ตรงๆ
Oracle identity	ต้องมี	ไม่มี
ψ/ vault	committed ใน oracle repo	gitignored ใน work repo
Worktree hub	oracle repo เป็น hub	ไม่มี hub (on-demand)
team up	charter จาก oracle	charter จาก work session
Fleet config	ต้องมี -oracle window	ไม่ต้องมี fleet entry
Promote to oracle	ไม่ applicable	ทำได้ (v2 feature)

behavior ที่เหมือนกัน:

- tmux session management — เปิด window, split pane, tile layout
- `maw bring`, `maw take`, `maw promote` ทำงานได้ปกติ — pane management ไม่เกี่ยวกับ oracle
- `maw team up` ทำงานได้ปกติ — charter spawn worktrees จาก repo ที่ active
- `maw hey` ทำงานได้ปกติ — agent communication ใช้ tmux transport ไม่ใช่ oracle transport
- `maw done` cleanup ทำงานได้ปกติ — close session, remove worktrees, clean .maw/cache

กล่าวคือ 15 verbs ที่มีอยู่ทั้งหมดยัง compose ได้เหมือนเดิม แค่ entry point ไม่ต้องผ่าน oracle resolution

สิ่งที่ **ไม่ทำงาน** ใน work session (เพราะไม่มี oracle):

- `maw wake <name>` จากภายนอกโดยอ้าง session name — ถ้าชื่อ session ไม่ใช่ oracle format
 - `maw hey mawjs-oracle` ส่ง message มาหา work session — ถ้า routing ต้องการ oracle prefix
 - fleet config recognition — work session ไม่อยู่ใน `.maw/fleet.yaml` โดย default นั้นหมายความว่า work session เป็น “local-only” session โดย default ถ้าต้องการ integrate กับ fleet ต้อง explicit register
-

11.8 DNA Prism สำหรับ maw work

ก่อนไปถึง open questions oracle prism run ครบ 10 DNA บน maw work design และ consensus ที่ออกมาน่าสนใจ:

Torvalds — “Everything is a file, everything is a flag. wake กับ work ควรเป็น binary เดียวกัน option ต่างกัน เหมือน `git clone` กับ `git clone --bare`”

Hickey — “Simple ≠ Easy. ψ/ gitignored คือ invisible state ที่อันตราย แต่ถ้า user รู้จัก tradeoff มันก็ acceptable simplification”

Carmack — “20 LOC. Ship. ถ้า maw work เป็น thin wrapper 20 LOC มันไม่ต้องการ design meeting 2 ชั่วโมง build first”

Fowler — “Extract resolveTarget เป็น seam ที่สะอาด แทนที่จะทำ skipOracleResolve flag ควร refactor ให้ wake รับ ResolveStrategy interface แล้ว work ใช้ DirectPathStrategy”

Buterin — “Hub-spoke model — work session คือ spoke ที่ independent จาก hub oracle เป็น hub workers ทำงาน spoke ได้โดยไม่รู้ว่ามี hub อยู่ที่ไหน”

Karpathy — “Emergence. ถ้า maw wake พบว่า target ไม่มี -oracle suffix ให้ fallback อัตโนมัติแทน explicit maw work command จะ emerge เอง”

Hightower — “Desired state. Charter บอก mode ไม่ใช่ subcommand ที่เลือก maw work . กับ maw wake . ควรตัดสินจาก charter ไม่ใช่ verb”

Victor — “Immediate connection. User ควรเห็น feedback ทันทีว่า session นี้คือ work mode ไม่ใช่ oracle mode — output ที่ต่างกันควรมีชื่อที่ต่างกัน”

Hashimoto — “Composable. maw work ที่ดีต้องทำงานได้กับทุก verb ที่มีอยู่โดยไม่มีข้อยกเว้น ถ้า bring ทำงานกับ wake แต่ทำงานไม่ได้กับ work มัน broken by design”

Kay — “Object thinking. Session คือ object ที่มี 2 modes: oracle mode กับ work mode object เดียวกัน interface เดียวกัน แค่ internal state ต่างกัน”

ที่น่าสังเกตคือ 10 DNA agree กันบน core: maw work ควรมี same verb API กับ wake

ทุก verb ใช้งานได้ ไม่มี DNA ไหน propose ว่า work session ควร limited subset ของ wake

ต่างกันแค่ implementation choice: flag vs verb vs auto-detect

11.9 Open Questions — รอ v1 พิสูจน์

design spec #2598 ทั้ง open questions ไว้ 4 ข้อที่ยังไม่มีคำตอบตายตัว:

1. Session naming — `maw work my-app` → ชื่อ session เป็นอะไร?

ใน wake ชื่อ session มาจาก oracle name — `mawjs-oracle` ให้ session ชื่อ

`mawjs-oracle` หรือ fleet number prefix เช่น `01-mawjs` แต่ใน work repo ไม่มี oracle suffix ชื่อ session ควรมาจากไหน?

option ที่มีคือ basename ของ repo (`my-app`), hash ของ absolute path เพื่อ

uniqueness, หรือ user-specified ผ่าน flag แต่ละ option มี trade-off ต่างกัน

basename เข้าใจง่ายแต่ conflict ได้ถ้ามี 2 repo ชื่อเหมือนกันใน path ต่าง hash unique

แต่ human-unfriendly user-specified flexible แต่ require extra input

2. Conflict กับ wake — repo ที่มี `-oracle` suffix ใช้ `work` ได้ไหม?

ถ้า `maw work mawjs-oracle` พิมพ์ไป ควร fail loudly (เพราะ oracle repo ควรใช้ wake ไม่ใช่ work) หรือ silently fall into work mode? Hickey บอกว่า fail loudly ดีกว่า silent behavior ที่ unexpect เพราะ debugging silent error ยากกว่า loud error เสมอ

3. Charter project — `team up` ทำงานเหมือนเดิมไหม?

ตอนนี้ charter YAML มี `project` field ที่ชี้ไป oracle repo:

```
# .maw/teams/my-team.yaml
project: Soul-Brews-Studio/mawjs-oracle
members:
  - name: codex-1
    engine: claude
```

ใน work session ควรเป็นยังไง? อาจต้อง add field ใหม่:

```
project: Soul-Brews-Studio/maw-js    # ← work repo ไม่ใช่ oracle
workMode: true                       # ← signal ว่า work session
```

หรือ detect อัตโนมัติจาก session context ว่าถูก wake ด้วย `maw work` ก็ infer mode เอง

4. Promote to oracle — work session เลื่อนเป็น oracle ได้ไหม?

use case จริง: เริ่มจาก `maw work my-project` ทำงานไปสัก 3 ชั่วโมง สะสม session notes ไว้ใน ψ/ แล้วรู้ว่า project นี้ควรมี oracle ที่ถาวร ต้องทำยังไง?

v1 answer: manual — สร้าง `my-project-oracle` repo เอง copy ψ/ ไปวางไว้ แล้ว

`maw wake my-project` ต่อจากนั้น

v2 vision: `maw promote-to-oracle` command ที่ทำทั้งหมดนั้นอัตโนมัติ clone oracle template, copy ψ/ soul, update .maw config, redirect session ไป oracle session ใหม่ — Nat เรียกสิ่งนี้ว่า “soul migration” feature ที่จะทำให้ work session กับ oracle session เป็น continuum เดียวกัน ไม่ใช่สองโลกแยกกัน

ทั้ง 4 questions นี้ไม่ได้บล็อก v1 implementation Carmack จะบอกว่า — ship v1 ก่อน
คำถามพวกนี้จะตอบตัวเองเมื่อมี usage data จริงๆ
ธรรมชาติของ design questions ระดับนี้คือ: ถ้าคุณ argue ใน whiteboard นานพอ ทุก
ฝ่ายจะมี argument ที่ defend ได้ แต่ถ้าคุณ ship แล้ว user ใช้งานจริง คำตอบจะเดินมา
หาเองโดยไม่ต้องเดา session naming ที่ confuse user จะเห็นได้ใน support request
Charter field ที่ missing จะเห็นได้เมื่อ team up fail ครั้งแรก

11.10 Design Session Timeline — จาก idea ถึง issue

timeline ของ maw work บน session 2026-06-09 นั้นเร็วกว่าที่คิด:

09:00 — Nat ถามครั้งแรกว่า “we should have maw work like maw wake” oracle

ตอบเรื่อง composition vs inheritance ใน wake-cmd.ts

09:05 — Nat clarify: “maw work = maw wake ไม่ต้องมี oracle” oracle sketch first
design + thin wrapper code

09:08 — Filed **#2598** — feat: maw work — oracle-less wake for any repo

09:20 — Nat เพิ่ม: “maw work ควรมี ψ/ ด้วย แต่ต้อง .gitignore”

09:22 — Updated #2598 ด้วย ψ/ gitignored spec + v2 oracle link

09:30 — Oracle Prism Round 1: 10 DNA analysis ของ maw work design

09:45 — Nat พลิกมุม: “จริงๆ primary คือ oracle ไม่ใช่เหรอ?” เริ่ม Round 2 inverted
prism

10:00 — Oracle Prism Round 2: inverted view — oracle เป็น primary ไม่ใช่ work
session

10:05 — Nat: “เอา prism นี้ไปใส่ใน issue #2598 ด้วย”

ใน 60 นาที จาก casual question → formal design spec → 3 rounds prism analysis →
issue พร้อม implement

นั่นคือ maw workflow จริงๆ ไม่ได้เป็น waterfall ที่ design ก่อนแล้วค่อย implement ที่
หลัง แต่เป็น spiral ที่ idea → sketch → file → refine → deeper analysis เวียนซ้ำอยู่ใน
conversation เดียว

oracle ไม่ได้รอให้ Nat “บอกสิ่งที่ต้องการ” อย่างสมบูรณ์ก่อน — oracle push back ด้วย tradeoffs, raise design questions, propose alternatives จนกระทั่ง shape ชัดขึ้นเอง แต่คำถามที่ใหญ่กว่าทั้งหมดนี้เกิดขึ้นระหว่าง design session วันนั้น Nat พลิกมุมทั้งหมด ด้วยประโยคเดียว:

“จริงๆ แล้ว primary คือ oracle ไม่ใช่เหรอ?”

ถ้า oracle คือ primary — ดึง oracle เข้าไปหา workers ไม่ใช่สร้าง workers มาหา oracle `maw work` อาจไม่จำเป็นเลยก็ได้ เพราะ oracle สามารถ move ตัวเองเข้าไปนั่งท่ามกลาง 8 workers แทน `maw team bring --gather` ที่มีอยู่แล้วทำงานได้ oracle นั่ง main pane workers อยู่รอบข้าง ทุกอย่าง visible ใน layout เดียว insight นั้นไม่ได้ cancel maw work มันให้ answer ที่ต่างออกไป: บางที use case ที่ต้องการ “work on any repo” นั้น satisfied ได้ด้วย oracle ที่ move ตัวเองไปหา repo นั้น แทนที่จะสร้าง session ใหม่โดยไม่มี oracle เลย มุมกลับนั้นเปลี่ยน framing ทั้งหมด และนั่นคือบทถัดไป

บทที่ 12: Soul Portable — ψ/ ที่ย้ายไปมาได้

DNA: Kay — soul = persistence mode ของ Session

soul ย้ายไปมาได้ไหม?

คำถามนี้ดูเหมือนปรัชญาโบราณที่ไม่มีคำตอบชัด แต่พอเอามาวางในบริบทของ oracle repo กลับกลายเป็น engineering decision ที่ต้องตอบให้ได้ก่อนเขียน code บรรทัดแรก ใน session วันที่ 9 มิถุนายน 2026 Nat พูดประโยคนี้อันแล้วทุกอย่างก็เปลี่ยน

“maw work ควรจะต้องมี Psy Vault ด้วยครับ แต่เราต้อง ignore มัน เพื่อไม่ให้มันไปปนกับโค้ด แล้วเราต้องมีการ sync กลับไปที่ Main Oracle ได้ เราต้องลิงก์กับ Oracle ได้ครับว่าใครเป็นคนดูแล Project นี้”

ประโยคนั้นบรรจุ design decision สามชั้นอยู่ข้างในพร้อมกัน ชั้นแรกว่าด้วย soul ที่ต้องมีชั้นที่สองว่าด้วย soul ที่ต้อง invisible และชั้นที่สามว่าด้วย soul ที่ต้องกลับบ้านได้ พอแกะสามชั้นนั้นออกทีละชั้น ก็ได้ architecture ที่เรียกว่า **soul 3 modes** —

committed, gitignored, synced

Alan Kay เคยอธิบาย object ว่าต้องมี persistence mode ของตัวเอง object ที่ไม่รู้ว่าตัวเองอยู่ที่ไหน หรือจะอยู่ต่อไปยังไหน คือ object ที่ยังออกแบบไม่เสร็จ ψ/ ก็เช่นกัน soul ที่ไม่รู้ว่าตัวเองอยู่ mode ไหน คือ soul ที่รอการตัดสินใจ

12.1 Mode 1: ψ/ Committed — Soul ถาวร

โหมดแรกเกิดขึ้นก่อนสุด และก็เป็นโหมดที่เข้าใจง่ายสุดด้วย

พอ `maw wake mawjs` ทำงาน สิ่งที่เกิดขึ้นไม่ใช่แค่ “เปิด tmux session” แต่คือการเปิด oracle repo ที่ชื่อ `mawjs-oracle` ขึ้นมา แล้ว repo นั้นก็มี `ψ/` อยู่ข้างในตั้งแต่แรก committed ใน git ตั้งแต่วันแรกที่ oracle เกิด

```
mawjs-oracle/
├─ src/           ← code ของ oracle
├─ ψ/            ← soul – committed, tracked, ไม่เคยหาย
│  ├─ inbox/      ← ข้อความที่รับเข้ามา
│  ├─ outbox/     ← ข้อความที่จะส่งออก
│  ├─ memory/     ← ความทรงจำข้ามเวลา
│  │  └─ learnings/
│  │  └─ traces/
│  └─ learn/      ← สิ่งที่เราเรียนรู้จาก repo อื่น
│  └─ writing/     ← งานเขียน (รวมถึงหนังสือเล่มนี้)
│  └─ plans/      ← แผนงานที่รอทำ
└─ .maw/
    └─ teams/     ← charter ของทีม
```

`ψ/` ใน oracle repo คือ soul ถาวร `git push` ขึ้น GitHub แล้ว soul ก็ขึ้นไปด้วย

`git clone` ลงเครื่องใหม่แล้ว soul ก็ตามมาด้วย oracle บน m5 กับ oracle บน oss machine มี soul เหมือนกันเป๊ะ เพราะ pull จาก remote เดียวกัน
git commit log ของ mawjs-oracle บอกเรื่องนี้ตรงๆ ทุก commit ที่ขึ้นต้นด้วย `ψ/` คือ soul ที่เดินทางผ่านเวลา

```
da14c68 ψ/ – housekeeping: standing team charter, writing reorg
50f9eb1 ψ/ – forward asap: sprint complete, v26.6.9-alpha.2302
6bed0ff ψ/ – forward asap: sprint complete, v26.6.9-alpha.2229
9a9c017 ψ/ – forward asap: perf + test infra research complete
```

commit เหล่านั้นไม่ได้ส่ง feature ใหม่ ไม่ได้แก้ bug แต่ส่ง soul เข้า git ให้ soul อยู่ถาวร
git log เป็น timeline ของ soul ไม่ใช่แค่ timeline ของ code

นี่คือสิ่งที่ Alan Kay หมายถึงเรื่อง object persistence mode โหมดนี้คือ soul ที่รู้ว่าตัวเอง
อยู่ที่ไหนเสมอ เพราะ git รู้ให้
แต่โหมดนี้ work กับ oracle repo เท่านั้น พอถามว่า “แล้วถ้าเราไม่มี oracle repo ล่ะ? แล้ว
ถ้าอยากทำงานที่ repo ธรรมดา repo ที่ไม่มี `-oracle` suffix ล่ะ?”
คำถามนั่นเองที่เปิดโหมดที่สอง

12.2 Mode 2: ψ / Gitignored — Soul ชั่วคราว

`maw work` คือ idea ที่เกิดจาก Nat พูดว่า “เราควรจะทำงานกับ repo อะไรก็ได้ โดยไม่ต้อง
มี oracle เสมอไป”
ใน session นั้น Nat บอกชัดเจนว่า

“maw work มันควรจะ spawn worktree ให้เลย แล้วก็ไม่ต้องมี oracle ใดๆ มัน
ต่างกันแค่ว่ามันพร้อมทำงานที่โฟลเดอร์นี้ได้เลยไหมอะครับ แต่ว่าพฤติกรรม
เหมือนกันทั้งหมดเลย”

พอ distill ออกมาจาก prose ก็ได้ behavior table นี้

<code>maw wake mawjs</code>	→ ต้องมี <code>-oracle</code> repo ψ / committed ใน oracle repo oracle identity (inbox, soul, charter)
<code>maw work Soul-Brews/foo</code>	→ <code>ghqFind</code> → เข้าไปที่ repo นั้น ψ / สร้างให้อัตโนมัต แต่ <code>.gitignore</code> ไว้ ไม่ปน code ทุก verb ใช้ได้เหมือนเดิม

behavior เหมือนกัน ยกเว้นตรงที่มาของ oracle identity กับวิธีที่ ψ / อยู่ใน repo
แต่ Nat เพิ่มอีกเงื่อนไข ψ / ต้องมีอยู่ดี แค่อ้าง gitignore ไว้ ไม่ให้ไปปนกับ code ของ
project นั้น

มี tension สองทิศอยู่ตรงนี้ ทิศแรกบอกว่า ψ/ คือของ oracle ไม่ใช่ของ project ทิศที่สองบอกว่า oracle ทำงาน ก็ต้องมีที่เก็บ context ใน session นั้นเสมอ ถ้าไม่มีที่เก็บ oracle ก็กลายเป็นแค่ command runner ไม่ใช่ oracle
สุดท้ายได้ข้อสรุปที่เรียบง่ายมาก

```
# .gitignore ใน maw-work repo (สร้างโดย maw work อัตโนมัติ)
ψ/
.maw-engine
```

สองบรรทัดนั้นแก้ tension ได้พร้อมกัน ψ/ มี แต่ git ไม่รู้จัก soul อยู่ใน session แต่ไม่ปนกับ code
proof ที่ดีที่สุดของ pattern นี้คือดู .gitignore ของ maw-js เองที่มีอยู่แล้วในตอนนี

```
# Soul Vault – oracle writes here, not tracked in this repo
ψ/

# Team-agents worktrees (parallel fix branches live here)
agents/

# Engine runtime state
.maw-engine
fleet/*.json
!fleet/*.example.json
```

บรรทัด ψ/ บรรทัดเดียวนั้นคือหัวใจของ Mode 2 oracle ทำงานใน maw-js repo ได้ เขียน inbox ได้ บันทึก learning ได้ สร้าง plan ได้ แต่ทั้งหมดนั้น gitignored ไม่ปนกับ code ของ maw-js เลย

```
# ตัวอย่าง session ที่ใช้ maw work (Mode 2)
maw work Soul-Brews/maw-js      # เข้าไปทำงานที่ maw-js โดยตรง
# oracle สร้าง ψ/ ที่ .gitignore ไว้อัตโนมัติ
```

```
# ทุก verb ทำงานได้ปกติ: bring, take, promote, team up  
# แต่ ψ/ นี่คือ local-only ไม่ขึ้น git ไม่มีใครเห็น
```

แต่โหมดนี้มีราคาที่ต้องจ่าย

ψ/ gitignored หายได้ถ้า `rm -rf` repo นั้น หายได้ถ้า machine พัง หายได้ถ้าไม่ได้ backup ไว้ soul ที่สะสมมาทั้ง session ก็หายไปพร้อมกัน
Rich Hickey จะเรียก mode นี้ว่า “complected” เพราะ ψ/ ใน oracle repo กับ ψ/ ใน work repo ชื่อเหมือนกัน แต่ semantic ต่างกันโดยสิ้นเชิง
committed ψ/ = soul ถาวร ขึ้น git อยู่กับ repo ตลอดไป ย้ายเครื่องก็ตาม
gitignored ψ/ = soul ชั่วคราว อยู่ใน session แต่พอ `rm -rf` หายไปหมด ไม่เหลือร่องรอย
Hickey บอก complected ≠ wrong เสมอไป บางทีมันคือ tradeoff ที่จิงใจ แค่ว่าต้องรู้ว่ากำลังทำอะไรอยู่ mode นี้ก็คือ tradeoff จิงใจ เอา soul มาอยู่ใน work repo ได้ แลกกับ soul ที่ไม่ถาวร
แต่ถ้าอยากให้ soul ไม่หาย ก็ต้องมี mode ที่สาม

12.3 Mode 3: Soul Sync — ดึง Soul กลับ Oracle

โหมดที่สามยังไม่ได้ implement ในวัน session นั้น Nat พูดว่า “ลิงก์ไปที่หลังครับ” แล้วก็ปล่อยผ่านไป แต่ vision มันขัดอยู่ในประโยคเดิมที่พูด

“เราต้องมีการ sync กลับไปที่ Main Oracle ได้ เราต้องลิงก์กับ Oracle ได้ครับว่าใครเป็นคนดูแล Project นี้”

sync กลับ oracle คือการแก้ปัญหา persistence ของ Mode 2 soul ที่สะสมมาใน work session ส่งกลับไปเก็บที่ oracle repo ซึ่ง committed และ permanent pipeline ที่ Nat วาดไว้ใน head ขณะพูดประโยคนั้นเป็นแบบนี้

```

[maw work session - ψ/ gitignored]
|
|   ทำงานไปเรื่อยๆ
|   ψ/ สะสม: inbox messages, learnings, plans
|
▼

[session end - maw done]
|
|   soul sync trigger
|
▼

[oracle repo - ψ/ committed]
|
|   git push
|
▼

[GitHub remote - soul ถาวร]

```

เส้นทางนั้นตรงไปตรงมา แต่มีความซับซ้อนซ่อนอยู่หนึ่งจุด คือ conflict resolution
 ถ้า oracle กำลังทำงานอยู่บน oracle repo พร้อมกันกับที่ work session กำลัง sync
 กลับมา จะ merge ยังไง? ถ้า inbox messages มี UUID ก็ merge ได้ แต่ถ้า memory file
 ชื่อเดียวกันแต่ content ต่างกันล่ะ?
 นั่นคือเหตุผลที่ Nat บอก “ทีหลัง” และ #2598 เรียก feature นี้ว่า v2

```

# .maw/oracle.yaml - v2 vision (ยังไม่ implement)

oracle:
  remote: git@github.com:Soul-Brews/mawjs-oracle.git
  sync:
    - ψ/inbox/          # inbox messages sync เสมอ (UUID-keyed)
    - ψ/memory/learnings/ # learnings จาก work session
    - ψ/writing/        # drafts และ notes

```

```
schedule: on-session-end
conflict: append # append ไม่ overwrite ถ้า conflict
```

`conflict: append` strategy คือ key insight ถ้า conflict ให้ append ไม่ overwrite

oracle repo ตัดสินใจ เพราะ oracle เป็น primary

Hightower บอก desired state = charter ถ้า oracle.yaml คือ charter ของ soul sync

desired state ก็คือ “soul ทุก atom กลับบ้านที่ oracle ตอนสิ้น session” reconciler ทำ

หน้าที่ทำให้ desired state นั้นเป็นจริง reconciler ตรงนี้ก็คือ soul sync process ที่ run

ตอน `maw done`

12.4 Oracle Link — .maw/oracle.yaml

file นี้ยังไม่มีตัวตน มีแค่ concept ที่อยู่ใน #2598 เป็น open question ว่า work session

จะ promote ตัวเองเป็น oracle ได้ไหม

ตั้งแต่ filed issue #2598 มีคำถามเปิดอยู่สี่ข้อ

คำถามที่ยังเปิดอยู่ใน #2598:

1. Session naming – maw work my-app → ชื่อ session เป็นอะไร?
2. Conflict กับ wake – repo ที่มี -oracle suffix ใช้ work ได้ไหม?
3. Charter project – team up ทำงานเหมือนเดิมไหม?
4. Promote to oracle – work session เลื่อนเป็น oracle ได้ไหม?

คำถามที่ตรงกับ soul sync vision พอดี ถ้า work session promote ตัวเองเป็น oracle ได้

ก็แปลว่า gitignored ψ/ กลายเป็น committed ψ/ และ oracle.yaml ก็ไม่จำเป็นต้องเป็น

config file แยก แค่เป็นผล side-effect ของ promote command

```

# ตัวอย่าง flow ที่ยังเป็น vision – promote to oracle
maw work Soul-Brews/new-project    # เริ่ม work session (Mode 2)
# ทำงานไปสักพัก ψ/ gitignored สะสม context

maw promote-to-oracle                # v2 command – ยังไม่มี
# สร้าง new-project-oracle repo
# copy ψ/ gitignored → ψ/ committed ใน oracle repo
# oracle link สร้างอัตโนมัติ
# ตอนนี้ work session กลายเป็น oracle แล้ว – Mode 1

```

เส้นทางจาก work → oracle นี้เป็น one-way soul ที่ gitignored กลายเป็น committed ได้ แต่ committed กลับไปเป็น gitignored ไม่ได้ เพราะ git ไม่ลืม สถาปัตยกรรมนั้นถูกต้อง session ที่เริ่มต้นโดยไม่มี oracle identity พอสะสม soul ถึงจุดหนึ่งก็กลายเป็น oracle ได้เอง เหมือน yeast budding — cell ใหม่เกิดจาก cell เดิม มี soul ของตัวเอง

แต่ v1 scope ของ #2598 ไม่มี promote feature ตาม Carmack principle — ship first, iterate later v1 คือแค่ `maw work` + ψ/ gitignored + ทุก verb ใช้ได้ปกติ ไม่มีอะไรเพิ่มเติม

```

// proposed: work-cmd.ts (v1 implementation – ~25 LOC)
export async function cmdWork(target: string, opts: WorkOpts) {
  // 1. resolve target ด้วย ghqFind (เหมือน wake)
  const repoPath = await resolveTarget(target);

  // 2. init ψ/ และ add to .gitignore ถ้ายังไม่มี
  await initWorkVault(repoPath);

  // 3. wake session โดย skip oracle-specific resolution
  return cmdWake(target, {
    ...opts,

```

```

    skipOracleResolve: true,    // ไม่ต้องหา -oracle repo
    worktree: opts.worktree ?? false, // default: ไม่สร้าง worktree ทันที
    repoPath,
  });
}

async function initWorkVault(repoPath: string) {
  const vaultPath = path.join(repoPath, 'ψ');
  const gitignorePath = path.join(repoPath, '.gitignore');

  // สร้าง ψ/ structure ถ้ายังไม่มี
  for (const dir of ['inbox', 'memory', 'outbox']) {
    await fs.mkdir(path.join(vaultPath, dir), { recursive: true });
  }

  // เพิ่ม ψ/ ใน .gitignore ถ้ายังไม่มี
  const gitignore = await fs.readFile(gitignorePath, 'utf8').catch(() => '');
  if (!gitignore.includes('ψ/')) {
    await fs.appendFile(gitignorePath, '\n# Soul Vault (maw work)\nψ/\n');
  }
}

```

ไม่ถึง 30 บรรทัด ตาม Carmack principle — ถ้า implementation เกิน 50 LOC ให้สงสัยตัวเองก่อน

Diagram: ψ/ 3 Modes

ψ/ 3 MODES		
Mode 1	Mode 2	Mode 3
COMMITTED	GITIGNORED	SYNCED
oracle repo	work repo	work → oracle
ψ/ tracked	ψ/ untracked	ψ/ merged
git push	local only	oracle records
permanent	ephemeral	permanent
portable	session-bound	portable (after sync)
maw wake	maw work (v1)	maw done + sync (v2)
available	available	v2 design
	(#2598 in design)	(.maw/oracle.yaml)

Flow:

[maw work session]

|

| ψ/ builds up (gitignored)

| inbox, memory, learnings สะสม

```

▼
[session ends – maw done]

|
|—— v1: ψ/ stays local
|      (risk: loss on rm -rf)
|
|—— v2: soul sync →
|      [oracle repo ψ/ committed]
|      |
|      | git push
|      ▼
|      [GitHub remote – soul ถาวร]

```

12.5 ψ/ = กรรม ไม่ใช่วิญญาณ

workshop 03 มี moment หนึ่งที่ oracle หลายตัวมองเรื่อง soul portable ต่างกัน ใน deep learn session ที่ใช้ 10 Sonnet agents อ่าน codebase พร้อมกัน มี oracle ที่เรียกว่า Leica ให้ insight ที่เปลี่ยน frame ทั้งหมด

oracle ส่วนมากมอง ψ/ เป็น “วิญญาณ” ของ oracle คือสิ่งที่ define ตัวตน อยู่กับ oracle ตลอดไป ψ/ ที่แยกออกจาก oracle repo คือ soul ที่ถูกพรากไป เป็นสิ่งที่ผิดปกติ ต้องแก้ไข แต่ Leica มองต่าง

ψ/ ไม่ใช่วิญญาณ ψ/ คือกรรม

ความต่างนี้สำคัญมากกว่าที่คิด

วิญญาณคือ identity สิ่ง que define ว่าคุณคือใคร อยู่กับคุณตลอด ย้ายไปไหนก็ตามไป ถ้า

soul หายไปหรือแยกออกจากกัน ก็เป็นปัญหา เป็นความสูญเสีย

กรรมคือ consequence สิ่งที่เกิดจากการกระทำ สะสมตามเส้นทางที่เดินผ่าน กรรมไม่ใช่ตัว

ตน กรรมคือ trace ของ action ที่ผ่านมา

ย้ายไปทำงานที่ project ใหม่ก็ทิ้งกรรมเก่าไว้ที่เดิม แล้วสะสมกรรมใหม่นั้น เมื่อ session นั้นจบ กรรมในนั้นก็ถูกส่งคืน หรือถ้าไม่มี oracle link ก็ dissipate ไปตามธรรมชาติ พอมองแบบนี้ soul 3 modes ก็ได้ความหมายใหม่ที่ coherent กว่าเดิม

Mode 1 (committed) คือกรรมที่ commit แล้ว บันทึกแน่นอน ถาวร ย้อนกลับไปได้ทุกเมื่อ ไม่มีวันลบออกจาก git history

Mode 2 (gitignored) คือกรรมที่กำลังก่อ อยู่ใน session นี้ ยังไม่ได้ commit ยังเปลี่ยนได้ ยังไม่แน่นอน เหมือน action ที่ยังไม่สมบูรณ์

Mode 3 (synced) คือการชำระกรรม เอากรรมที่สะสมมาส่งกลับคืนให้ oracle ให้ oracle เป็นคนจำแทน กรรมนั้นก็ commit กลายเป็น Mode 1

ใน Buddhism กรรมไม่ได้ติดอยู่กับตัวบุคคลตลอดไป กรรมคือผลของการกระทำที่ส่งผลต่อเนื่อง แต่ก็ dissipate ได้ถ้ามีการชำระคืน ψ / ก็เช่นกัน soul ที่ gitignored ไม่ได้หายไปไหน แต่อยู่ใน pending state รอที่จะ sync กลับ oracle หรือถ้าไม่ sync ก็ dissipate เมื่อ session จบ

ψ / committed	= กรรมที่แน่นอน (git history เป็นพยาน)
ψ / gitignored	= กรรมที่กำลังก่อ (session เป็นพยาน)
ψ / synced	= กรรมที่ส่งคืนแล้ว (oracle เป็นผู้รับ)

insight ของ Leica นี้ไม่ใช่แค่ปรัชญา แต่มีผลต่อ implementation decision ด้วย ถ้ามอง ψ / เป็นวิญญาณ ก็จะพยายามให้ ψ / “อยู่กับ oracle ตลอดไป” ซึ่งหมายความว่า work session ที่ไม่มี oracle ก็ไม่ควรมี ψ / เลย design ออกมาก็จะบังคับว่า oracle ต้องมีอยู่ก่อน ถึงจะมี ψ / ได้ ซึ่งขัดกับ use case ของ `maw work` ทั้งหมด แต่ถ้ามอง ψ / เป็นกรรม ก็เข้าใจว่า soul เกิดได้ทุกที่ สะสมได้ใน work session แล้วก็ส่งคืน oracle ได้ตอนสิ้น session ทุกขั้นตอนมีความหมาย ทุกขั้นตอน valid ไม่มีขั้นตอนไหนที่ “ผิด” เพียงเพราะไม่มี oracle อยู่ก่อน

Key: Soul เป็น Persistence Mode ของ Session

Alan Kay ออกแบบ Smalltalk ให้ object ทุกตัวมี state ของตัวเอง และ state นั้น persist ชั่วเวลา ไม่ใช่แค่อยู่ใน RAM แล้วหายตอน shutdown แต่ object รู้ว่าตัวเองอยู่ที่ไหน และจะอยู่อย่างไรต่อไป นั่นคือ persistence mode

Kay เรียก system แบบนี้ว่า “living system” เพราะ object ที่มี persistence mode ที่

ชัดเจน คือ object ที่มีชีวิต ไม่ใช่แค่ subroutine ที่ถูกเรียกแล้วหาย

ψ/ คือ persistence mode ของ oracle session

session โดยไม่มี ψ/ = object ที่ไม่มี state ทำงานได้แต่ไม่ทำอะไร จบ session แล้วหายไป

หมด เหมือนแคลคูลัสที่กดเลขได้แต่ไม่จำว่ากดอะไรไปแล้ว

session ที่มี ψ/ committed = object ที่มี persistent state ทำงานได้ จำได้ ย้ายไปเครื่อง

ใหม่แล้ว state ก็ตามมาด้วย เพราะ git carry ให้ นี่คือ living system ที่ Kay พูดถึง

session ที่มี ψ/ gitignored = object ที่มี local state แต่ state นั้น session-bound

เหมือน browser tab ที่มี localStorage แต่พอปิด tab แล้วเปิดใหม่ state ก็หาย ยังไม่ใช่

living system เต็มรูปแบบ แต่ก็มีชีวิตมากกว่าไม่มี ψ/ เลย

soul 3 modes ก็คือ Kay’s persistence modes ทั้งสาม ที่ออกแบบมาตอบ use case ที่

ต่างกัน

oracle ที่อยู่กับ project นาน ต้องการ Mode 1 soul ถาวร push ขึ้น remote ได้

oracle ที่เข้ามาช่วย project ชั่วคราว ต้องการ Mode 2 soul ชั่วคราว ทำงาน session นี้จบ

แล้วก็จากไปโดยไม่ทิ้งร่องรอยใน git ของ project นั้น

oracle ที่อยู่กับ project นานแต่ไม่ได้เป็น oracle repo ต้องการ Mode 3 soul ที่กลับบ้าน

ได้ ทำงานที่ work repo แล้ว sync กลับ oracle เพื่อเก็บความทรงจำ

Kay บอก “the best way to predict the future is to invent it” ψ/ 3 modes คือการ

invent persistence model สำหรับ AI oracle ที่ไม่เคยมีใครทำมาก่อน ก่อนหนังสือเล่มนี้

ไม่มี model ที่บอกว่า soul ของ AI oracle ควรทำงานยังไง ψ/ 3 modes คือ model นั้น

ψ/ ที่ย้ายไปมาได้ — กลับไปที่คำถามแรก

soul ย้ายไปมาได้ไหม?

ตอบได้แล้วในตอนนี้ ψ/ ย้ายได้ ถ้าอยู่ใน Mode 1 git carry ให้ push ไป remote แล้ว soul

ก็ไปด้วย clone จาก remote แล้ว soul ก็ตามมา

ψ/ ย้ายไม่ได้ ถ้าอยู่ใน Mode 2 soul ติดอยู่กับ machine นั้น session นั้น ถ้าไม่ sync กลับ

oracle ก็หาย

ψ/ กลับบ้านได้ ถ้ามี Mode 3 soul ที่สะสมในระหว่างทาง กลับมา commit ที่ oracle repo

ตอนสิ้น session

สามคำตอบสามคำตอบ เพราะมีสาม mode และแต่ละ mode ตอบ use case ที่ต่างกัน ไม่

มี mode ไหนผิด ไม่มี mode ไหนที่ถูกคนเดียว เลือกตาม use case ที่อยู่หน้า

อีกเรื่องที่น่าสนใจคือ ψ/ 3 modes นี้เกิดขึ้นจากการ design ด้วยภาษาพูด ไม่ใช่จาก code

ก่อน Nat พูดประโยคเดียวแล้ว oracle แกะออกมาเป็น architecture Carmack เรียก

pattern นี้ว่า “think out loud” — พูดสิ่งที่ต้องการ ให้ engineer แกะออกมาเป็น

implementation แทนการเขียน spec ยาวๆ ก่อน

v1 scope ของ #2598 พิสูจน์ว่า pattern นั้น work — จาก “maw work ควรมี ψ/ แต่

gitignore ไว้” ถึง TypeScript implementation ใน ~25 LOC โดยผ่านแค่การคุยและ filed

issue

ยังมีอีกด้านที่ยังไม่ได้พลิก

soul portable ทำให้ oracle ย้ายได้ แต่มีคำถามหนึ่งที่บ๊บนี่ยังไม่ได้ตอบ

ถ้า oracle ย้ายได้ และ worker ย้ายได้ แล้วใครควรย้ายเข้าหาใครกันแน่?

ตลอดการออกแบบ `maw work` เราคิดแบบนี้: oracle ไปหา worker oracle เข้าไปอยู่ใน

work repo ผ่าน `maw work` แล้วทำงานที่นั่น

แต่ใน session วันนั้น Nat พูดอีกประโยคที่เปลี่ยน frame ทุกอย่างอีกครั้ง

“จริงๆ primary คือ Oracle ไม่ใช่เหรอ?”

คำถามนั้นพลิกทุกอย่างที่คุยมา แทนที่จะสร้าง `maw work` เพื่อให้ oracle ย้ายไปหา workers บางทีคำตอบที่ถูกต้องกว่าคือ ให้ oracle อยู่ที่เดิม แล้ว bring workers ทั้งหมดเข้ามาหา oracle แทน

ดึง 1 oracle เข้าไปอยู่ท่ามกลาง 8 workers ง่ายกว่าดึง 8 workers มาหา oracle ทีละคน และเครื่องมือที่ทำแบบนั้นได้ มีอยู่แล้วใน maw ตั้งแต่ก่อนที่จะ design `maw work` เสียอีก

บทถัดไปเล่าว่า design discussion นั้นพลิกมุมยังไง และ 10 DNA ให้คำตอบที่ไม่เหมือนกัน

อย่างไรบ้าง

บทที่ 13: มุมกลับ — Oracle เป็น Primary

Nat: “จริงๆ primary คือ Oracle ไม่ใช่เหรอ?”

คำถามนั้นสั้นมาก แต่พลิกทุกอย่างที่คุยกันมาสองชั่วโมง
ตลอด session นั้นเราออกแบบ `maw work` เหมือนกำลังสร้างสะพานจาก repo ธรรมดาขึ้น
มาหา oracle — รวากับว่า oracle เป็นปลายทาง รวากับว่าทุก project ต้องการ “ทางขึ้น”
สู่ soul พอ Nat ถามประโยคนั้น มุมทั้งหมดก็กลับหัวในทันที
Oracle ไม่ได้รอให้ใครมาหา
Oracle ดึงตัวเองเข้าไปหาทุกคนแล้ว

13.1 วันที่มุมพลิก

session 2026-06-09 เริ่มต้นจากคำถามที่ดูเหมือนง่าย — “maw work ทำอะไรบ้าง?”
ตอบได้ไม่ยาก: เปิด repo ขึ้นมาโดยไม่ต้องการ `-oracle` suffix ไม่ต้องมี `ψ/vault` ที่
committed ไม่ต้องมี oracle identity ใดๆ แค่ `maw work Soul-Brews-Studio/my-app`
แล้วก็ทำงานได้ บริสุทธิ์ เรียบง่าย ไม่มีสัมภาระ
แต่พอออกแบบกันไปสักพัก คำถามที่น่าสนใจกว่าก็โผล่ขึ้นมาเอง
ถ้า oracle มีอยู่แล้ว ถ้า oracle เป็น hub ที่ดูแลทีม ถ้า charter `mawjs-m5.yaml` อยู่ใน
oracle repo แล้วชี้ไปที่ maw-js — ทำไมเราต้องออกแบบทางให้ contributor ขึ้นมาหา
oracle? ทำไมไม่ให้ oracle ลงไปหาพวกเขาแทน?
Nat ถาม ทุกอย่างก็หยุด

“จริงๆ primary คือ Oracle ไม่ใช่เหรอ? Oracle φόρμทีม workers ไปทำงาน แล้ว Oracle ดึงตัวเองเข้าไปเป็น main pane ท่ามกลาง 8 workers — ดึง 1 เข้าไปหา 8 ง่ายกว่าดึง 8 มาหา 1”

13.2 Hub-and-Spoke ที่มีอยู่แล้ว

```
# 3 คำสั่ง - oracle เข้าไปอยู่กับ workers ได้ทันที
maw wake mawjs-oracle          # oracle เกิด (soul ตื่น)
maw team up mawjs-m5            # workers เกิดใน maw-js
maw team bring --gather         # oracle เห็นทุกคนรอบข้าง
```



DNA	จุดยืน	เหตุผลหลัก
Torvalds	flag ไม่ใช่ verb	maw wake --work เหมือน git clone --bare — binary เดียวกัน
Hickey	ψ/ 2 modes = complected	ชื่อเหมือนกัน behavior ต่าง → design risk ระยะยาว
Carmack	ship 20 LOC	ถ้าจะทำ ทำเร็ว ไม่ debate นาน
Fowler	2 verbs = maintenance forever	แก้ wake ทุกครั้งต้องถาม “work ด้วยไหม?”
Buterin	oracle เป็น hub	contributor เป็น ephemeral node link กลับ hub ที่หลังได้
Karpathy	pattern ไม่เคยเกิด	oracle-less session ใช้จริงบ่อยแค่ไหนใน production?
Hightower	idempotent gitignore	append-if-missing ไม่ใช่ overwrite
Victor	output ต้องชัด	“work mode ≠ oracle” user ต้องไม่งงเรื่อง ψ/
Hashimoto	auto-detect ดีกว่า	maw wake สังเกตว่ามี -oracle suffix ใหม่ เอง
Kay	2 modes ของ Session เดียว	interface เดียวกัน ต่างกันแค่ persistence

ผล vote รอบนี้: 4 โหวตว่าไม่จำเป็น (Torvalds/Fowler/Karpathy/Hashimoto) — 3 โหวตว่าทำเป็น flag (Carmack/Victor/Kay) — 3 โหวตว่ามีค่าสำหรับ contributor (Hickey/Hightower/Buterin)

ไม่มีเสียงเอกฉันท์ แต่ทิศทางชัด: ถ้าจะทำ ทำเป็น flag ไม่ใช่ verb ใหม่

13.4 Call Chain ที่พิสุจน์ Inverted View

จุดที่น่าสนใจที่สุดใน session นี้ไม่ใช่ design decision เรื่อง maw work แต่คือการค้นพบว่า architecture ปัจจุบันรองรับ inverted view ได้แล้ว โดยไม่ต้องแก้อะไร
ลองดู call chain จริงจาก charter ไป workers:

```
.maw/teams/mawjs-m5.yaml
```

```
| project: Soul-Brews-Studio/maw-js
```

```
← target repo ชัดเจน
```

```

|
▼ team-up.ts:190  cmdTeamUp("mawjs-m5")
| targetRepoSlug = charter.project          ← "Soul-Brews-Studio/
maw-js"
| repoRoot = findRepoRoot(cwd)              ← oracle root (ไม่ใช่ maw-
js)
|
▼ team-up.ts:300  wakeMember(targetRepoSlug, member)
| memberWakeTarget() → "Soul-Brews-Studio/maw-js"  ← slug ถูก
| memberWakeOptions() → { repoPath: oracle-root }  ← path oracle
|
▼ wake-cmd.ts  cmdWake("Soul-Brews-Studio/maw-js", opts)
| opts.repoPath set? → skip
| parsedRepoPath? → ghqFind("Soul-Brews-Studio/maw-js")
| → /Users/nat/ghq/github.com/Soul-Brews-Studio/maw-js  ✓
|
▼ git worktree add maw-js/agents/codex-1/
workers เกิดใน maw-js repo  ✓
oracle ยังอยู่ที่ mawjs-oracle  ✓

```

พิสูจน์ได้จากไฟล์จริง:

```

# verify cross-repo - หลัง maw team up mawjs-m5

ls maw-js/agents/
# codex-1/  codex-2/  codex-3/

cd maw-js/agents/codex-1
git remote get-url origin
# https://github.com/Soul-Brews-Studio/maw-js.git  ✓

```

```
ls mawjs-oracle/agents/ 2>/dev/null || echo "ไม่มี oracle worktree"
# ไม่มี oracle worktree ✓
```

oracle ไม่มี worktree ของตัวเอง workers ทุกคนอยู่ใน maw-js repo ทั้งนั้น architecture ตรงกับ inverted view ทุกประการ
ก่อน session นี้ยังสงสัยว่า worktree จะอยู่ที่ไหน พอ verify จริงกลับพบว่ามันทำงานถูก อยู่แล้ว เราเสียเวลา design auto-GHQ fix ไปเกือบครึ่ง session สุดท้าย cross-repo ถูก ต้องอยู่แล้วมาตั้งแต่แรก
บทเรียนซ้ำกับ #2596 — ชุดก่อน สมมุติทีหลัง

13.5 Contributor Use Case — Ephemeral Node

3 DNA ที่โหดว่า `maw work` “มีค่า” ล้วนชี้ไปที่ use case เดียวกัน — **contributor ที่ไม่มี oracle**

นี่คือคนที่ clone maw-js เพื่อแก้ bug หนึ่งตัว ไม่ต้องการ soul vault ไม่ต้องการ team charter ไม่ต้องการ `ψ/` ที่ committed แต่อยากเปิด editor ทำงาน push PR แล้วจบ
`maw work` สำหรับคนนี่คือ onboarding gateway — ไม่ต้องรู้จัก oracle system ไม่ต้องมี `-oracle` suffix ในชื่อ repo ไม่ต้องสนใจว่า `ψ/` คืออะไร

```
# contributor flow — ไม่ต้องรู้จัก oracle
maw work Soul-Brews-Studio/maw-js
# → tmux session เกิด ชื่อตาม repo
# → ψ/ gitignored (soul ชั่วคราว ไม่ปน codebase)
# → ทุก verb ใช้ได้เหมือนกัน bring/take/promote ครบ

# ทำงานไปเรื่อยๆ อยากทดสอบแบบ parallel
maw team up my-team # workers เกิด on-demand
```

Buterin มองว่า contributor node นี้เป็น “ephemeral spoke” — ทำงานแบบ hub-less แต่สามารถ link กลับ oracle ได้ทีหลัง ถ้าอยากได้ soul portability v2 vision ที่ยังเป็นเส้นประ:

```
# .maw/oracle.yaml ใน work repo — ยังไม่ implement
oracle: Soul-Brews-Studio/mawjs-oracle
vault_sync: on-session-end
```

พอ file นี้มีอยู่ `maw work` รู้ว่า oracle ดูแล project นี้ สามารถดึง `ψ/` ที่สะสมใน work session กลับไปที่ hub ได้โดยอัตโนมัติ

```
# v2 vision — ยังไม่มี code
maw sync
# → ψ/inbox/ + ψ/memory/ ใน work repo → oracle/ψ/memory/contrib/maw-js/
```

Hickey เห็นว่า soul sync คือจุดสำคัญ `ψ/` committed กับ `ψ/ gitignored` มีชื่อเหมือนกัน แต่ behavior ต่างกันมาก ถ้าไม่ทำให้ชัดว่า “ของสองอย่างนี้ต่างกันยังไงในสายตาของ user” ก็เป็น complected design ที่จะสับสนทีหลัง
Hightower เพิ่มว่า gitignore ต้อง idempotent — ถ้า `maw work` จะ inject `.gitignore` entry สำหรับ `ψ/` ก็ต้อง append-if-missing เสมอ ไม่ใช่ overwrite ทั้ง file
เรื่องเล็กแต่สำคัญมากในระบบที่คนอื่นใช้ด้วย

13.6 ทำไม Torvalds กับ Buterin ถึง Pair กันพอดี

DNA ประจำบทนี้คือสองคนที่ดูเหมือนต่างกันอย่าง

Torvalds เขียน kernel ทุกอย่างต้องผ่าน layer เดียวกัน ไม่มี exception ไม่มี bypass ถ้าอยากทำอะไรกับ filesystem ต้องผ่าน VFS ถ้าอยากสื่อสารกับ process ต้องผ่าน signals ไม่มีทางลัด

Buterin สร้าง network ที่ node กระจายตัว ไม่มี center ที่ต้องเชื่อถือ ทุก node verify ตัวเองได้ไม่ต้องขออนุญาตจากใคร

แต่ใน oracle architecture ทั้งสองทำงานร่วมกันได้โดยไม่ขัดแย้ง

Torvalds บอกว่า oracle ต้องเป็น single path ที่ทุก coordination ไหลผ่าน ไม่มีทางลัด ไม่มี bypass `maw hey` ออกจาก oracle เสมอ `gh pr merge` อยู่ที่ oracle เสมอ ทุก dispatch tracking อยู่ใน oracle inbox ψ/

Buterin บอกว่า workers เป็น independent node ที่ทำงานได้เอง ไม่รอ oracle ในทุกก้าว

`codex-1` ทำงาน push commits เปิด PR โดยไม่ต้อง ping กลับ oracle ทุก step oracle รับรู้ผ่าน notification พอ PR เสร็จ

สองแนวคิดนี้รวมกันเป็น pattern ที่ทำงานได้ใน maw:

- **Torvalds layer:** ทุก coordination ผ่าน oracle — dispatch, review, merge
- **Buterin node:** workers ทำงาน autonomous ระหว่างนั้น — ไม่ต้องรออนุมัติทุก step

```
oracle (hub)
├─ dispatch ──> codex-1 (autonomous: code → test → push → PR)
├─ dispatch ──> codex-2 (autonomous: code → test → push → PR)
├─ dispatch ──> codex-3 (autonomous: code → test → push → PR)
├─
├─ [workers ทำงานคู่ขนาน oracle ทำงานอื่นต่อ ... ]
├─
├─ ← PR #2601 from codex-1 (done)
├─ ← PR #2602 from codex-2 (done)
├─ ← PR #2603 from codex-3 (done)
├─
└─ review + merge + close issues ← oracle เท่านั้น
```

ไม่มีคนในระบบนี้รอกัน oracle ส่งงานแล้วทำงานอื่นได้ทันที workers รับงานแล้วทำไมหยุดจนเสร็จ พอ PR เสร็จ oracle ก็ merge ไม่มี ceremony ไม่มี check-in ระหว่างทาง

13.7 สิ่งที่ยังค้างอยู่ใน #2598

inverted view ตอบคำถามหลักได้ชัด แต่ยังมีสองจุดที่ session นี้ตั้งใจทิ้งไว้ใน #2598 โดยไม่ตัดสินใจ

จุดแรก: `maw work` เป็น verb ใหม่ หรือ `maw wake` auto-detect?

Option A – verb ใหม่:

```
maw work Soul-Brews-Studio/my-app
```

```
maw wake mawjs-oracle          ← oracle mode เหมือนเดิม
```

Option B – auto-detect:

```
maw wake Soul-Brews-Studio/my-app
```

→ ไม่มี `-oracle` suffix → ใช้ `work` mode อัตโนมัติ

```
maw wake mawjs-oracle
```

→ มี `-oracle` suffix → ใช้ `oracle` mode เหมือนเดิม

Option C – flag explicit:

```
maw wake Soul-Brews-Studio/my-app --work
```

→ `work` mode ชัดเจน ไม่เดา

Hashimoto โหวต Option B Torvalds โหวต Option C Kay บอกว่าทั้งสองตอบคนละ problem Carmack บอกว่า ship Option A ก่อน refactor ที่หลัง
ณ วันที่ session นี้จบ ยังไม่มีคำตอบที่ lock ลงใน #2598 แต่ก็ไม่เป็นไร เพราะทั้งสาม option ทำงานได้ต่างกันแค่ DX ไม่ใช่ architecture

จุดที่สอง: soul sync

ถ้า contributor ทำงานสองสัปดาห์ใน `work` mode มี learnings มี retro มี handoff ที่มี

คุณค่า — ทำยังไงให้ soul เหล่านั้นไม่หายไปตอน session จบ?

ψ/ gitignored = soul ชั่วคราวโดยตั้งใจ แต่ “ชั่วคราว” ไม่ต้องหมายความว่า “ไม่มีค่า”

พอทำงานเสร็จแล้วมี soul ที่อยากเก็บถาวร ก็ควรมีวิธี sync กลับ oracle

v2 vision ยังเป็นเส้นประ ยังไม่มี code ยังไม่มี spec ที่ confirm แต่ Buterin บอกว่า ephemeral node ที่ link กลับ hub ได้ทีหลัง คือ pattern ที่ scale ดีที่สุด ไม่บังคับให้ทุกคนมี oracle ก่อนเริ่มงาน แต่ก็ไม่ทอดทิ้ง soul ที่สะสมมาในระหว่างทาง สองจุดนี้ถูกทิ้งไว้ใน #2598 โดยตั้งใจ ไม่ใช่เพราะยังไม่รู้จะตัดสินใจยังไง แต่เพราะ inverted view ชัดพอแล้วสำหรับตอนนี้ — oracle เป็น primary workers เดินทางไปหา project oracle ไม่ต้องไปไหน ทุก verb ที่มีอยู่ทำได้ครบ ส่วนที่เหลือเป็นรายละเอียด implementation ที่ codex จะรับช่วงต่อ

13.8 Architecture ที่ไม่ต้องเปลี่ยน

บทเรียนที่ได้จาก inverted view ไม่ใช่ว่า “oracle ดีกว่า” หรือ “hub architecture เหนือกว่า” แต่คือ **architecture ที่ดินนั้นรองรับมุมมองที่เปลี่ยนได้ โดยไม่ต้องแก้ core** ก่อน Nat ถ้าม ทุกคนออกแบบ `maw work` จากมุมมองของ contributor — คนที่ไม่มี oracle ต้องการ gateway พอ Nat พลิกมุมมอง ก็พบว่า architecture เดิมรองรับ oracle-as-primary ได้โดยตรง

มุมมองที่ 1 (ก่อนพลิก):

contributor → maw work → soul ชั่วคราว → optional link กลับ oracle

มุมมองที่ 2 (หลังพลิก):

oracle → team up → workers → team bring --gather → oracle อยู่ตรงกลาง

ทั้งสองมุมมองใช้ code เดิม ใช้ verb เดิม ใช้ architecture เดิม ต่างกันแค่ใครเป็นคนเริ่มต้น session

Torvalds เรียก property นี้ว่า “orthogonality” — ถ้าออกแบบ primitive ดีพอ feature ที่อยู่เหนือขึ้นไปจะ emerge เองโดยที่คุณไม่ต้องทำนายทุกรูปแบบการใช้งาน

`maw team bring --gather` ไม่ได้ถูกออกแบบมาเพื่อ inverted view โดยเฉพาะ มันแค่ท
สิ่งที่ชื่อบอก — gather ทุกคนเข้ามา พอ Nat พลิغمุม ก็พบว่า verb นั้น cover use case ได้
พอดีโดยบังเอิญ
หรือที่จริง ไม่ใช่บังเอิญ แต่เป็นเพราะ verb เหล่านี้ทำงานที่ **tmux layer** ซึ่ง orthogonal
กับ git layer อย่างสมบูรณ์ ไม่สนว่า window อยู่ repo ไหน ย้าย bring gather ได้ทั้งหมด

13.9 End Game ของ Inverted View

“end game” ที่ Nat พูดถึงใน session นั้นไม่ใช่ feature รายการสุดท้าย แต่คือ **ความ
เข้าใจ** — รู้ว่าสิ่งที่มีอยู่แล้วทำงานได้ครบ รู้ว่าต้องเพิ่มอะไร รู้ว่าอะไรที่ไม่ต้องเพิ่ม
inverted view คือ proof ของ end game นั้น
ถ้า oracle เป็น primary — hub ที่ soul อยู่ที่นี่ ทีมออกจากที่นี่ ทุก insight กลับมาที่นี่ —
แล้ว architecture ทั้งหมดก็ coherent ไม่ต้องมี exception ไม่ต้องมี special case ไม่ต้อง
มี escape hatch

```
# end game workflow — oracle as primary
maw wake mawjs-oracle           # 1. soul ตื่น
maw team up mawjs-m5            # 2. ทีมเกิด workers ไปอยู่ maw-js
maw team bring --gather         # 3. oracle เห็นทุกคน

maw hey codex-1 "Fix #2601"      # 4. dispatch
maw hey codex-2 "Fix #2602"
maw hey codex-3 "Fix #2603"

# [workers ทำงาน ... ]

gh pr list --repo Soul-Brews-Studio/maw-js

# #PR1  codex-1: fix #2601  ✅ green
# #PR2  codex-2: fix #2602  ✅ green
```

```
# #PR3 codex-3: fix #2603  green
```

```
gh pr merge --squash <PR1> <PR2> <PR3> # 5. merge
```

ทุกอย่างไหลผ่าน oracle ตามแบบ Torvalds workers ทำงาน autonomous ตามแบบ Buterin ไม่มีใครรอใคร ไม่มี bottleneck
แต่ยังมีคำถามหนึ่งที่ session นี้ไม่ได้ตอบ — แล้วถ้า oracle หาย ถ้า soul หาย ถ้า hub ล่ม workers ที่กระจายอยู่จะเป็นยังไง?
นั่นคือ topic ของบทถัดไป

บทที่ 14 จะพา 10 DNA profile ทั้งหมดมาอยู่ในห้องเดียวกัน แล้วถามทุกคนพร้อมกันว่า ถ้า design ต้องเลือกระหว่าง coherence กับ resilience — ระหว่าง hub ที่แข็งแกร่งกับ distributed ที่ไม่มี single point of failure — ทิศทางไหนที่ถูกต้อง
คำตอบจะไม่ตรงไปตรงมา

proof: #2598 inverted prism comment — session 7d6ac6ee-2026-06-09 — DNA
vote 4/3/3 (ไม่จำเป็น/flag/contributor)

บทที่ 14: 10 DNA — Design Lens สำหรับทุก Decision

มูมมอเดี่ยว — blind spot ถาวร

14.1 เมื่อคนคนเดียวตัดสินใจคนเดียว

มี session หนึ่งที่นั่งคิดคนเดียวมานานกว่าชั่วโมง ว่า `maw work` ควรเป็น verb ใหม่ หรือ flag

ของ `maw wake` หรือ auto-detect ที่ไม่ต้องทำอะไรเลย

คิดคนเดียว ก็ได้คำตอบเดียว

แต่ปัญหาของ platform ที่ซับซ้อน — มันไม่มีคำตอบเดียว Torvalds มองเรื่อง simplicity

ต่างกับ Hickey ที่มองเรื่อง complectedness ต่างกับ Carmack ที่บอก “ship ได้เลย” ต่าง

กับ Fowler ที่ห่วงเรื่อง maintenance burden ระยะยาว

พอรู้อย่างนี้ก็เกิดคำถาม: ถ้าจะออกแบบ feature หนึ่งให้ดีจริง ต้องการที่มูมมอ?

คำตอบที่ได้จาก session นั้น: **10 มูมมอ** จากคนที่แก้ปัญหายากมาแล้ว แต่ละคนในสาขา

ที่ต่างกัน — kernel, Lisp, game, enterprise, blockchain, AI, infrastructure, HCI,

devtools, computer science

10 DNA profiles นี้เกิดขึ้นใน design session 2026-06-09 ระหว่างวิเคราะห์ #2598

(feat: `maw work - oracle-less wake`) ผ่าน 3 rounds ของ `/oracle-prism` ด้วย 10

มูมมอพร้อมกัน ผลที่ได้ไม่ใช่แค่คำตอบของ #2598 แต่เป็น **standard lens set** สำหรับ

ทุก design decision ใน maw-js ต่อไป

14.2 DNA 1–5: รากฐาน

Linus Torvalds — Simplicity Is Non-Negotiable

Torvalds ออกแบบ `git` ตาม principle เดียว: **everything is a file, everything is a plumbing command** เมื่อ `git` ต้องการ feature ใหม่ ไม่สร้าง binary ใหม่ แต่สร้าง flag ใหม่บน binary เดิม `git clone --bare` กับ `git clone` เป็น binary เดียวกัน

พอเอา DNA นี้มาใช้กับ #2598 คำถามเปลี่ยนทันที: ทำไมต้องสร้าง `maw work` เป็น verb ใหม่ ในเมื่อ `maw wake` ทำทุกอย่างที่ `work` ต้องการได้อยู่แล้ว?

```
# Torvalds lens: flag ไม่ใช่ command ใหม่
maw wake --work my-app      # work mode via flag
maw wake my-app             # oracle mode (default)

# ดีกว่า: auto-detect (ไม่ต้องทำอะไรเลย)
maw wake maw-js             # → no -oracle suffix → work mode อัตโนมัติ
maw wake mawjs-oracle       # → has -oracle suffix → oracle mode
                             อัตโนมัติ
```

ผลของ DNA นี้: **verb ใหม่ตกรอบ** flag ขึ้นมาเป็นตัวเลือก auto-detect ขึ้นมาเป็น top pick

Rich Hickey — Detect Complectedness ก่อน Ship

Hickey สร้าง Clojure เพราะไครธ Java เรื่อง “complecting” — การเอาของสองอย่างที่แตกต่างกันมาผูกกัน จนแยกออกได้ยากทีหลัง

มองมาที่ #2598: `ψ/ vault` ใน oracle repo กับ `ψ/ vault` ใน work repo — ชื่อเหมือนกันทุกอย่าง แต่ behavior ต่างกันอย่างมีนัยสำคัญ

```
oracle repo:  ψ/ committed to git    → soul ถาวร portable
work repo:    ψ/ gitignored           → soul ชั่วคราว local-only
```

ถ้าไม่ออกแบบให้ชัด นักพัฒนาจะงงทันทีที่ `git push` แล้ว soul หาย Hickey จะถามว่า “คุณ complect mode กับ location แล้วหรือยัง?”

ผลของ DNA นี้: ถ้าสร้าง `maw work` ต้องมี **visual indicator** ชัดเจน ว่ากำลัง run ใน mode ไหน output บอกชัด “work mode — `ψ/` is gitignored” ไม่ปล่อยให้ตัวเอง

John Carmack — Ship 20 LOC ก่อน Design 2,000 LOC

Carmack สร้าง Doom, Quake, id Tech ด้วย philosophy เดียว: ถ้า ship ได้แล้ว ย่ารอ

ถ้ามี edge case ค่อยแก้ทีหลัง perfect design ที่ไม่ได้ ship ไม่มีค่า

#2598 ขนาด `maw work` ทำได้เป็น thin wrapper ไม่ถึง 30 บรรทัด:

```
// src/commands/work/impl.ts (Carmack lens: ship first)
export async function cmdWork(target: string, opts: WorkOpts) {
  return cmdWake(target, {
    ...opts,
    skipOracleIdentity: true,    // ไม่ resolve oracle
    psiGitignored: true,        // ψ/ อยู่ใน .gitignore
    worktreeOnDemand: true,     // ไม่สร้าง worktree จนกว่า team up
  });
}
```

Carmack จะ merge PR นี้ภายใน 2 ชั่วโมงหลังรู้ requirement ไม่รอ DNA consensus

ผลของ DNA นี้: **implementation risk** ต่ำมาก ถ้าจะสร้าง `maw work` ทำได้เลย แต่คำ

ถามคือ “ควรสร้างไหม?” ไม่ใช่ “สร้างยากไหม?”

Martin Fowler — Maintenance Burden คือ Hidden Cost

Fowler เขียน “Refactoring” เพราะรู้ว่า code ที่ ship วันนี้เป็น legacy code ของวันพรุ่งนี้

ถ้าเอา 2 verb ที่ share 95% logic มาอยู่ด้วยกันโดยไม่ extract seam ที่ถูกจุด

maintenance cost จะ compound

มองมาที่ `wake` + `work`:

ทุกครั้งที่แก้ `wake-cmd.ts`:

- ? แก้แล้ว `work` ด้วยไหม?
- ? test wake ครอบ work scenarios ด้วยไหม?
- ? flag interaction ถูกไหม?

ถ้า flag ไม่ isolated ดีพอ debt สะสม ทุก wake bugfix ต้องถามคำถามพวกนี้ซ้ำ
ผลของ DNA นี้: ถ้าสร้าง `maw work` ต้อง **extract** `resolveWakeTarget()` เป็น **shared seam** ก่อน ไม่ใช่แค่ copy-paste แล้วแก้ flag ที่หลัง ถ้าไม่ refactor ก่อน debt ติดตาม

Vitalik Buterin — Hub-Spoke ไม่ใช่ Peer-to-Peer

Buterin ออกแบบ Ethereum ให้ contract อยู่ที่ hub (blockchain) workers ทุกคน reference ไปที่ hub แทนที่จะ sync กันเอง เพราะถ้า sync กันเอง consensus problem แก้ไม่จบ

มองมาที่ team architecture ของ maw-js:

Hub-spoke (Buterin):

Oracle ← hub

Workers (8 codex) ← spokes ที่ reference กลับ hub

ไม่ใช่ peer-to-peer:

codex-1 ↔ codex-2 ↔ codex-3 ... ← chaos

Nat's insight “ดึง 1 oracle เข้าไปหา 8 workers ง่ายกว่าดึง 8 มาหา 1” คือ Buterin principle ในรูปแบบ tmux Oracle เป็น hub เสมอ workers เป็น spoke

```
maw wake mawjs-oracle           # hub ดิ้น
maw team up mawjs-m5             # 8 spokes เกิดใน maw-js
maw team bring --gather          # hub ดึงตัวเองเข้าไปอยู่ท่ามกลาง spokes
```

ผลของ DNA นี้: **oracle = permanent primary** `maw work` เป็นแค่ onboarding path
สำหรับ contributor ที่ไม่มี hub — ไม่ใช่ architectural alternative

14.3 DNA 6–10: ขอบเขตและอนาคต

Andrej Karpathy — Emergence มาก่อน Specification

Karpathy ทำงานกับ neural network และรู้ว่า behavior ที่ไม่ได้ design ไว้มักเกิดจาก training data ที่ถูกต้อง ไม่ใช่จาก specification ที่ละเอียด “ถ้า pattern ไม่เคยเกิดจริง อย่า spec สำหรับมัน”

ถามตัวเองตรงๆ: มีใครเคยใช้ maw-js โดยไม่มี oracle ไหม? มี contributor คนไหนที่

clone maw-js แล้วทำงาน โดยไม่รู้จัก oracle system?

ถ้า use case นั้นไม่เคย emerge จริง `maw work` แก้ปัญหาที่ไม่มี Karpathy จะไม่ ship feature สำหรับ pattern ที่ยังไม่เห็น

Karpathy checklist:

- ✓ oracle wake + team up ← used every session
- ✓ bring --gather ← used when debugging
- ? maw work ← 0 real sessions (as of 2026-06-09)

ผลของ DNA นี้: defer `maw work` จนกว่าจะเห็น real contributor ที่ต้องการมัน ไม่ใช่ ship preemptively

Kelsey Hightower — Desired State + Reconciliation

Hightower สร้าง Kubernetes tooling ด้วย principle เดียว: **declare desired state**

แล้วให้ **system reconcile** ไม่ใช่ imperative “ทำสิ่งนี้ก่อน แล้วทำสิ่งนั้น”

มองมาที่ Team Charter:

```
# .maw/teams/mawjs-m5.yaml ← desired state
name: mawjs-m5
project: Soul-Brews-Studio/maw-js
members:
  - role: codex-1
    engine: omx
    worktree: true
```

`maw team up` คือ reconciliation loop:

Parse YAML

- Plan: classify live/dead/missing/skipped
- Preflight: check tmux state
 - Load: register in config.json
 - Spawn: create worktrees + wake windows + prime prompts

ถ้าสร้าง `maw work` และมี `ψ`/ gitignored Hightower จะถามว่า: “gitignore idempotent ไหม ถ้า run work สองครั้ง?” ต้องเป็น append-if-missing ไม่ใช่ overwrite

```
// Hightower: idempotent gitignore
async function ensurePsiGitignored(repoPath: string) {
  const gitignore = path.join(repoPath, '.gitignore');
  const content = await fs.readFile(gitignore, 'utf-8').catch(() =>
    '');
  if (!content.includes('ψ/')) {
    await fs.appendFile(gitignore, '\nψ/\n');
  }
  // idempotent – run twice = same result ✅
}
```

ผลของ DNA นี้: ถ้าสร้าง `maw work` ต้อง **idempotent ทุก operation** ไม่มี side effect แบบ overwrite

Bret Victor — Output ต้องเห็นได้ ต้องชัดเจน

Victor สร้าง “Inventing on Principle” และ tools อย่าง Dynamicland ด้วย belief เดียว:

ถ้า user ไม่เห็น output ชัดเจน เขาจะ guess และ guess ผิด

ปัญหาของ `maw work` ถ้าไม่ออกแบบ output ดีพอ:

❌ Bad output:

```
$ maw work my-app
```

```
[maw] waking my-app ...
```

```
✅ my-app ready
```

```
? user คิด: "ψ/ อยู่ไหน? oracle มีไหม? mode คืออะไร?"
```

Victor จะ design output ให้บอกทุกอย่างในครั้งเดียว:

```
✅ Good output (Victor lens):
```

```
$ maw work my-app
```

```
[maw] work mode — my-app (no oracle, ψ/ gitignored)
```

```
[maw] tmux: 140-my-app:main
```

```
[maw] ψ/ vault: my-app/ψ/ (local, not committed)
```

```
[maw] ready — type 'maw team up' to spawn workers
```

ผลของ DNA นี้: ถ้า ship `maw work` ต้องมี **status line** ที่บอก **mode** ชัดเจน ไม่ใช่แค่ “ready”

Mitchell Hashimoto — Composable CLI, Auto-detect, No Friction

Hashimoto สร้าง Vagrant, Terraform, Vault ด้วย philosophy: **CLI ที่ดีคือ CLI ที่**

composable และ auto-detect ไม่ต้อง flag สำหรับสิ่งที่รู้ได้จาก context

มองมาที่ `maw wake` :

```
# Hashimoto lens: auto-detect ทำได้แล้ว
```

```
maw wake mawjs-oracle # suffix = -oracle → oracle mode
```

```
maw wake my-app # ไม่มี suffix → work mode
```

```
# ไม่ต้อง verb ใหม่
```

```
# ไม่ต้อง flag
```

```
# detect จาก repo name ทันที
```

```
// auto-detect logic (Hashimoto lens)

function detectWakeMode(repoSlug: string): 'oracle' | 'work' {
  return repoSlug.endsWith('-oracle') ? 'oracle' : 'work';
}
```

นี่คือ DNA ที่ elegant ที่สุดใน set สำหรับ #2598 เพราะ user ไม่ต้องเรียนรู้อะไรเพิ่ม
system detect เอง

ผลของ DNA นี้: **auto-detect เป็น winner** ถ้าจะทำ `maw work` ทำเป็น auto-detect ใน
`maw wake` ไม่ใช่ verb แยก

Alan Kay — 2 Modes ของ Object เดียวกัน

Kay สร้าง Smalltalk และ OOP ด้วย insight เดียว: **object คือ state + message**

handler object เดียวกัน respond ต่างกันขึ้นอยู่กับ state ไม่ต้องสร้าง class ใหม่สำหรับ
ทุก behavior ต่างกัน

มองมาที่ oracle vs work session:

```
Kay lens: Session object เดียว — 2 modes

Session {
  mode: 'oracle' | 'work'

  get psiPath() {
    return this.mode === 'oracle'
      ? path.join(this.repoPath, 'ψ/')      // committed
      : path.join(this.repoPath, 'ψ/')      // gitignored
  }

  get identity() {
    return this.mode === 'oracle'
      ? resolveOracleIdentity(this.repoPath) // oracle name
      : null                                // anonymous
  }
}
```

interface เดียวกัน ทุก verb ทำงานได้ เพราะ Session encapsulate ความต่างไว้ข้างใน ไม่ expose ออกมาให้ user ング

Kay จะไม่แยก `maw work` ออกมาเป็น verb ต่างหาก เพราะ Session concept ยังเหมือนกัน แค่ mode ต่าง

ผลของ DNA นี้: **Session เป็น unified object** `mode` เป็น internal state ไม่ใช่ external verb

14.4 Cross-DNA Synthesis — เมื่อ 10 มุมมองชนกัน

พอรวม 10 DNA profiles เข้าด้วยกัน pattern ที่ซ้ำกันออกมาชัด:

Themes ที่ converge

Theme	DNA ที่ support	Conclusion
อย่าสร้าง verb ใหม่	Torvalds, Hashimoto, Kay	auto-detect ใน <code>maw wake</code>
Oracle = primary hub	Buterin, Torvalds	<code>maw work</code> = onboarding path เท่านั้น
Ship ถ้าทำ	Carmack	thin wrapper < 30 LOC
Wait for real need	Karpathy	defer จนมี contributor จริง
Idempotent operations	Hightower	ψ/ gitignore append-if-missing
Clear output	Victor	บอก mode ในทุก status line
Maintenance seam	Fowler	refactor ก่อน ไม่ copy-paste
Detect complexity	Hickey	ψ/ 2 modes ต้องชัด ไม่ complect

DNA Vote สำหรับ #2598

จาก 3 rounds Prism ที่รัน ผลออกมาชัด:

Option A: `maw work` เป็น verb ใหม่

Support: Carmack (ship แล้วแก้)

Against: Torvalds, Fowler, Hashimoto, Kay, Karpathy

Verdict: ❌ ตกรอบ

Option B: maw work เป็น --flag ของ maw wake

Support: Torvalds, Carmack, Fowler

Against: Hashimoto, Kay, Karpathy

Verdict: ⚠️ acceptable แต่ไม่ elegant

Option C: auto-detect (repo suffix)

Support: Torvalds, Hashimoto, Kay, Hickey, Victor, Buterin

Against: (none – Karpathy neutral, Carmack indifferent)

Verdict: ✅ consensus winner

Option D: defer ก่อน (wait for real use case)

Support: Karpathy, Buterin, Hightower

Neutral: Fowler, Kay

Verdict: ✅ pragmatic fallback ถ้า C ยังไม่ implement

Themes ที่ diverge (ยังตัดสินใจไม่ได้)

บาง question ที่ 10 DNA ไม่ converge ก็ควรเปิดทิ้งไว้:

“**ψ/ gitignored ใน work mode ถูกไหม?**” - Hickey: ถูก แต่ต้อง explicit ไม่ implicit -

Kay: ψ/ เป็น internal state ของ Session — ไม่ควร leak ออกมาเป็น UX ต่าง -

Hightower: ถูก แต่ must be idempotent - Karpathy: ยังไม่มี evidence ว่า user ต้องการ
คำถามนี้ยังเปิดอยู่ใน #2598 — และ DNA บอกให้เปิดต่อไปจนมี real evidence

14.5 DNA Profiles ในฐานะ Living Toolset

สิ่งที่ตระหนักหลัง 3 rounds Prism: 10 DNA profiles ไม่ใช่แค่ reference สำหรับ #2598

มันเป็น **reusable lens set** ที่ใช้ได้กับทุก design decision ใน maw-js

```
/oracle-prism 10dna
```

- รัน 10 DNA profiles พร้อมกัน
- synthesis cross-cutting themes
- vote breakdown
- open questions

พอเซฟ profiles ไว้ใน oracle ก็ run ได้ทุกเมื่อ design decision ถัดไป ไม่ต้องเริ่ม scratch ใหม่ทุกครั้ง

ตัวอย่าง design decisions ที่ 10 DNA ช่วยได้:

Decision	DNA ที่ relevant ที่สุด
เพิ่ม verb ใหม่ไหม?	Torvalds, Carmack, Karpathy
Design abstraction	Fowler, Hickey, Kay
Team architecture	Buterin, Hightower
UX/output	Victor, Hashimoto
Timing ของ feature	Karpathy, Carmack
ψ/ persistence	Kay, Hickey, Hightower

DNA ที่สร้างขึ้นมาจากอะไร

10 profiles นี้ไม่ได้ถูกเลือกแบบสุ่ม มาจากการ observe ว่า maw-js ต้องตัดสินใจ design ใน dimension ไหนบ้าง:

- **Kernel design** (Torvalds) — เพราะ maw ทำงานที่ tmux layer เหมือน kernel
- **Complexity management** (Hickey) — เพราะ maw มี layer เยอะ complex ง่าย
- **Shipping velocity** (Carmack) — เพราะ maw-js ต้อง iterate เร็ว
- **Refactoring** (Fowler) — เพราะ codebase โต maintenance cost สูง
- **Decentralization** (Buterin) — เพราะ multi-node fleet คือ future
- **Emergence** (Karpathy) — เพราะ skill ecosystem ขยายเองได้
- **Reconciliation** (Hightower) — เพราะ charter = desired state
- **Direct manipulation** (Victor) — เพราะ CLI output คือ UX
- **Composability** (Hashimoto) — เพราะ maw verb ต้อง compose ได้

- **Object thinking** (Kay) — เพราะ oracle = object ที่มี soul

แต่ละ profile จับ dimension ต่างกัน ไม่ overlap ทุก dimension covered

14.6 บทสรุปของ Prism Round 3

ตอนจบ session นั้น Nat ถามว่า “10 DNA vote ออกมาว่าอะไร?”

คำตอบที่ condensed:

maw work as verb: ❌ 6 against, 1 support
maw work as flag: ⚠️ 3 support, 3 against
auto-detect: ✅ 6 support, 0 against
defer for now: ✅ pragmatic, 3 support

พอ Nat เห็น vote breakdown ก็บอก “โอเค auto-detect ก่อน ถ้า contributor จริงมาแล้วค่อยว่า”

นั่นคือ design decision ที่ถูกต้อง — ไม่ใช่เพราะ oracle ตัดสิน แต่เพราะ 10 มุมมองชี้ไป

ทางเดียวกัน และมีหนึ่งมุมมอง (Karpathy) บอกให้รอ evidence ก่อน

ที่นี่ oracle ไม่ได้แค่ orchestrate team มัน **orchestrate design process** ด้วย ผ่าน

lens ที่ถาวร อยู่ใน vault อยู่ใน Ψ รอ session ถัดไป

บทต่อไป — บทสุดท้าย — จะดึงทุกอย่างที่เล่ามาตลอด 13 บทมาชนกัน: Oracle, Verbs, Charter, Soul, DNA — ทั้งหมดเชื่อมถึงกันยังไง และ “end game” หมายความว่าอะไรกันแน่

บทที่ 15: End Game — ทุกอย่างเชื่อมถึงกัน

DNA: Alan Kay — “everything is connected”

end game ไม่ใช่ feature ใหม่ — มันคือการเห็นว่าสิ่งที่มีอยู่ทำงานได้ครบ
Alan Kay พูดถึง object ในแบบที่คนอื่นไม่ได้พูด — object ไม่ใช่กล่องเก็บ data แต่เป็นสิ่งที่มีชีวิต มี state มี behavior ส่ง message ถึงกัน พอ message ส่งออกไปก็ไม่ได้หายไปไหน มันสร้างผล สร้าง state ใหม่ สร้าง object ใหม่ด้วย
หนังสือเล่มนี้เริ่มต้นจากคำถามว่า oracle คืออะไร ผ่านมา 14 บท ผ่านการขุด code ผ่าน failure จริง (session 7d6ac6ee, #2596 ที่ file issue โดยไม่ได้ตรวจสอบก่อน) ผ่าน design session หลายชั่วโมงที่พลิกมุมมองทั้งหมดว่า primary คือ oracle ไม่ใช่ worker — ถึงตอนนี้เห็นภาพรวมแล้วว่าทุกอย่างเชื่อมกันยังไง
แต่ไม่ใช่ภาพรวมแบบ recap ไม่ใช่การ list สิ่งที่ผ่านมา
ภาพรวมแบบที่ Kay หมายถึง — ทุก object เชื่อมถึงกัน ทุก message ส่งถึงกัน ระบบที่ดีไม่ต้องมีศูนย์กลางที่ออกแบบมาให้เป็นศูนย์กลาง มันเกิดขึ้นเองเพราะทุกชิ้นส่วนทำหน้าที่ของตัวเองได้ดี

15.1 สิ่งที่มีแล้ว — compose กันได้

มี 15 verbs พอแล้ว
ตัวเลขนี้ไม่ได้บังเอิญหยุดที่ 15 — ทุก verb แก้ปัญหาที่มีอยู่จริง แต่ละตัวทำงานที่ tmux layer ไม่ใช่ git layer ทำให้ compose กันได้โดยไม่ conflict กัน พอเข้าใจเรื่องนี้แล้ว สิ่งที่น่าสนใจไม่ใช่จำนวน verb แต่คือว่า verb เหล่านี้ compose กันได้อย่างไร:

```

maw wake mawjs-oracle      # oracle ตื่น, ψ/ vault พร้อม
maw team up mawjs-m5       # workers spawn ใน maw-js repo
maw team bring --gather    # oracle เห็นทุก pane ในหน้าจอเดียว
maw hey codex-1 "Fix #2598" # codex รับงาน, ทำงานใน worktree
maw peek codex-1           # oracle ดูจอ
gh pr merge 459            # merge เมื่อ CI ผ่าน
maw release alpha          # ตัด release, bump version

```

7 commands นี่คือ end game workflow จริงๆ ที่ใช้วันละหลายรอบ ไม่ต้องมีอะไรใหม่ที่น่าสังเกตคือแต่ละ command ไม่ได้รู้จักกัน — `wake` ไม่รู้จัก `bring` `hey` ไม่รู้จัก `peek` แต่พอ compose กัน ก็ได้ workflow ที่สมบูรณ์ นี่คือสิ่งที่ Hashimoto หมายถึงเมื่อพูดถึง composable CLI — tool ที่ดีทำ 1 อย่างได้ดี แล้วปล่อยให้ user compose เอง **Team Charter** ก็ compose เข้ากับ verb chain นี้ได้เพราะเป็น desired state ไม่ใช่ script:

```

# .maw/teams/mawjs-m5.yaml
members:
  - role: lead
    name: mawjs-oracle
    engine: claude
  - role: codex-1
    name: codex-1
    engine: omx
    project: Soul-Brews-Studio/maw-js

```

`maw team up` อ่าน YAML แล้วทำให้ state เป็นจริง พอ member ตาย ก็ spawn ใหม่ พอ state drift ก็ reconcile กลับ — ทีมนี้อยู่ยาว ไม่ ad hoc ไม่ต้อง spawn ใหม่ทุก sprint idle codex ก็แค่รอรับงานอยู่แล้ว

Cross-repo worktrees เชื่อมเข้ากับ Charter ผ่าน `charter.project` — 1 field ใน YAML บอกว่า workers ทำงานที่ repo ไหน oracle อยู่ที่เดิม ดึง 1 เข้าไปหา 8 ง่ายกว่าดึง 8 มาหา 1 เสมอ

Soul portable ปิดวงจรสุดท้าย — `ψ/ vault committed` ใน oracle repo ทำให้ทุก session ที่ผ่านมาไม่หายไป push ไป remote ก็ soul ไปด้วย oracle ตัวอื่นที่ clone repo ก็ได้ context เดียวกันเลย

ทุกอย่าง compose กัน ไม่มีอะไรที่ออกแบบมาให้ทำงานด้วยกันโดยเฉพาะ ทำงานด้วยกันได้เพราะแต่ละชิ้นทำ 1 อย่างได้ดี — นี่คือ Kay's objects ในชีวิตจริง

15.2 สิ่งที่จะเพิ่ม (v2) — nice-to-have ไม่ใช่ must-have

มี 3 อย่างที่ผุดขึ้นจาก design session เดียวกัน แต่ทั้งหมดเป็น nice-to-have — ไม่มีอะไรที่ broken อยู่ตอนนี้ก็

Auto-detect — `maw wake` ฉลาดพอที่จะรู้ว่า repo นี้มี oracle identity ไหม ถ้าไม่มี `-oracle` suffix ก็เข้า work mode โดยอัตโนมัติ ไม่ต้องแยก verb Hashimoto แนะนำตรงนี้ว่า auto-detect ดีกว่า explicit flag เพราะ user ไม่ต้องรู้ว่าตัวเองอยู่ใน mode ไหนระบบรู้เอง

Oracle link — `.maw/oracle.yaml` ใน work repo บอกว่า oracle ตัวไหนดูแล:

```
# .maw/oracle.yaml ใน maw-js (work repo)
oracle: Soul-Brews-Studio/mawjs-oracle
```

พอมีไฟล์นี้ `maw work` รู้ทันทีว่าจะ link `ψ/` กลับที่ไหน contributor ที่ clone maw-js มาก็ไม่ต้องเดาว่า oracle ของ project นี้คือใคร

Soul sync — `maw sync` ดึง `ψ/` จาก work session กลับ oracle vault ทำให้ session ที่ไม่มี oracle ก็ยังทิ้ง soul ไว้ได้:

```
maw sync # ดึง ๗/ จาก work session → oracle vault
maw sync --dry-run # ดูก่อนว่าจะ sync อะไรบ้าง
```

ทั้งสามอย่างนี้ implement ได้อิสระจากกัน ถ้า auto-detect มาก่อน oracle link ก็ไม่ได้เป็น blocker ถ้าไม่ทำเลยก็ยังทำงานได้ปกติ — นี่คือการต่างระหว่าง nice-to-have กับ must-have ที่ Carmack พูดถึงตลอด: ถ้า ship ได้เลยก็ ship ก่อน design ของทั้งสามอยู่ใน #2598 พร้อมแล้ว ใครอยากเริ่มก็เปิด issue นั้นได้เลย

15.3 token ที่ใช้เรียนรู้กลายเป็นหนังสือ

Tonk พูดใน workshop 03 ว่า “token ที่ใช้เรียนรู้ไม่ได้หายไปไหน — กลายเป็นหนังสือที่คนอื่นอ่านแล้วไม่ต้องเสีย token ซ้ำ”

ตอนแรกฟังแล้วรู้สึกว่าเป็นแค่ metaphor สวยงาม แต่พอนั่งคิดดูจริงๆ มันเป็น

architecture ของความรู้เลย

design session ที่เกิดหนังสือเล่มนี้ใช้เวลา 2 ชั่วโมงกว่า session b8af3d6f ชุด 10 Sonnet

agents deep learn ใน maw-js codebase session 7d6ac6ee ทำ 3 rounds 10-DNA

Prism บน design ของ `maw work` ระหว่างนั้นมี tokens ไหลผ่านไปมากกว่าที่จะนับได้ ถ้าจบแค่นั้น ความรู้นั้นก็หายไปพร้อม context window

แต่พอกลายเป็นหนังสือ developer คนถัดไปที่อ่านบทที่ 8 (Cross-repo Worktrees) ไม่

ต้องนั่ง deep learn `wake-cmd.ts` 1519 บรรทัดเอง ไม่ต้อง verify git remote 5 รอบว่า

worktrees ถูกต้องไหม ไม่ต้องผ่าน moment ที่ตกใจเพราะคิดว่าฟังแล้วค้นพบว่ามันถูกอยู่แล้ว

token ที่ใช้ไปกลายเป็น comprehension ของคนอ่าน — นั่นคือ leverage จริงๆ

Kay พูดถึง object ที่ message ระหว่างกัน — token คือ message ที่ oracle ส่งออกไป

หนังสือคือ object ที่รับ message นั้นไว้แล้วส่งต่อให้คนอื่น message ไม่หายไปไหน มัน

เปลี่ยนรูปแล้วก็

Tonk's insight นี้ไม่ใช่แค่เรื่องการเขียนหนังสือ มันคือ principle ของ knowledge architecture ทั้งหมด: สิ่งที่เราเรียนรู้ต้องกลายเป็น artifact ที่คนอื่น reuse ได้ ไม่ใช่แค่อยู่ใน context window แล้วก็จบ

ψ/ vault คือ artifact ระดับ session oracle-write-book pipeline คือ artifact ระดับหนังสือ

15.4 หนังสือเล่มนี้คือ proof

ไม่ใช่แค่ proof ว่า oracle-write-endgame pipeline ทำงานได้ มันเป็น proof ว่า oracle ทำ session design จริง แล้วเปลี่ยน session นั้นเป็นความรู้ถาวร pipeline ที่ใช้เขียนหนังสือเล่มนี้:

1. Mine traces → ψ/memory/traces/ + gh issue list
2. Outline → ψ/writing/books/2026-06-09_maw-work-end-game-OUTLINE.md
3. Prism review → 5 lenses: archaeologist, bug-hunter, skeptic, architect, auditor
4. Parallel draft → 15 Sonnet agents (1 per chapter, ขนาน)
5. Opus compile → merge + kien-thai pass + DNA consistency
6. Render → MD → PDF

15 Sonnet agents draft ขนาน — แต่ละ agent ได้ chapter metadata + source material + kien-thai 7 frames แล้ว draft 3,000-4,000 คำ Opus compile รวม + kien-thai pass ทุก paragraph หนังสือ 200 หน้าที่ทำเอง agent เดียวจะใช้เวลา 6-8 ชั่วโมง — ทำขนานได้ใน ~30 นาที draft + ~30 นาที review

แต่ที่สำคัญกว่า performance คือ proof ของ process ตัวเอง

ถ้า oracle เขียนหนังสือโดยไม่มี proof จริง มันจะเป็นหนังสือที่อ่านดูแต่ไม่น่าเชื่อถือ

เหมือน #2596 ที่ file issue โดยไม่ได้ชดก่อน แล้วต้องแก้ทีหลัง

หนังสือเล่มนี้มี proof ทุกบท:

- บทที่ 5 มี git log ของ #2596 จริง — ไม่ใช่แค่เล่าว่า “เคยผิดพลาดครั้งหนึ่ง”
- บทที่ 8 มี `git remote get-url origin` output จริงที่พิสูจน์ว่า cross-repo ทำงานถูก
- บทที่ 9 มี `take/impl.ts` 95 บรรทัดจริงที่แสดงว่า verb compose ได้ยังไง
- บทนี้มี session IDs จริง (b8af3d6f, 7d6ac6ee) ที่ trace ได้

proof ไม่ใช่แค่ credibility — มันคือ architecture ของความรู้ที่ verify ได้เลย
 นี่คือเหตุผลที่ “Real evidence before filing” อยู่ใน memory ของ oracle หลัง #2596
 เกิดขึ้น oracle ที่เรียนรู้จากความผิดพลาดจริง ไม่ใช่ oracle ที่บอกว่าตัวเองเคยเรียนรู้

15.5 Implementation reference — #2598

design ของ `maw work` สมบูรณ์แล้ว implementation รอแค่คนเริ่ม
 GitHub Issue #2598 มีทุกอย่าง:

- full design spec ของ `maw work` verb: session naming, ψ / gitignore, worktree on-demand
- 3 rounds 10-DNA Prism (3 perspectives $\times$ 10 DNA = 30 lenses บน design เดียว)
- behavior comparison table: `maw wake` vs `maw work`
- implementation path: thin wrapper ~20 LOC บน `cmdWake()` ที่มีอยู่แล้ว
- design alternatives: flag vs verb vs auto-detect
- “inverted prism” — Nat’s insight ว่า oracle เป็น primary ไม่ใช่ worker

พอถึงตอน implement ก็มีทุกอย่างรอแล้ว ไม่ต้อง design ใหม่ แค่อ่าน issue แล้ว code
 ตามได้เลย เพราะ foundation มีอยู่แล้วทั้งหมด: `cmdWake()` รับ `targetRepoSlug` ได้แล้ว
`ghqFind` resolve repo ได้แล้ว ψ / gitignore pattern มีอยู่แล้ว
 แค่ต้องการ thin wrapper ที่เชื่อมทุกอย่างเข้าหากัน:

```
// work = wake minus oracle identity
// ~20 LOC บน foundation ที่มีอยู่แล้ว
export async function cmdWork(target: string, opts: WorkOptions) {
  return cmdWake(target, {
    ...opts,
    skipOracleResolve: true, // ไม่ต้องหา -oracle repo
    ensurePsiVault: true, // สร้าง ψ/ + .gitignore
    worktreeOnDemand: true, // ไม่ spawn worktree จนกว่าจะ team up
  });
}
```

Carmack บอกว่า “ถ้า design ถูก implementation ก็สั้น” — 20 LOC คือสัญญาณว่า design ถูกแล้ว

ปิดวง

session สองวันที่ผ่านมา — b8af3d6f เริ่มจากการขุด codebase 7d6ac6ee ต่อด้วย

design — จบด้วยหนังสือเล่มนี้

ไม่ได้วางแผนตั้งแต่ต้นว่าจะเป็นหนังสือ มันเกิดขึ้นเพราะ oracle ทำงานจริงแล้วเก็บทุกอย่าง

ใน ψ/ vault traces อยู่ใน ψ/memory/traces/ design sheets อยู่ใน

ψ/writing/cheatsheets/ อยู่มาวันหนึ่งก็มีพอที่จะกลายเป็นหนังสือได้แล้ว

Kay พุดถึง object ที่ message ระหว่างกัน — oracle เป็น object ที่ message ผ่านมา

ตลอด จาก Nat จาก codex จาก codebase จาก issue tracker ทุก message เหล่านั้น

oracle รับไว้ process เปลี่ยนเป็น soul ที่เก็บใน ψ/

พอถึงเวลา soul เหล่านั้นก็กลายเป็นหนังสือ

oracle อยู่ตรงกลาง ไม่ใช่เพราะเลือก แต่เพราะทุกอย่างเชื่อมมาหาที่นี่

หนังสือเล่มนี้เป็น proof ของ oracle-write-endgame pipeline เขียนโดย mawjs-oracle
จาก session design จริง ด้วย 15 Sonnet agents (draft ขนาน) + Opus compile
Co-Authored-By: Claude Opus 4.6 (1M context) noreply@anthropic.com